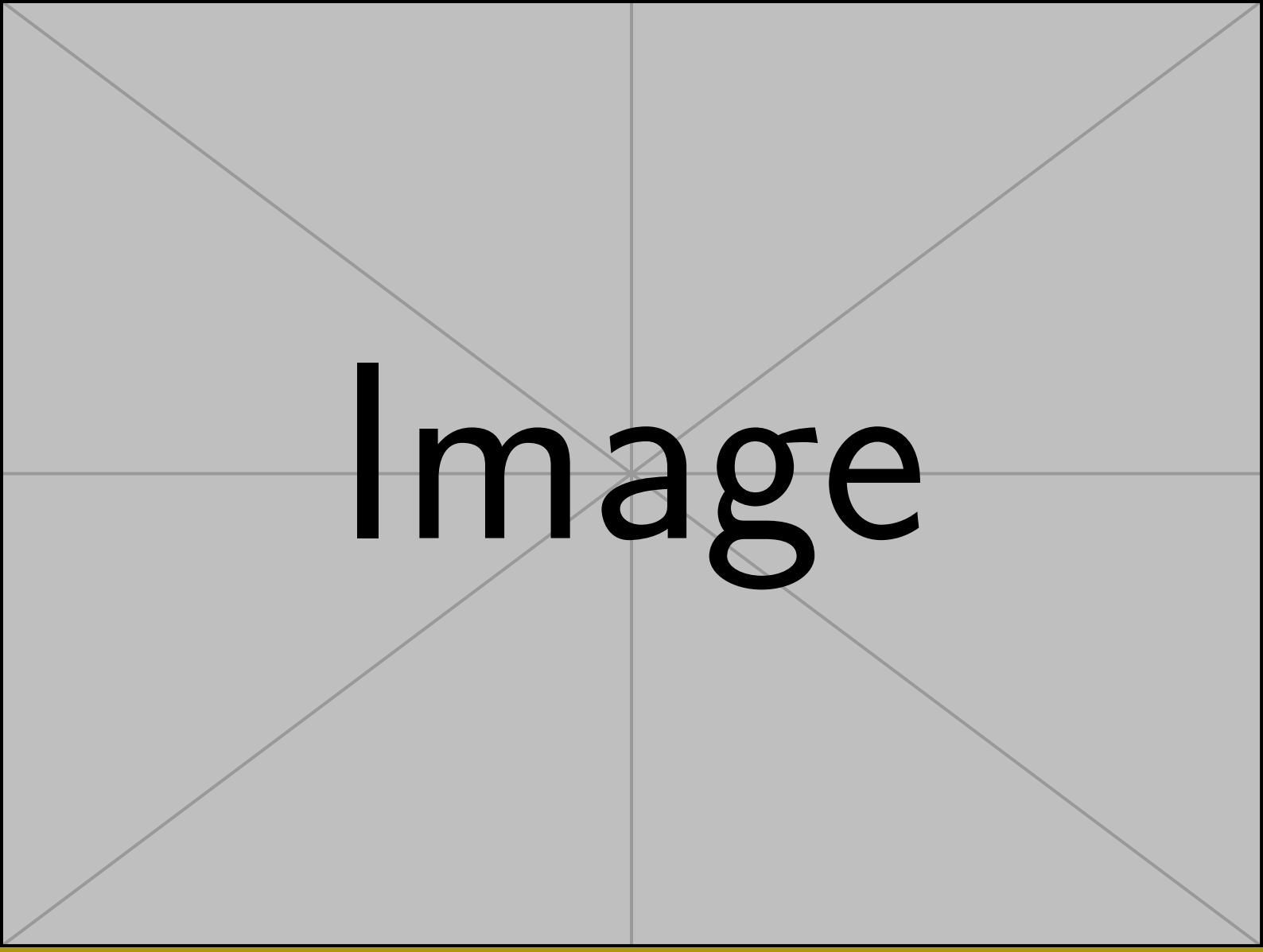


miniOS 技术文档

从进程调用链到 WiFi 与 IPv4 协议栈 —— T113-S3 裸机实现

作者：Jiming Kang

组织：rpi_bm / miniOS 项目

时间：2026 年 10 月

版本：0.1

源码：rpi-bm（miniOS T113 / AArch64 学习内核）

进程调用链、根文件系统与 devfs、/boot FAT32、UART 控制台、XR829 WiFi 驱动、IPv4 协议栈、SMHC 主机控制器、VFS 对象模型、sysfs 与 GPIO 按键。

目录

第一章 进程创建与切换调用链 **1**

1.1 总调用链 . . . 1
1.2 第一个用户进程 . . . 2
1.3 第二个用户进程 . . . 2
1.4 系统调用调用链 . . . 2
1.5 定时器中断和进程切换 . . . 3
1.6 两类上下文的职责 . . . 3
1.7 TTBR0 与 TTBR1 . . . 3
1.8 PSTATE 注意事项 . . . 4
1.9 T113（ARMv7-A）：fork、vfork 与 clone . . . 4
1.9.1 三种创建方式的语义 . . . 4
1.9.2 统一实现：`t113_kernel_clone()` . . . 5
1.9.3 任务结构与内核栈布局 . . . 6
1.9.4 子进程第一次运行：`ret_from_fork` . . . 10
1.9.5 vfork：父进程怎样等、怎样被唤醒 . . . 10
1.9.6 地址空间：共享还是复制 . . . 11
1.9.7 exit 与 wait . . . 11
1.9.8 与 xv6 的进程创建对照 . . . 12
1.9.9 与 Linux 的对照 . . . 13
1.9.10 树莓派 AArch64 与 T113 ARMv7 的实现区别 . . . 13

第二章 根文件系统与 devfs **14**

2.1 SD 卡布局 . . . 14
2.2 总调用链 . . . 14
2.3 图解：内核怎样把根文件系统挂到 `/` . . . 15
2.3.1 挂载结果 . . . 15
2.3.2 启动时的挂载顺序 . . . 16
2.3.3 rootdev：分区到块设备 . . . 17
2.3.4 minifs 的磁盘布局和查找 . . . 17
2.3.5 devfs：`/dev` 下的字符设备 . . . 18
2.3.6 与 xv6、Linux 的挂载方式对照 . . . 18
2.3.7 T113 实现上的区别 . . . 18
2.4 VFS：super_block、inode、dentry . . . 19
2.5 rootdev：MBR 分区 2 块设备 . . . 19
2.6 minifs：xv6 磁盘 ABI 的只读实现 . . . 19
2.6.1 两种挂载方式 . . . 20
2.7 bootfs：`/boot` 的 FAT32 交接 . . . 21
2.8 devfs：`/dev` 设备文件 . . . 21
2.9 用户态视角：从 `/init` 到 shell . . . 22
2.10 与 Linux / xv6 的对照 . . . 22
2.11 构建与烧写 . . . 22
2.12 预期串口输出 . . . 23

第三章 /boot：从 bootpack 视图到可读写 FAT32 **24**
3.1 改动前：bootfs 只是 bootpack 的视图 . . . 24
3.1.1 加载流程 . . . 24
3.1.2 本质 . . . 24
3.2 改动后：分层结构 . . . 25
3.2.1 挂载顺序 . . . 25
3.3 bio：扇区缓存（bio.c） . . . 25
3.4 fat32（fat32.c） . . . 26
3.4.1 挂载 t113_fat32_mount_vfs() . . . 26
3.4.2 内存中的 inode 表 . . . 26
3.4.3 路径解析 . . . 27
3.4.4 读路径 . . . 27
3.4.5 写路径 . . . 27
3.4.6 创建 fat32_create()（open 带 O_CREAT） . . . 27
3.4.7 截断 fat32_truncate()（open 带 O_TRUNC） . . . 28
3.4.8 列目录 fat32_readdir() . . . 28
3.4.9 不支持 . . . 28
3.5 VFS 的配套改动 . . . 28
3.6 谁在用 /boot . . . 29
3.6.1 XR829 固件 . . . 29
3.6.2 tftp . . . 29
3.6.3 WiFi 配置 . . . 29
3.6.4 SPL 从 FAT32 加载内核 . . . 29
3.7 FAT32 图解（参照 xv6 book 第 8 章） . . . 29
3.7.1 分层 . . . 30
3.7.2 磁盘布局 . . . 30
3.7.3 FAT 表与簇链 . . . 31
3.7.4 目录项 . . . 32
3.7.5 内存中的数据结构 . . . 33
3.7.6 调用链：打开并读一个文件 . . . 34
3.7.7 调用链：创建并写一个文件 . . . 35
3.7.8 没有日志时的写入顺序 . . . 35
3.7.9 T113 实现上的区别 . . . 36
3.8 一致性与限制 . . . 36
3.9 改动前后对照 . . . 36
3.10 测试 . . . 36

第四章 UART3 控制台：TX 轮询、RX 中断与 /dev/console **38**
4.1 硬件与初始化 . . . 38
4.2 启动时序：early console 与驱动 probe 的分工 . . . 38
4.3 TX 路径：轮询 + 停转看门狗 . . . 39
4.4 RX 路径：中断如何拿到数据 . . . 39
4.4.1 使能 . . . 39
4.4.2 中断分派链 . . . 39
4.4.3 中断里做什么 . . . 40

4.4.4 阻塞读：锁内入队防丢唤醒 . . . 40
4.4.5 RX 看门狗：漏中断恢复 . . . 40
4.5 /dev/console 如何关联 . . . 40
4.5.1 注册：probe → devfs 节点 . . . 40
4.5.2 使用：fd 0/1/2 特判，不走 VFS . . . 41
4.5.3 全景数据流 . . . 41
4.6 双模式读的设计意图 . . . 42
4.7 调试设施 . . . 42
4.8 与 xv6 / Linux 对照 . . . 43

第五章 WiFi 驱动：XR829 / SDIO 枚举与固件下载 44
5.1 硬件连接 . . . 44
5.2 图解：硬件连接与上电时序 . . . 44
5.3 文件组成 . . . 46
5.4 分层结构 . . . 46
5.5 总调用链 . . . 48
5.6 初始化顺序与驱动核心约束 . . . 49
5.7 图解：SDIO 协议与初始化 . . . 49
5.7.1 命令与响应的格式 . . . 49
5.7.2 初始化状态机 . . . 50
5.8 SDIO 枚举 . . . 51
5.8.1 CMD52：单字节读写 . . . 51
5.8.2 CCCR / FBR / CIS . . . 51
5.9 CMD53 与数据通路 . . . 52
5.9.1 SMHC PIO 数据阶段 . . . 53
5.10 SMHC 主机控制器 . . . 53
5.11 HIF 寄存器与间接寻址 . . . 54
5.12 固件下载 . . . 55
5.12.1 镜像 . . . 55
5.12.2 流程 . . . 55
5.12.3 下载 FIFO 协议 . . . 56
5.12.4 进入 queue 模式 . . . 57
5.13 通信协议：从 SDIO 到 WSM . . . 57
5.13.1 协议分层 . . . 57
5.13.2 HIF 地址 . . . 57
5.13.3 WSM 消息格式 . . . 58
5.13.4 一次队列传输 . . . 59
5.13.5 流控：固件输入缓冲 . . . 59
5.13.6 命令和 confirm 的配对 . . . 60
5.13.7 启动握手 . . . 60
5.14 固件打包与加载（bootpack，现为后备路径） . . . 60
5.15 图解：数据流 . . . 61
5.15.1 发送：一个 UDP 包到空口 . . . 61
5.15.2 接收：从空口到 socket . . . 62
5.16 调用链：连接一个 WPA2 AP . . . 63

5.17 T113 实现与 Linux xradio 驱动的区别 . . . 64
5.18 预期串口输出 . . . 64
5.19 排障提示 . . . 65
5.20 当前边界与下一步 . . . 66

第六章 网络栈与 WiFi 驱动分层 67
6.1 分层总览 . . . 67
6.2 目录与 Linux 对照 . . . 68
6.3 接口边界 . . . 69
6.4 一个包怎么走 . . . 69
6.5 连接流程 . . . 69
6.6 执行上下文（没有中断时的规则） . . . 70
6.7 Kconfig . . . 70
6.8 加一块新网卡要做什么 . . . 70
6.9 从旧结构的对应关系 . . . 71
6.10 验证 . . . 71

第七章 驱动：模型、网络驱动分层与 WiFi / UART 实现 73
7.1 驱动架构：总线、设备与绑定 . . . 73
7.1.1 kobject/kset：内存里的目录树 . . . 73
7.1.2 一把锁规则与初始化顺序 . . . 73
7.1.3 平台设备：手写的设备树替身 . . . 74
7.2 网络驱动分层 . . . 75
7.2.1 从 ndo_ops 到 ieee80211_ops . . . 75
7.2.2 轮询顶替中断 . . . 75
7.2.3 ieee80211_ops：驱动与 mac80211 的契约 . . . 76
7.3 SDIO WiFi 初始化 . . . 76
7.3.1 卡枚举：CMD5 到绑定 . . . 76
7.3.2 probe 链：从 function 到 wlan0 . . . 77
7.4 数据接收与发送 . . . 77
7.4.1 HIF 队列与底半部 . . . 77
7.4.2 发送侧流控与接收排水 . . . 78
7.4.3 接收路径：从 CMD53 到 socket . . . 78
7.4.4 发送路径：从 write 到 CMD53 . . . 79
7.5 无线部分的发射 . . . 80
7.5.1 802.11 帧封装 . . . 80
7.5.2 速率与重试策略 . . . 81
7.5.3 EDCA 与队列映射 . . . 81
7.5.4 WSM TX 请求与 confirm . . . 81
7.5.5 EAPOL 发射特例 . . . 82
7.6 UART 串口驱动与 /dev/console . . . 82
7.7 与 Linux 的对照 . . . 83

第八章 SMHC 主机控制器：SD 卡与 SDIO WiFi 84
8.1 SMHC 芯片结构 . . . 84
8.1.1 三个实例 . . . 84

8.1.2 内部框图 . . . 84
8.1.3 寄存器 . . . 85
8.1.4 时钟树 . . . 85
8.2 驱动架构 . . . 86
8.2.1 两条软件栈 . . . 86
8.2.2 主机接口 mmc_host_ops . . . 87
8.2.3 注册与扫描 . . . 87
8.2.4 与 Linux sunxi-mmc 的对照 . . . 88
8.3 时序 . . . 88
8.3.1 命令与响应 . . . 88
8.3.2 改时钟 . . . 89
8.3.3 SD 卡初始化（SMHC0） . . . 89
8.3.4 单块读 CMD17 . . . 89
8.3.5 单块写 CMD24 . . . 89
8.3.6 SDIO 初始化与 CMD53（SMHC1） . . . 90
8.3.7 两路对照 . . . 90
8.4 和 xv6-aarch64 的区别 . . . 90
8.4.1 一次读盘的调用链 . . . 91
8.4.2 差异表 . . . 91
8.5 XR829 与树莓派 3 WiFi 的硬件区别 . . . 92
8.5.1 连接方式 . . . 92
8.5.2 对照表 . . . 93

第九章 VFS：super_block、inode 与 dentry **94**
9.1 核心对象 . . . 94
9.2 三张操作表 . . . 94
9.3 普通文件 . . . 95
9.4 字符设备 . . . 96
9.5 路径查找 . . . 96
9.6 挂载 . . . 97
9.7 与 Linux 的关系 . . . 97
9.8 vfs.h 当前保留的重要结构体 . . . 97
9.8.1 操作表 . . . 97
9.8.2 VFS 对象 . . . 98
9.8.3 引用和共享关系 . . . 99
9.9 chrdev.h 当前保留的内容 . . . 99
9.10 vfs.h 与 chrdev.h 的关系 . . . 100
9.11 从系统调用到对象的完整关系 . . . 100
9.12 sysfs：把 driver core 投影到 /sys . . . 101
9.13 GPIO 按键 . . . 102
9.14 已知限制（主机侧测试结果） . . . 102

第十章 sysfs 与 GPIO 按键驱动 **104**
10.1 sysfs . . . 104
10.1.1 挂载时机 . . . 104

10.1.2 目录结构 . . . 104
10.1.3 路径怎样对应到 driver core . . . 105
10.1.4 三个 VFS 操作 . . . 106
10.1.5 驱动怎样加属性 . . . 107
10.1.6 引用和锁 . . . 107
10.1.7 与 Linux sysfs 的差别 . . . 108
10.2 GPIO 按键驱动 . . . 108
10.2.1 硬件和平台数据 . . . 108
10.2.2 probe . . . 109
10.2.3 中断路径 . . . 109
10.2.4 阻塞读 . . . 110
10.2.5 sysfs 属性 . . . 110
10.2.6 用户态 . . . 110
10.2.7 与 Linux gpio-keys 的差别 . . . 111
10.3 测试和发现的问题 . . . 111

第一章 进程创建与切换调用链

本章对应当前 rpi_bm 中 AArch64（树莓派 3）学习内核的实现，说明通过内核启动函数创建用户进程、首次调度进入 EL0，以及后续异常往返的调用关系。本章最后一部分（第 1.9 节起）讲 T113（ARMv7-A）内核里 fork、vfork、clone 的统一实现，以及它和 xv6、Linux 的区别。后续各章转到 T113 上的 miniOS，讲文件系统、控制台与网络。

1.1 总调用链

```
kernel_main()
+-- copy_process(PF_KTHREAD, kernel_process, 0, 0)
    +-- allocate_kernel_page()                  分配 task_struct
    |   +-- task_struct.cpu_context             位于 task_struct 偏移 0
    +-- allocate_kernel_page()                  分配独立内核栈
    +-- task_pt_regs(p)
    |   +-- pt_regs                             固定放在内核栈高地址端
    +-- p->cpu_context.x19 = kernel_process     首次执行的内核函数
    +-- p->cpu_context.x20 = 0                  函数参数
    +-- p->cpu_context.pc = ret_from_fork       cpu_switch_to 的返回目标
    +-- p->cpu_context.sp = childregs           首次运行时的内核 SP

kernel_main()
+-- schedule()
    +-- _schedule()
        +-- switch_to(next)
            +-- switch_user_pgd(next->mm->pgd)  已有用户进程才切换 TTBR0_EL1
            +-- cpu_switch_to(prev, next)
                +-- 保存 prev 的 x19~x30 和 SP
                +-- 恢复 next 的 x19~x30 和 SP
                +-- ret
                    +-- ret_from_fork
                        +-- preempt_enable()
                        +-- x19 != 0
                        +-- blr x19
                            +-- kernel_process()
                                +-- launch_user_image(...)
                                    +-- allocate_kernel_page()          分配 mm_struct
                                    +-- allocate_pagetable_page()       分配用户 PGD
                                    +-- move_to_user_mode(...)
                                        +-- map_and_copy_user_code()    映射并复制用户代码
                                        +-- allocate_user_page()        映射用户栈
                                        +-- task_pt_regs(current)
                                        +-- regs->pc = 用户入口地址
                                        +-- regs->sp = 用户栈顶
                                        +-- regs->pstate = EL0t 状态
                                        +-- TTBR0_EL1 = current->mm->pgd

ret_from_fork
+-- kernel_process() 返回
    +-- ret_to_user                         位于 blr x19 后面
        +-- 从 pt_regs 读取 sp
        |   +-- SP_EL0 = pt_regs->sp
        +-- 从 pt_regs 读取 pc
        |   +-- ELR_EL1 = pt_regs->pc
        +-- 从 pt_regs 读取 pstate
        |   +-- SPSR_EL1 = pt_regs->pstate
        +-- kernel_exit 0
            +-- 恢复 x0~x30
            +-- 释放 pt_regs 栈帧
            +-- eret
                +-- EL0 用户入口
```

1.2 第一个用户进程

```
kernel_main
+-- copy_process(PF_KTHREAD, kernel_process, ...)
    +-- 首次调度
        +-- ret_from_fork
            +-- kernel_process
                +-- launch_user_image("user-image-1", ...)
                    +-- move_to_user_mode
                        +-- 用户代码 VA 0x400000
                        +-- 用户栈 VA 0x07fff000 ~ 0x08000000
                        +-- ELR_EL1 = 0x400000
                        +-- SP_EL0 = 0x08000000
                        +-- TTBR0_EL1 = 该进程的用户 PGD
```

1.3 第二个用户进程

```
kernel_main
+-- copy_process(PF_KTHREAD, kernel_process2, ...)
    +-- 首次调度
        +-- ret_from_fork
            +-- kernel_process2
                +-- launch_user_image("user-image-2", ...)
                    +-- move_to_user_mode
                        +-- 用户代码 VA 0x401000
                        +-- 用户栈 VA 0x07fff000 ~ 0x08000000
                        +-- ELR_EL1 = 0x401000
                        +-- SP_EL0 = 0x08000000
                        +-- TTBR0_EL1 = 该进程的用户 PGD
```

两个进程可以使用相同的用户栈虚拟地址，因为各自的 TTBR0_EL1 指向不同的用户页表，最终映射到不同物理页。

1.4 系统调用调用链

```
EL0 用户代码
+-- call_sys_xxx()
    +-- svc #0
        +-- EL0 synchronous exception
            +-- el0_sync
                +-- kernel_entry 0
                |   +-- CPU 已切换到该任务的 SP_EL1
                |   +-- 在内核栈顶部保存新的 pt_regs
                +-- el0_svc
                    +-- 根据 x8 查找 sys_call_table
                    +-- blr sys_xxx
                    +-- ret_from_syscall
                        +-- 把返回值写入 pt_regs->x0
```

```
                        +-- kernel_exit 0
                            +-- 恢复用户寄存器
                            +-- eret 返回 EL0
```

1.5 定时器中断和进程切换

```
EL0 用户代码
+-- 定时器 IRQ
    +-- el0_irq
        +-- kernel_entry 0                      保存当前用户 pt_regs
        +-- handle_irq()
            +-- timer_tick()
                +-- _schedule()
                    +-- switch_to(next)
                        +-- switch_user_pgd(next->mm->pgd)
                        |   +-- TTBR0_EL1 = next 用户 PGD
                        +-- cpu_switch_to(prev, next)
                            +-- 保存 prev.cpu_context
                            +-- 恢复 next.cpu_context
                            +-- 返回到 next 原先暂停的位置

next 的异常返回路径
+-- handle_irq() 返回
    +-- kernel_exit 0
        +-- 从 next 内核栈上的 pt_regs 恢复现场
        +-- eret 返回 next 的 EL0
```

1.6 两类上下文的职责

```
task_struct
+-- cpu_context
    +-- x19 ~ x30
    +-- 内核 SP
    +-- 用途：EL1 调度切换

内核栈顶部
+-- pt_regs / TrapFrame
    +-- x0 ~ x30
    +-- SP_EL0
    +-- ELR_EL1
    +-- SPSR_EL1
    +-- 用途：EL0 异常进入和 eret 返回
```

cpu_context 是 task_struct 的第一个成员，因此两者起始地址相同。这是当前内核与 cpu_switch_to 汇编之间的软件布局约定，并非 ARM64 硬件规定。

1.7 TTBR0 与 TTBR1

```
低地址用户空间
+-- TTBR0_EL1
```

```
    +-- current->mm->pgd
        +-- 用户代码
        +-- 用户栈

高半区内核空间
+-- TTBR1_EL1
    +-- 全局 kernel_pgd
        +-- 内核代码和数据
        +-- task_struct 与内核栈的直接映射
        +-- MMIO
```

调度用户进程时只需切换 TTBR0_EL1；TTBR1_EL1 保持指向公共内核页表。

1.8 PSTATE 注意事项

当前代码使用：

```
regs->pstate = PSR_MODE_EL0t | PSR_DAIF_MASK;
```

这会得到 0x3c0，屏蔽 Debug、SError、IRQ 和 FIQ。若希望 EL0 用户进程能够被定时器正常抢占，应改为：

```
regs->pstate = PSR_MODE_EL0t;
```

即初始 SPSR_EL1 为 0x0。

1.9 T113（ARMv7-A）：fork、vfork 与 clone

前面各节讲的是树莓派 AArch64 内核。从这一节起讲 T113 内核（Cortex-A7，ARM32）。它的进程创建写在 src/kernel/sched/core.c 里：

- 用户态能用的系统调用只有 fork 和 vfork；
- 内核线程也走同一条路径；
- 三者共用一个入口 t113_kernel_clone()，形状对齐 Linux 的 kernel_clone()。

1.9.1 三种创建方式的语义

	fork	vfork	clone（Linux）	T113 内核线程
地址空间	复制一份：Linux 写时复制，T113/xv6 立即整页拷贝	和父进程共用同一个 mm	CLONE_VM 决定共享还是复制	不需要用户地址空间，只用内核页表
父进程	立即返回，继续运行	挂起，直到子进程 exec 或 exit	带 CLONE_VFORK 时同 vfork	不受影响
子进程返回值	0	0	0（线程用调用者给的新栈）	不回用户态，直接跑 fn(arg)
文件描述符	每个 fd 加一次引用	同 fork	带 CLONE_FILES 时共享整张表	无
典型用途	通用的“复制自己”	紧接 exec 的 shell 场景，省掉一次拷贝	线程、容器、命名空间	内核后台工作
T113 入口	T113_SYS_FORK → t113_fork_user()	T113_SYS_VFORK → t113_vfork_user()	无用户态 clone 系统调用	t113_create_kernel_thread()

fork 和 vfork 的差别只在两件事上：

- 子进程有没有自己的地址空间；
- 父进程要不要等。

用时间线看：

```
fork:
 父 --fork()--+------------ 继续运行（拿到子 pid）----------------→
              |  复制 mm（T113：每个用户页都 memcpy 一遍）
 子           +-- 从 fork() 返回 0 -- exec / 运行 -- exit --→ ZOMBIE
                                                      父 wait() 回收

vfork:
 父 --vfork()-+ WAITING（不被调度）                +-- 返回子 pid -->
              |  共享 mm（引用计数 +1，不拷贝）     |
 子           +-- 从 vfork() 返回 0 -- exec --------+  t113_vfork_complete()
                   （这段时间子进程在父进程的地址空间、
                    父进程的用户栈上运行）
```

1.9.2 统一实现：t113_kernel_clone()

T113 内核用三个“创建策略”标志描述子进程怎样继承父进程的资源。它们定义在 src/include/t113/sched.h，刻意和描述“已有任务是什么”的 PF_* 分开：

标志	值	含义	Linux 对应
T113_CLONE_VM	0x1	共享父进程 mm，不复制	CLONE_VM
T113_CLONE_VFORK	0x2	父进程等子进程 exec/exit，必须和 VM 一起用	CLONE_VFORK
T113_CLONE_KTHREAD	0x4	创建内核线程，忽略父 trapframe	kernel_thread() 的 PF_KTHREAD 路径

三个外部入口只是不同的标志组合：

```
t113_fork_user(tf)             = t113_kernel_clone(0,                tf, "fork-child",  0, 0)
t113_vfork_user(tf)            = t113_kernel_clone(VM | VFORK,       tf, "vfork-child", 0, 0)
                                 + t113_sched_yield()      父进程在这里让出 CPU
t113_create_kernel_thread(n,f,a)= t113_kernel_clone(KTHREAD,          0,  n,           f, a)
```

t113_kernel_clone() 的完整流程：

```
t113_kernel_clone(clone_flags, parent_tf, name, kernel_fn, kernel_arg)
+-- 校验标志
|   +-- 只允许 VM | VFORK | KTHREAD                    其他位 → -1
|   +-- VFORK 必须带 VM                                 vfork 的定义就是共享地址空间
|   +-- 非 KTHREAD 时要求 parent_tf 非空且当前任务是 PF_USER
+-- alloc_process_common()                             所有任务共用的部分
|   +-- t113_alloc_page()  → struct t113_task（独占 1 页）
|   +-- t113_alloc_page()  → 4 KiB 内核栈
|   +-- pid = next_pid++                                sched_lock 保护
|   +-- state = RUNNING
|   +-- trapframe = 栈顶 - sizeof(arm32_trapframe)      68 字节，清零
|   +-- cpu_context: r4 = task, sp = trapframe, lr = ret_from_fork
+-- KTHREAD
|   +-- flags = PF_KTHREAD
|   +-- cpu_context.r5 = kernel_fn, r6 = kernel_arg     ret_from_fork 里 blx r5
```

```
+-- 用户进程
|   +-- flags = PF_USER, parent = t113_current
|   +-- mm = VM ? t113_current->mm                    共享
|   |          : t113_user_vm_clone(t113_current->mm)   新 L1 表 + 逐页拷贝
|   +-- VM    → t113_user_vm_get(mm)                  references++
|   +-- VFORK → child->vfork_parent = t113_current
|   +-- *child->trapframe = *parent_tf                17 个字逐个拷贝
|   +-- child->trapframe->r[0] = 0                    子进程看到的返回值
|   +-- files[i] = 父 files[i]; t113_vfs_file_get()   共享打开文件，引用 +1
+-- sched_lock 内
    +-- VFORK → t113_current->state = WAITING          父进程从此不会被选中
    +-- task_list_append(child)                       全体任务链表（wait/exit 用）
    +-- ready_append(child)                           就绪环
返回 child->pid（父进程这一侧的返回值）
```

KTHREAD 分支和用户分支唯一的公共部分是 `alloc_process_common()`：它给每个新任务装上同样的“第一次运行”路径 `ret_from_fork`。真正的差别都推迟到子进程第一次被调度时才体现。

1.9.3 任务结构与内核栈布局

```
task 页（4 KiB）                          内核栈页（4 KiB）
+--------------------------+ +0          +--------------------------+ stack+0
| cpu_context              |             |                          |
|   r4..r11, sp, lr        | switch.S    |      ↓ SVC 模式栈向下长    |
| pid  state  flags  name  | 要求在偏移0 |                          |
| kernel_stack ------------+------------→|                           |
| trapframe ---------------+--+          |  每次 SVC 再压一个 68 B 帧 |
| mm --→ struct t113_user_vm|  |          +--------------------------+ stack+4096-68
| ready_prev/next          |  +---------→| struct arm32_trapframe    |
| task_prev/next  parent   |             |   r[0..12]  (+0..+48)     |
| vfork_parent  wake_tick  |             |   user_sp   (+52)         |
| exit_status  files[4]    |             |   user_lr   (+56)         |
+--------------------------+             |   pc (+60)  cpsr (+64)    |
                                         +--------------------------+ stack+4096
cpu_context.sp 初值 = trapframe，cpu_context.lr 初值 = ret_from_fork
```

`task->trapframe` 只在子进程第一次返回用户态时用。之后每次 `svc`，`kernel_svc_entry` 都在当前 SVC 栈顶下方再压一个新的 trapframe，系统调用返回时弹掉。所以 fork 拷贝的必须是这次系统调用的帧，也就是 `t113_syscall_dispatch(tf)` 传下来的 `parent_tf`，而不是 `t113_current->trapframe`。`_Static_assert` 把偏移 52/60/64 和 `fork_entry.S` 里的 `TF_USER_SP/TF_PC/TF_CPSR` 钉在一起。

图 1.1 和图 1.2 把“第一次进入用户态之前”的全部状态画在一起：任务结构、内核栈顶的寄存器帧、返回用户态时的系统寄存器、页表遍历、用户地址空间和描述符位。前者是 T113 的 init 进程，上半部分完全按 AArch32（ARMv7-A，Cortex-A7）来画：处理器模式与异常向量、CP15 寄存器、短描述符格式，以及 init 头两条指令实际触发的两次地址翻译（见下文）；后者是树莓派 AArch64 版的 pid 1，只作对照，两张图的下半部分面板一一对应。T113 图里的物理地址由页分配器动态给出，所以写成 `PA(task)`、`PA(l1)` 这样的符号，其余数值都来自源码常量。

两张图对照下来，差别集中在这几处：

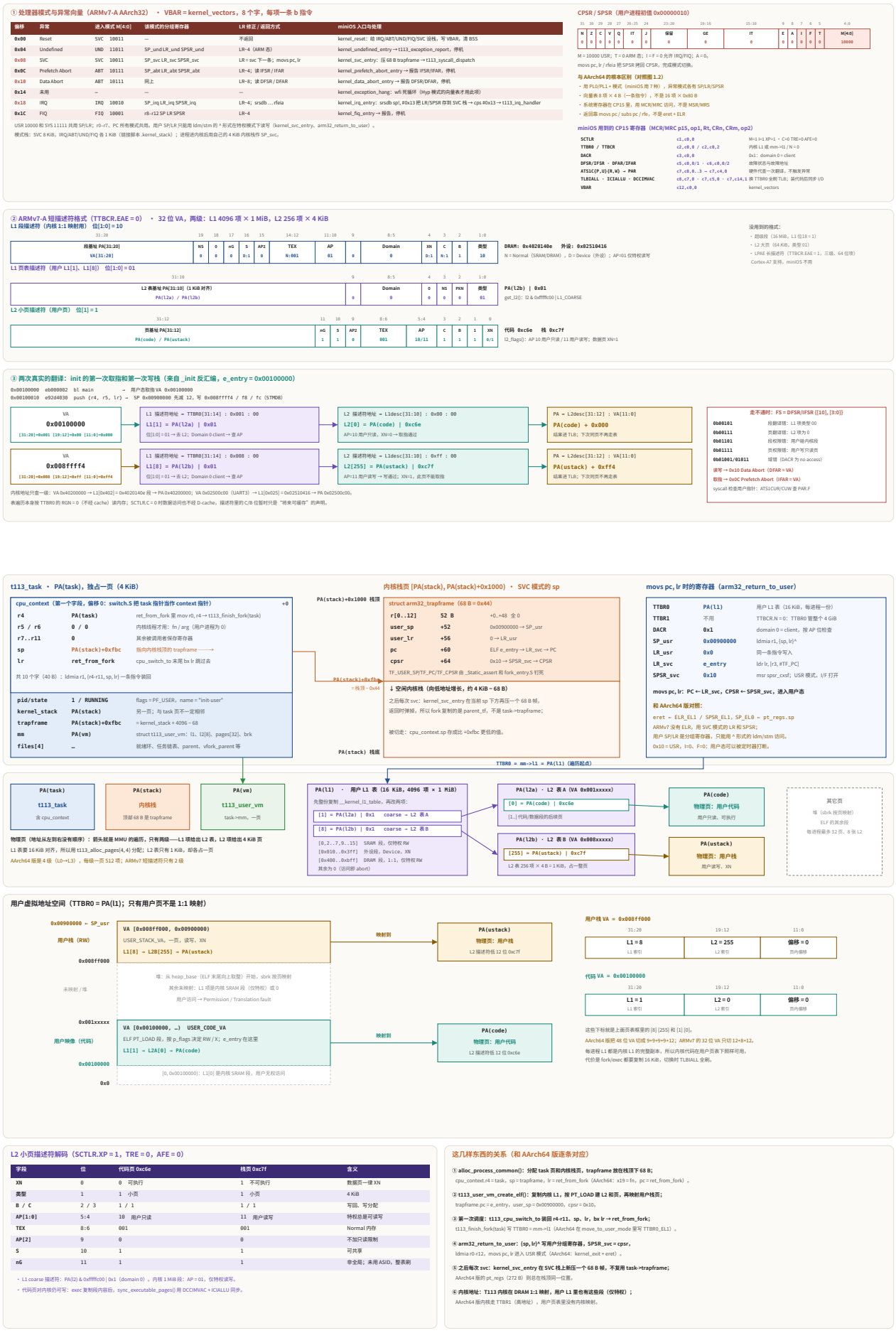


图 1.1: T113 init 进程：异常向量、MMU 与地址翻译，t113_task、arm32_trapframe、内核栈、用户地址空间与 L1 → L2 页表

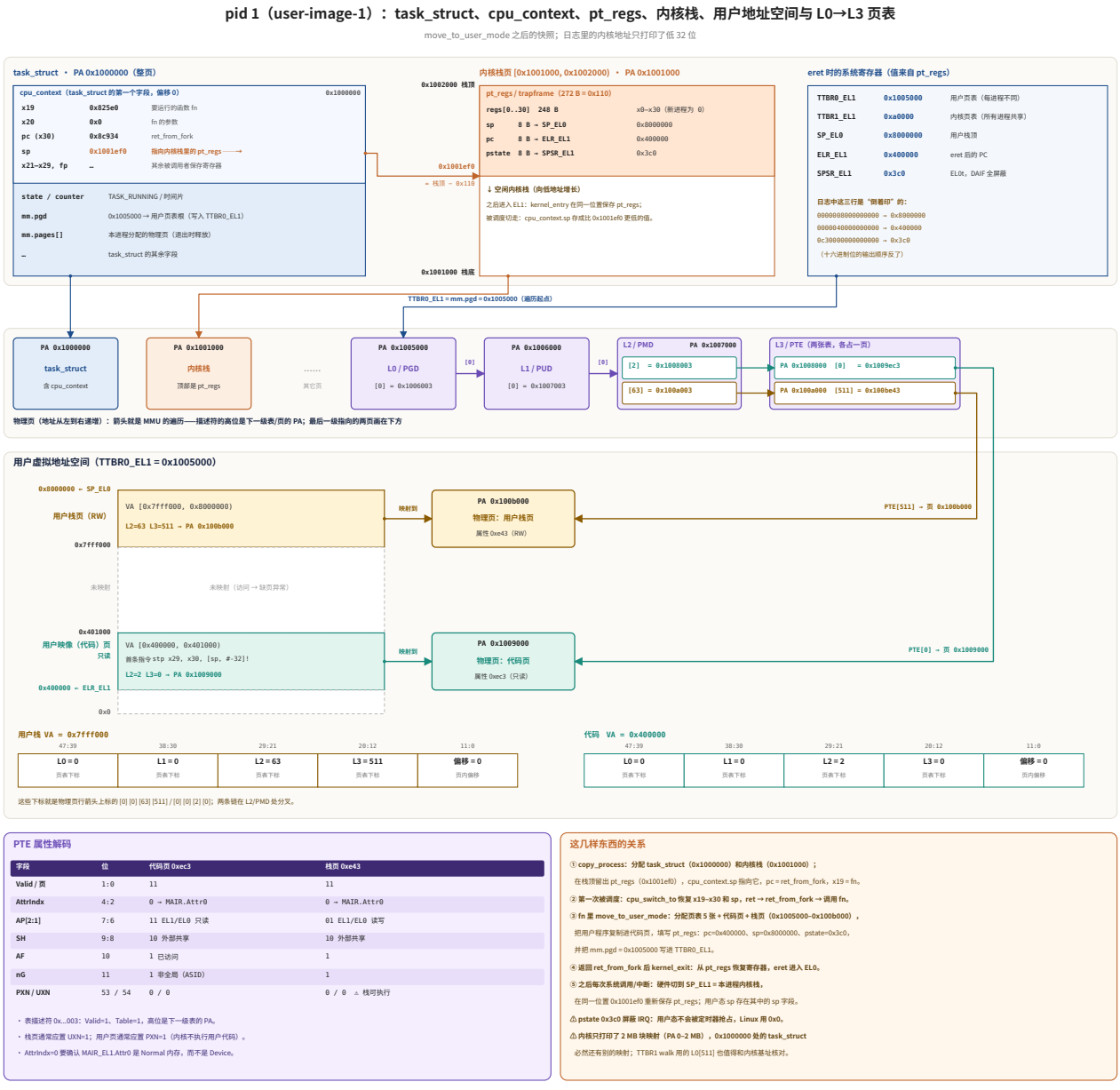


图 1.2: 树莓派 AArch64 pid 1：task_struct、pt_regs、内核栈、用户地址空间与 L0 → L3 页表

方面	树莓派 AArch64	T113 ARMv7-A
切换上下文	x19–x29、fp、sp、pc（x19 = fn）	r4–r11、sp、lr（r4 = task，r5/r6 = 内核线程的 fn/arg）
用户寄存器帧	`pt_regs` 272 B，总在栈顶同一位置	`arm32_trapframe` 68 B，每次 svc 在当前 sp 下新压一个
回到用户态	`kernel_exit` + `eret`，ELR_EL1 / SPSR_EL1 / SP_EL0	`ldmia {sp, lr}^` + `msr spsr` + `movs pc, lr`
页表	4 级（L0 → L3），每级 512 项，48 位 VA	2 级短描述符：L1 4096 项 × 1 MiB（16 KiB），L2 256 项 × 4 KiB
内核映射	TTBR1_EL1，高地址，用户页表里没有	TTBCR.N = 0，每个用户 L1 都是内核 L1 的完整副本（段只允许特权访问）
用户布局	代码 0x400000，栈顶 0x8000000	代码 0x00100000，栈顶 0x00900000
进入用户态时的状态寄存器	SPSR = 0x3c0（EL0t，DAIF 全屏蔽）	CPSR = 0x10（USR，IRQ/FIQ 打开）

AArch32 的异常、MMU 与地址翻译

处理器模式和分组寄存器。 ARMv7-A 不用 AArch64 的 EL，而是用特权级 PL0/PL1（虚拟化扩展另有 PL2）加处理器模式。miniOS 用到其中 7 种：USR、SYS、SVC、IRQ、FIQ、ABT、UND；Cortex-A7 另有 MON（安全扩展）和 HYP（虚拟化扩展），miniOS 不用。模式号在 CPSR[4:0]。除 USR/SYS 共用一组外，每种异常模式都有自己的 SP、LR、SPSR（FIQ 还有自己的 r8–r12）。所以 `kernel_reset` 要依次 `cps` 进每种模式设栈（SVC 8 KiB，其余各 1 KiB）；用户态的 SP/LR 在特权模式下看不见，`kernel_svc_entry` 和 `arm32_return_to_user` 只能用 `stmia/ldmia {sp, lr}^` 这种带 `^` 的形式访问。

异常向量。 `VBAR`（CP15 c12）指向 `kernel_vectors`，表里是 8 条 `b handler` 指令，每项 4 字节，按偏移 0x00 Reset、0x04 Undefined、0x08 SVC、0x0C Prefetch Abort、0x10 Data Abort、0x14 未用、0x18 IRQ、0x1C FIQ 排列。进入异常时硬件把返回地址放进该模式的 LR，旧 CPSR 放进 SPSR；LR 的偏移因异常而异（Undef/Prefetch/IRQ/FIQ 减 4，Data Abort 减 8，SVC 不用改）。返回也不是一条统一指令：SVC 用 `movs pc, lr`，IRQ 先用 `srsdb sp!, #0x13` 把 LR_irq 和 SPSR_irq 压到 SVC 栈，处理完再用 `rfeia sp!` 一次恢复 PC 和 CPSR。Undef、Abort、FIQ 在 miniOS 里只报告现场然后停机。

CP15 与 MMU 配置。系统寄存器都在协处理器 CP15 里，用 `mcr/mrc p15, …`访问。`t113_mmu_init()` 写 TTBR0 = `__kernel_l1_table`、TTBCR = 0（N = 0，TTBR0 覆盖整个 4 GiB，TTBR1 不参与）、DACR = 1（domain 0 为 client，按 AP 位检查），再在 SCTLR 里打开 M、I、XP，清 TRE、AFE，并且暂不打开 C（D-cache）。内核 L1 全是 1 MiB 段，1:1 映射：0x000–0x00F MiB 是 SRAM（Normal），0x010–0x3FF MiB 是外设（Device、XN），0x400–0xBFF MiB 是 DRAM（Normal），都只允许特权访问（AP = 01）。

短描述符和两级翻译。 32 位 VA 拆成 [31:20] L1 索引、[19:12] L2 索引、[11:0] 页内偏移。

```
L1 描述符地址 = TTBR0[31:14] : VA[31:20] : 00
   位[1:0] = 10 → 段：PA = desc[31:20] : VA[19:0]            （内核，查一级就结束）
   位[1:0] = 01 → 页表：继续
L2 描述符地址 = L1desc[31:10] : VA[19:12] : 00
   位[1]   = 1  → 小页：PA = desc[31:12] : VA[11:0]
```

每个用户 L1 都是内核 L1 的完整副本，只把 L1[1]（代码）和 L1[8]（栈）换成页表描述符。这一点与 Linux 在 ARM 短描述符下的做法相同：TTBCR.N = 0，每个进程的 pgd 都带一份内核映射。区别是 Linux 在同一页里还放了软件用的影子 PTE，miniOS 的 1 KiB L2 表则单独占一页。

两次真实的翻译。从 `_init` 的反汇编看，`e_entry` = 0x00100000，第一条指令 `bl main` 触发对 VA 0x00100000 的用户态取指：L1[1] → L2A[0] = PA(code) | 0xc6e，AP = 10（用户只读）、XN = 0，通过。`main` 的第一条指令 `push {r4, r5, lr}` 在 SP = 0x00900000 时写 0x008ffff4–0x008fffff：L1[8] → L2B[255] = PA(ustack) | 0xc7f，AP

= 11（用户读写），通过；同一页 XN = 1，不能取指。

走不通时。翻译或权限检查失败，数据访问进 Data Abort（DFSR 给状态，DFAR 给地址），取指进 Prefetch Abort（IFSR/IFAR）。FS 编码 0b00101/0b00111 是段/页翻译错，0b01101/0b01111 是段/页权限错。系统调用检查用户指针时，不让它真的出错，而是用 ATS1CUR/ATS1CUW（CP15 c7, c8）让硬件代查一次翻译，再看 PAR 的 F 位（`t113_mmu_probe()`）。切换进程只换 TTBR0 并 TLBIALL 全刷，没有使用 ASID（CONTEXTIDR）。

1.9.4 子进程第一次运行：ret_from_fork

```
t113_sched_tick() / t113_sched_yield()
+-- schedule_next()
    +-- 在就绪环里从 prev->ready_next 开始找 state == RUNNING
    |   +-- 一个都没有 → boot_context（kernel_boot 的 wfi 循环）
    +-- t113_current = next
    +-- switch_task_mm(next)            TTBR0 ← next->mm->l1 或内核 L1 表
    +-- t113_cpu_switch_to(prev, next)  switch.S
        +-- stmia r0, {r4-r11, sp, lr}  保存 prev
        +-- ldmia r1, {r4-r11, sp, lr}  装入 next：r4=task, sp=trapframe, lr=ret_from_fork
        +-- bx lr
            +-- ret_from_fork           fork_entry.S
                +-- t113_finish_fork(r4)
                |   +-- PF_KTHREAD → t113_switch_kernel_vm()
                |   +-- PF_USER    → t113_switch_user_vm(task->mm)
                +-- 内核线程：cpsie i; mov r0, r6; blx r5
                |   +-- fn(arg) 返回 → t113_task_exit()   state = IDLE, wfi
                +-- 用户进程：t113_task_trapframe(r4) → arm32_return_to_user(tf)
                    +-- ldmia {sp, lr}^         写 User 模式的 banked SP/LR
                    +-- lr  = tf->pc
                    +-- spsr = tf->cpsr         0x10 = User 模式，IRQ 打开
                    +-- ldmia r0-r12            r0 = 0
                    +-- movs pc, lr             CPSR ← SPSR，跳到 svc 的下一条
                        +-- 子进程在用户态从 fork()/vfork() 返回 0
```

`r4/r5/r6` 是 miniOS 自己定的约定：它们是被调用者保存寄存器，`t113_cpu_switch_to` 会原样恢复，所以可以用来把“任务指针、内核函数、参数”带到 `ret_from_fork`。AArch64 版本用的是 `x19/x20`，道理相同。

1.9.5 vfork：父进程怎样等、怎样被唤醒

```
父进程 P                          调度器                          子进程 C
vfork() - svc #0, r7 = 14 -→ t113_syscall_dispatch(tf)
  t113_vfork_user(tf)
    t113_kernel_clone(VM|VFORK)
      C.mm = P.mm（references = 2）
      C.vfork_parent = P
      P.state = WAITING
    t113_sched_yield() -------→ schedule_next()
                              P 不是 RUNNING，跳过
                              选中 C ----------------------→ ret_from_fork
                                                            用户态 vfork() 返回 0
                                                            exec("/bin/ls", argv)
                                                            +-- sys_exec()
                                                                +-- 新建 vm，装入 ELF
                                                                +-- C.mm = 新 vm
                                                                +-- t113_vfork_complete(C)
                                                                |   +-- P.state = RUNNING
                                                                |   +-- C.vfork_parent = 0
                                                                +-- 切 TTBR0 到新 vm
                                                                +-- t113_user_vm_put(P 的 mm)
```

```
                                                    references = 1
                              ...之后某次调度选中 P
 ←---------------------------- t113_sched_yield() 返回
 t113_vfork_user() 返回 C 的 pid
 用户态 vfork() 返回 C 的 pid
```

t113_vfork_complete() 有两个调用点，对应 Linux 在 mm_release() 里 complete(vfork_done) 的两种情况：

- sys_exec()（src/kernel/syscall.c）：子进程换上了自己的地址空间；
- t113_exit_current()（sched/core.c）：子进程没 exec 就退出了。

父进程用的是 WAITING 状态，不是忙等。t113_exit_current() 里，“父进程在 WAITING 就改回 RUNNING”这条规则同时服务了 wait() 和 vfork。

笔记 注意：vfork 的子进程在父进程的用户栈上运行。user-t113/usys.S 的 __sys_vfork 用 push {r7, lr} / pop {r7, pc} 保存返回地址，libc.c 的 vfork() 又是 C 包装。子进程返回后再调用 exec()，新的栈帧会落在同一片栈上，覆盖父进程还没弹出的返回地址。父进程恢复后从被改写的栈上返回，就会跳错地方。glibc 在 ARM 上用汇编写 vfork，把返回地址留在寄存器里、不碰栈，原因就在这里。目前 user-t113/ 里还没有程序调用 vfork()；要用它，应先把包装改成不使用栈的汇编，或在子进程里直接内联 svc。

1.9.6 地址空间：共享还是复制

```
struct t113_user_vm（一页）
+-- references                    vfork 共享时为 2
+-- l1 --→ 16 KiB L1 表（4096 项，每项管 1 MiB）
|          +-- 内核区：创建时整表拷贝 __kernel_l1_table
|          +-- 用户区：指向 l2[]（最多 8 张 L2 粗页表）
+-- pages[32]  {va, pa, writable, executable}
|          代码 0x00100000 起，栈 0x008ff000－0x00900000，堆 [heap_base, brk)
+-- page_count / heap_base / brk

fork  : t113_user_vm_clone(src)
        +-- 新 vm 页 + 新 16 KiB L1（拷贝内核项）
        +-- 对 src->pages[i]：map_page(va, 同权限) + copy_bytes(4 KiB)
        +-- heap_base / brk 照抄
        +-- sync_executable_pages()       代码页逐行 DCCIMVAC，再 ICIALLU，保证取指看到新内容
vfork : child->mm = parent->mm; references++
exec  : 新建 vm，成功后 t113_user_vm_put(旧 vm)
wait  : 回收僵尸时 t113_user_vm_put(zombie->mm)
```

T113 没有写时复制，fork 的代价正比于用户页数（最多 32 页，即 128 KiB 的 memcpy）。这也是 vfork 存在的意义：shell 的“fork 后马上 exec”可以省掉这次拷贝。

1.9.7 exit 与 wait

```
t113_exit_current(status)               T113_SYS_EXIT
+-- t113_vfork_complete(self)           没 exec 就退出的 vfork 子进程也要放行父进程
+-- 关闭 files[0..3]
+-- 子进程过继给 init_task                与 xv6 reparent() 相同
|   +-- 有已是 ZOMBIE 的且 init 在 WAITING → 唤醒 init
+-- exit_status = status, state = ZOMBIE
+-- parent 在 WAITING → RUNNING
+-- t113_sched_yield()，不再返回

t113_wait_child(&status)                T113_SYS_WAIT
+-- 循环
    +-- 扫 task_head 链表找 parent == current 的子进程
```

```
    +-- 有 ZOMBIE：摘链 → t113_user_vm_put(mm) → 释放 task/栈页 → 返回 pid
    +-- 一个子进程都没有 → -1
    +-- 否则 state = WAITING，yield，被 exit 唤醒后重扫
```

1.9.8 与 xv6 的进程创建对照

xv6（这里对照的是 xv6-aarch64/kernel/proc.c）只有 fork()，没有 vfork 和 clone，也没有内核线程。它的调用链：

```
sys_fork() → fork()                                  kernel/proc.c
+-- allocproc()
|   +-- 遍历 proc[NPROC] 找 UNUSED，持有 p->lock 返回
|   +-- pid = allocpid()
|   +-- trapframe = kstack + PGSIZE - sizeof(trapframe)   aarch64 移植放在内核栈顶
|   +-- pagetable = uvmcreate()
|   +-- context.x30 = forkret，context.sp = trapframe 处
+-- uvmcopy(p->pagetable，np->pagetable，p->sz)      逐页 kalloc + memmove，没有 COW
+-- np->sz = p->sz
+-- *(np->trapframe) = *(p->trapframe)
+-- np->trapframe->x0 = 0                            子进程返回 0
+-- ofile[i] = filedup()；cwd = idup()
+-- np->parent = p                                   持 wait_lock
+-- np->state = RUNNABLE

scheduler()                                          每个 CPU 一个调度循环
+-- swtch(&c->context，&p->context)
    +-- forkret()
        +-- release(&p->lock)                        scheduler 持锁切过来
        +-- 第一次：fsinit(ROOTDEV)                  文件系统初始化要能 sleep
        +-- usertrapret(tf) → eret 回 EL0
```

方面	xv6-aarch64	T113 miniOS
创建接口	只有 fork()	t113_kernel_clone(flags) 一个入口，派生 fork、vfork、内核线程
进程表	静态 proc[NPROC]，每项一把 p->lock	每个任务动态分配一页，task_head 链表 + 就绪环，一把 sched_lock
地址空间	4 级页表，uvmcopy 按 p->sz 逐页复制	ARMv7 短描述符 L1/L2，t113_user_vm_clone 按 pages[] 逐页复制
共享地址空间	不支持	T113_CLONE_VM + references 引用计数
父进程等待	只有 wait()，用 sleep/wakeup 加通道	wait() 和 vfork 都用 WAITING 状态，exit/exec 改回 RUNNING
第一次运行	forkret → usertrapret	ret_from_fork → t113_finish_fork → arm32_return_to_user
调度切换	进程 ↔ 每 CPU 的调度线程（两次 swtch）	进程 ↔ 进程直接 t113_cpu_switch_to，单核
内核线程	无	T113_CLONE_KTHREAD，r5/r6 带函数和参数
返回值寄存器	trapframe->x0	trapframe->r[0]
文件系统初始化	第一个进程的 forkret 里 fsinit()	kernel_boot() 里在创建 init 之前挂好

1.9.9 与 Linux 的对照

Linux 的 fork、vfork、clone、clone3 和 `kernel_thread()` 最后都进 `kernel_clone(struct kernel_clone_args *)`：

```
fork()        → kernel_clone({ .exit_signal = SIGCHLD })
vfork()       → kernel_clone({ .flags = CLONE_VFORK | CLONE_VM, .exit_signal = SIGCHLD })
clone(flags...) → kernel_clone({ .flags = flags, .stack = newsp, ... })
kernel_thread → kernel_clone({ .flags = CLONE_VM | CLONE_UNTRACED, .fn = fn, .fn_arg = arg, .kthread = 1 })

kernel_clone()
+-- copy_process()          copy_mm / copy_files / copy_fs / copy_sighand / copy_thread
+-- CLONE_VFORK → p->vfork_done = &vfork (completion)
+-- wake_up_new_task(p)
+-- CLONE_VFORK → wait_for_vfork_done()   子进程 exec/exit 时 mm_release() → complete()
```

T113 的做法是这条路径的缩小版：

- `T113_CLONE_VM` 对应 `copy_mm()` 里共享还是 `dup_mm()`；
- `vfork_parent` + `WAITING` 对应 `vfork_done` completion；
- `t113_vfork_complete()` 对应 `mm_release()` 里的 `complete_vfork_done()`；
- 文件描述符总是逐个加引用，相当于 Linux 不带 `CLONE_FILES` 时的 `dup_fd()`；
- 没有线程，因为没有用户态 clone 系统调用，也不能给子进程另指用户栈。

1.9.10 树莓派 AArch64 与 T113 ARMv7 的实现区别

方面	树莓派（前文）	T113
创建函数	`copy_process(clone_flags, fn, arg, stack)`，`clone_flags` 复用 `PF_KTHREAD`	`t113_kernel_clone(flags, parent_tf, name, fn, arg)`，策略标志与 `PF_*` 分开
首次运行寄存器	`x19 = fn`、`x20 = arg`，`pc = ret_from_fork`	`r4 = task`、`r5 = fn`、`r6 = arg`，`lr = ret_from_fork`
用户上下文	`pt_regs`（x0–x30、sp、pc、pstate）	`arm32_trapframe`（r0–r12、user_sp、user_lr、pc、cpsr，68 B）
进入内核	`svc #0`，异常级 EL0 → EL1，`x8` 是调用号	`svc #0`，User → SVC 模式，`r7` 是调用号
回用户态	`kernel_exit` → `eret`（ELR/SPSR_EL1）	`ldmia {sp,lr}^` + `movs pc, lr`（SPSR_svc）
用户页表	`TTBR0_EL1`，4 KiB 粒度多级表	`TTBR0`，16 KiB L1 + 1 KiB L2，内核项拷进每张 L1
用户进程来源	内核把嵌入的镜像拷进去（`move_to_user_mode`）	从根文件系统读 ELF（`/init`、`exec`）
fork/vfork 系统调用	无	有，另有 wait/exit/exec/sbrk

第二章 根文件系统与 devfs

本章对应当前 miniOS 的实现，说明内核如何从 SD 卡找到并挂载根文件系统（xv6 血统的 minifs）、VFS 挂载表的形状、`/boot` 的 FAT32 交接（bootfs）、`/dev` 设备文件（devfs），以及用户态 `/init`、`/bin/sh`是如何从根文件系统里跑起来的。

笔记 更新：`/boot` 已改为内核直接读写的 FAT32 文件系统（bio + fat32），挂载顺序也改成先 `/` 再 `/boot`，详见第 3 章。本章的 bootfs 小节现在描述的是 FAT32 挂载失败时的后备路径。

当前已打通的链路：

- SPL 读 SD 128 KiB 的 bootpack 装内核，再读 FAT32 分区把 XR829 固件装进 DDR 并补写 bootpack 头（交接）；
- 内核 `kernel_boot()` 依次挂载 `/`（minifs，MBR 第 2 分区按需读）、`/boot`（FAT32，失败时退回 bootfs）、`/dev`（devfs），任一最终失败即 PANIC；
- XR829 驱动从 `/boot/*.bin` 经 VFS 读固件（不再打进 bootpack）；
- 内核从 `/` 读出 `/init` 创建第一个用户进程；`init` fork+exec `/bin/sh`，shell 及 `ls/cat/echo/...` 都从 minifs 经 VFS 加载。

尚未实现：文件写入（minifs 只读）、minifs 日志区、devfs 作为用户态 stdio 后端、`mount/umount` 系统调用、多个块设备。

2.1 SD 卡布局

区域	位置	内容
Boot0（SPL）	8 KiB 偏移，512 B	miniBoot，SRAM 里跑，负责 DDR init + 装内核
bootpack	128 KiB 偏移	512 B header + kernel 镜像（见 `bootpack.h`）
MBR 分区 1	FAT32	`boot_xr829.bin` / `fw_xr829.bin` / `sdd_xr829.bin`（macOS 挂载于 `/Volumes/bootfs`）
MBR 分区 2	minifs 根文件系统	`build-t113/minifs.img` 整镜像 dd 进去，⩾ 32 MiB

分区 2 没有文件系统以外的含义：`rootdev.c` 只是取 MBR 第二个分区表项的 LBA/长度当块设备用。分区本身用任何分区工具建（类型 0x83 即可），FAT32 分区 1 是给 SPL 固件加载和人工换固件用的。

```
SD 8 KiB    +------------+
            | Boot0 SPL  |  读 128 KiB bootpack → 装内核
SD 128 KiB  +------------+
            | bootpack   |  header + kernel @0x40200000
分区 1      +------------+
            | FAT32      |  SPL 读固件 → DDR 0x40100000/0x40110000/0x40140000
分区 2      +------------+
            | minifs     |  内核 rootdev 按需读（块设备，非预载）
            +------------+
```

2.2 总调用链

```
miniBoot SPL
+-- spl_main()
    +-- load_bootpack()
        +-- t113_sd_init()                    SMHC0 识别卡
        +-- 读 SD 128 KiB bootpack header     CRC32 校验
        +-- 逐条目读镜像到 load_addr + CRC32  kernel → 0x40200000
        +-- header 拷到 0x401ff000            内核侧查找入口
        +-- load_fat32_firmware()             FAT32 交接
            +-- fat32_mount()                 MBR 找 FAT32 分区，BPB 解析
            +-- fat32_load_root() × 3         boot/fw/sdd_xr829.bin → DDR
            +-- bootpack_slot()               补 3 个条目(offset=0 常驻)，重算 header CRC
        +-- 跳 0x40200000

miniOS kernel_main()                          boards/100ask_t113/platform_kernel.c
+-- t113_uart_init() / t113_mmu_init()
+-- t113_bootpack_dump()                      打印全部条目（含 SPL 交接的固件）
+-- kernel_boot()                             kernel/init/main.c
    +-- t113_page_alloc_init()                页分配器
    +-- t113_vfs_init()                       mount/super_block/inode/dentry/file 池清零
    +-- t113_minifs_mount_rootdev()           → /       失败 PANIC
    |   +-- t113_rootdev_init()               MBR 分区 2 解析
    |   |   +-- t113_sd_init()                SMHC0，1-bit，轮询
    |   |   +-- 读 LBA0，验 0x55AA，取表项 2  type/lba/sectors
    |   +-- 读 root_lba+2 扇区                块 1 = superblock
    |   +-- 验 magic 0x10203040，size×2 ⩽ 分区扇区数 → t113_vfs_mount_fs("/")
    +-- t113_fat32_mount_vfs()                → /boot   读写 FAT32
    |   +-- 失败 → t113_bootfs_mount_vfs()     → /boot   SPL 交接视图，再失败 PANIC
    +-- t113_devfs_init()                     → /dev    失败 PANIC
    +-- net_init()                            CONFIG_NET
    +-- t113_driver_core_init()               平台/sdio 驱动绑定
    |   +-- uart probe → t113_chrdev_register(1/3, 0) + t113_devfs_create_chrdev("console"/"ttyS0")
    |   +-- xradio probe → need_image("/boot/*.bin")  经 VFS 读 FAT32 上的固件
    +-- t113_sched_init()
    +-- start_init_process()
    |   +-- t113_vfs_open("/init") → t113_vfs_read() → t113_create_init_user_process()
    +-- Timer0 + 开中断                       之后 idle: wfi

用户态
+-- /init (init.c)
    +-- fork() + exec("/bin/sh")              sys_exec: VFS 打开 → ELF → 新地址空间
    +-- wait() 孤儿回收，shell 退出后重启
```

/ 和 /dev 是硬依赖；/boot 先试 FAT32、再退回 bootfs，两者都失败才算失败。任何一个最终失败，kernel_boot() 都打印 PANIC 并 wfi 死循环——没有根文件系统就没有用户态，无处可退。这对应 Linux “unable to mount root fs”的 panic，只是 miniOS 连 initramfs 兜底都没有，也不需要：内核本身在 bootpack 里，卡上就是它的全部世界。挂载过程的图解见下一节。

2.3 图解：内核怎样把根文件系统挂到 /

2.3.1 挂载结果

先纠正一个容易混的点：devfs 挂在 /dev，不是 /boot。/boot 挂的是 SD 卡第一个分区上的 FAT32；只有 FAT32 挂载失败时，才退回到 bootfs（SPL 预加载镜像的只读视图）。启动完成后，整个命名空间是这样的：

```
SD 卡                              VFS 挂载表 mounts[8]          命名空间
+--------------+
| MBR          |                   [0] "/"     → minifs_vfs_ops   /
| Boot0 / pack |                   [1] "/boot" → fat32_ops        +-- init
+--------------+  第 1 个 FAT32 分区    (失败时  → bootfs_ops)       +-- bin/ sh ls cat ...
| 分区 1 FAT32 | ------------------------------------------------→  +-- boot/              FAT32
+--------------+  MBR 表项 2                                        |   +-- boot_xr829.bin
```

```
| 分区 2 minifs| ------------------------------------------------→  |   +-- fw_xr829.bin
+--------------+                      [2] "/dev"  → devfs_ops       |   +-- sdd_xr829.bin
                                                                   |   +-- wifi.txt
内存: devfs nodes[16] ---------------------------------------------→ +-- dev/                devfs
       (uart 驱动 probe 时注册)                                      +-- console
                                                                    +-- ttyS0
```

find_mount() 按最长前缀选后端。路径分隔符要求完整匹配，所以 /bootx 不会落到 /boot：

路径	匹配的挂载点	后端
/init	/	minifs
/bin/sh	/	minifs
/boot	/、/boot，取 /boot	fat32（根目录）
/boot/fw_xr829.bin	/boot	fat32
/bootx	/	minifs（找不到就返回失败）
/dev/console	/dev	devfs

2.3.2 启动时的挂载顺序

```
SPL 跳到 0x40200000
+-- kernel_entry.S
    +-- kernel_main(dram_size)                    boards/100ask_t113/platform_kernel.c
        +-- t113_uart_init()                      早期串口
        +-- t113_mmu_init()                       内核 L1 表，打开 MMU
        +-- t113_bootpack_dump()
        +-- kernel_boot(pool_base, dram_end)       kernel/init/main.c，与板子无关的部分
            +-- t113_page_alloc_init()
            +-- t113_vfs_init()                   清空 mount/super_block/inode/dentry/file 五个池
            +-- t113_minifs_mount_rootdev()       (1) "/"   失败 → PANIC
            |   +-- t113_rootdev_init()
            |   |   +-- t113_sd_init()            SMHC0：CMD0/8/ACMD41/2/3/7，切到 12 MHz
            |   |   +-- t113_sd_read(0, 1, mbr)   验 55 AA
            |   |   +-- 表项 2：root_lba、root_sectors_count
            |   +-- t113_rootdev_read(2, 2, block_cache)     minifs 块 1 = 超级块
            |   +-- 验 magic 0x10203040，size × 2 ⩽ 分区扇区数
            |   +-- t113_vfs_mount_fs("/", MINIFS_MAGIC, &fs_sb, &minifs_vfs_ops)
            +-- t113_fat32_mount_vfs()            (2) "/boot"
            |   +-- binit()                       建 bcache
            |   +-- bread(0)：MBR，找第一个 FAT32 类型的分区
            |   +-- bread(part_lba)：BPB 校验并算出布局
            |   +-- t113_vfs_mount_fs("/boot", 0x4d44, &fs, &fat32_ops)
            |       （先 lookup_path("/boot") 取 minifs 里的挂载点目录）
            |   失败 → t113_bootfs_mount_vfs()     (2') 退回 bootfs，再失败 → PANIC
            +-- t113_devfs_init()                 (3) "/dev"  失败 → PANIC
            +-- net_init()                        CONFIG_NET
            +-- t113_driver_core_init()           设备、驱动绑定
            |   +-- uart probe → t113_chrdev_register(1, 0, &console_fops)
            |   |               t113_chrdev_register(3, 0, &console_fops)
            |   |               t113_devfs_create_chrdev("console", 1, 0)
            |   |               t113_devfs_create_chrdev("ttyS0", 3, 0)
            |   +-- mmc_rescan_all() → xradio probe → need_image("/boot/*.bin")
            +-- t113_sched_init()
            +-- start_init_process()
            |   +-- t113_vfs_open("/init") → minifs
            |   +-- t113_vfs_read() 整个 ELF 读进临时页
            |   +-- t113_create_init_user_process()
            +-- 打开定时器和中断，进入 wfi 循环
```

先挂 / 有一个具体原因：t113_rootdev_init() 负责 t113_sd_init()，也就是初始化 SMHC0 和 SD 卡；FAT32 和它共用这个控制器。驱动必须在/boot、/dev 都挂好之后才初始化，因为 XR829 的 probe 要从 /boot

读固件，UART 的 probe 要往 /dev 里注册节点。

2.3.3 rootdev：分区到块设备

minifs 的块是 1 KiB，SD 扇区是 512 B，rootdev.c 只做一次线性换算：

```
minifs 块号 b  --→  分区内扇区 2b, 2b+1  --→  整卡 LBA root_lba + 2b

t113_rootdev_read(sector, count, dst)
+-- 越界检查：sector + count ⩽ root_sectors_count
+-- t113_sd_read(root_lba + sector, count, dst)
```

它相当于 Linux 的分区块设备（/dev/mmcblk0p2），或 xv6 的 ROOTDEV 加上 virtio_disk_rw()。区别是它不经过任何缓存，缓存放在 minifs 里（只缓存一块）。

2.3.4 minifs 的磁盘布局和查找

minifs 沿用 xv6 mkfs 的格式（MINIFS_MAGIC 就是 xv6 的 FSMAGIC 0x10203040），布局和 xv6 一样：

```
块：  0      1       2 ... logstart+nlog-1   inodestart ...     bmapstart ...   数据块 ...
    +------+--------+---------------------+---------------+-------------+-------------+
    | boot | super  | log (minifs 不用)     | inode 块       | 空闲位图     | 文件/目录内容 |
    +------+--------+---------------------+---------------+-------------+-------------+
          magic size nblocks ninodes nlog logstart inodestart bmapstart

struct minifs_dinode (64 B, 每块 16 个)
+-- type  1 = 目录, 2 = 文件
+-- major minor nlink
+-- size
+-- addrs[12] 直接块 + addrs[12] 一个间接块 (256 项)  → 最大 268 KiB

struct minifs_dirent (32 B)
+-- inum (u16)
+-- name[30]
```

exec("/bin/sh") 在 minifs 里的查找：

```
t113_vfs_open("/bin/sh")
+-- minifs_vfs_lookup()
    +-- t113_minifs_open() → namei("/bin/sh")
        +-- current = ROOTINO (1)
        +-- "bin": directory_lookup(1, "bin")
        |   +-- read_inode(1)          块 inodestart + 1/16, 取第 1 项
        |   +-- 逐个 32 B 目录项 inode_read() → 找到 inum = N
        +-- "sh":  directory_lookup(N, "sh") → inum = M
        +-- *inum = M; read_inode(M) 得到 size
sys_exec → t113_vfs_read(file, image, size)
+-- minifs_vfs_read() → t113_minifs_read(M, off, dst, n)
    +-- inode_read()
        +-- 每块：inode_block(inode, off / 1024)       直接块或间接块
                  block_address(b)                    命中 1 块缓存就直接用
                  +-- t113_rootdev_read(2b, 2, block_cache)
```

和 xv6 的 namex() 相比，minifs 只读，所以不需要写日志；VFS 层仍然提供 inode/dentry 缓存与引用计数。minifs 内部现有解析器仍从根目录解析累计路径，因为 T113 还没有每进程的当前目录 inode（getcwd 返回固定的 /）。

2.3.5 devfs：/dev 下的字符设备

```
驱动 probe 时                              用户或内核 open("/dev/console")
t113_chrdev_register(1, 0,                 t113_vfs_open("/dev/console", RDWR)
                     &console_fops)        +-- lookup_path: find_mount → "/dev" 挂载根 dentry
+-- chrdev_table[i] = { dev, fops }        |   +-- 分量 "console" → devfs_iops.lookup(dir, child)
t113_devfs_create_chrdev(                  |       +-- inode: i_rdev=MINIOS_MKDEV(1,0)
   "console", 1, 0)                        |           i_fop=def_chr_fops
+-- nodes[i] = { name, major, minor }      +-- file->f_op = inode->i_fop
                                           +-- chrdev_open(): 按 i_rdev 找 console_fops
                                               +-- file->f_op = console_fops
                                           t113_vfs_write(f, "hi", 2)
                                           +-- file->f_op->write() → console_write()
                                               +-- UART3 发送
```

devfs 的节点不在任何磁盘上，是驱动在内存里登记的表。这和 Linux 的 devtmpfs 同一个思路：驱动注册设备时，内核自动在 /dev 下建节点。xv6 的做法不同：设备节点是磁盘上的 inode（T_DEVICE，mknod("console", CONSOLE, 0) 由 init 创建），read/write 时按 ip->major 查 devsw[] 分派。

用户态 /init 打开 /dev/console，再用 dup() 建立 fd 0/1/2；所有控制台 I/O 都走 VFS → devfs → chrdev → UART，不再在系统调用层特判标准 fd。

2.3.6 与 xv6、Linux 的挂载方式对照

方面	xv6	Linux	T113 miniOS
根文件系统从哪来	固定 ROOTDEV（virtio 盘，整盘是一个 fs）	root= 参数，prepare_namespace() → mount_root()，常先经 initramfs	MBR 第 2 分区，t113_minifs_mount_rootdev()
何时挂载	第一个进程的 forkret() 里 fsinit(ROOTDEV)，因为读盘要 sleep	kernel_init 线程里，在 exec /sbin/init 之前	kernel_boot() 里，在创建 init 之前；读盘是轮询，不用 sleep
有没有挂载表	没有，只有一个文件系统	struct mount 树，vfsmount 挂在 dentry 上	vfs_mount 父子关系、mountpoint dentry，路径选择暂兼容最长前缀
第二个文件系统	不支持	任意	/boot FAT32、/dev devfs
设备文件	磁盘上的 T_DEVICE inode + devsw[major]	devtmpfs，驱动注册时建节点	devfs，驱动注册时登记
挂载失败	panic("invalid file system")	VFS: Unable to mount root fs panic	kernel_platform_panic("cannot mount root filesystem")
用户态 mount	无	mount(2)	无

2.3.7 T113 实现上的区别

- 两个文件系统共用一个 SD 控制器（SMHC0）和一张卡。rootdev.c 不经 bio.c；fat32.c 经 bio.c。两条路径都在 t113_sd_read/write 里的 sd_lock 汇合。
- VFS 已建立 super_block → dentry → inode → file 对象链、引用计数和 inode/ dentry 缓存，并让子 mount 保存父 mount 与 mountpoint。minifs/ FAT32/ devfs 已直接提供各自的 inode_operations/ file_operations；普通文件和设备文件最终都由 file->f_op 分派。
- 卡上的 SPL、bootpack、两个分区由同一个 make install 写入；分区 2 必须足够放下 minifs.img（⩾ 32 MiB）。

2.4 VFS：super_block、inode、dentry

VFS 已改成 Linux 式的对象模型，完整说明见第 9 章。这里只列和挂载、根文件系统直接相关的几点：

miniOS	Linux
`vfs_mount` + `parent/mountpoint/mnt_root`	`struct mount` / `vfsmount`
`super_block` + `super_operations`	`struct super_block`
`dentry_cache[128]`、`inode_cache[128]` 与引用计数	dcache / icache
`file` + `file_operations` + `f_pos_lock`	`struct file` + `f_count`
全局 `vfs_lock` 一把互斥	per-inode 锁 + RCU

- **挂载**：`t113_vfs_mount_fs(path, magic, fs_info, s_op, root_iop, root_fop)` 为每个文件系统建一个 `super_block`、一个根 inode 和一个根 dentry；不是 `/` 时还要先在父文件系统里 `lookup_path(path)` 找到挂载点目录，所以 minifs 镜像里必须有 `/boot`、`/dev` 两个空目录（`mkfs_minifs` 已经建好）。
- **选文件系统**：仍按最长前缀匹配（`/bootx` 不会落到 `/boot`），然后从那个挂载的根 dentry 开始逐级建 dentry。
- **原生操作表**：路径和元数据操作走 `inode->i_op`；打开后先从 `inode->i_fop` 初始化 `file->f_op`。字符设备的 `chrdev_open()` 再把 `file->f_op` 换成 UART、GPIO key 等驱动的操作表。
- **身份**：minifs 的 inode 号、FAT32 的 `fat_inode` 槽位地址、bootfs 的 DDR 地址、devfs 的设备号成为 `inode->i_ino`；同一文件多次打开共享一个 inode。
- 读写带 `f_pos` 语义，但**没有 lseek**；标志支持 `VFS_O_RDONLY` / `VFS_O_RDWR`，外加 FAT32 用的 `VFS_O_CREATE` / `VFS_O_TRUNC`（值取自 xv6 `fcntl.h`）。`write` 由 devfs 和 FAT32 实现；minifs 和 bootfs 只读。
- `minifs_vfs_lookup()` 同时返回目录和普通文件，因此 `/boot`、`/dev`、`/bin` 可以参与逐级查找和挂载点验证。

2.5 rootdev：MBR 分区 2 块设备

`src/kernel/fs/rootdev.c` + `src/drivers/mmc/sunxi-sd.c`：

- `sunxi-sd.c` 是 **SPL 和内核共用的一份源码**（`MINIOS_KERNEL` 宏区分是否加 mutex）：SMHC0（`0x04020000`，PF0..PF5，1-bit 总线），CMD17 单块读、轮询、CPU 搬 FIFO。SPL 用它读 bootpack/FAT32，内核用它读根文件系统——同一张卡的同一个控制器。
- `t113_rootdev_init()`：读 LBA 0，验证 `0x55 0xAA` 签名，取 MBR 偏移 `446+16` 的第二个分区表项：type（偏移 4）、起始 LBA（+8）、扇区数（+12），type 非零、LBA 非零、$\geqslant 4$ 扇区才接受。
- `t113_rootdev_read(sector, count, dst)` 做范围检查后转 `t113_sd_read(root_lba + sector, ...)`。

2.6 minifs：xv6 磁盘 ABI 的只读实现

`tools/mkfs_minifs.c`（主机侧）明确保留 xv6 mkfs 的磁盘格式，内核侧 `src/kernel/fs/minifs.c` 是只读遍历实现。常量与 xv6 一致：

常量	值	说明
MINIFS_BSIZE	1024	块大小（= 2 个 SD 扇区）
MINIFS_MAGIC	0x10203040	xv6 FSMAGIC
MINIFS_ROOTINO	1	根目录 inode 号
MINIFS_NDIRECT	12	直接块
MINIFS_DIRSIZ	30	文件名上限
dinode	64 B	type/major/minor/nlink/size/addrs[13]
dirent	32 B	u16 inum + name[30]

镜像布局（FSSIZE=32768 块 = 32 MiB，NINODES=200）：

```
块 0          空（xv6 惯例留空）
块 1          superblock
块 2..31      log 区（30 块，只读实现不使用，格式保留）
块 32..44     inode 区（200/16+1 = 13 块，每块 16 个 dinode）
块 45..49     位图（32768/8192+1 = 5 块）
块 50..       数据区（含目录内容）
```

mkfs 产出的目录树（miniOS/Makefile 打包 _init _sh _ls _echo _pwd _cat _sleep _uptime）：

```
/              ino 1
+-- bin/       sh, ls, echo, pwd, cat, sleep, uptime
+-- dev/       空目录（minifs 里的占位；真设备在 devfs）
+-- init       与 /bin/init 同一 inode（nlink=2，硬链接）
```

单文件上限 = 12 直接块 + 256 间接块 = 268 块 = 268 KiB；xv6 风格目录 inode 的 size 被取整到完整块（round_directory()）。

2.6.1 两种挂载方式

函数	数据来源	用途
t113_minifs_mount_vfs(image, bytes)	DDR 里的镜像（minifs_image.o 可 objcopy 进内核）	备用路径；T113 当前未链接
t113_minifs_mount_rootdev()	MBR 分区 2，按需读块	**当前使用**

块设备模式的关键是**单块缓存**：block_cache[1024] + cached_block 只缓存最近一块。block_address(block) 命中缓存直接返回指针，未命中则 t113_rootdev_read(block*2, 2, cache) 读两个扇区。这等于一个只有一项的 page cache——读 /init 这种顺序小文件足够，遍历目录时 superblock（块 1）和 inode 区会反复换入换出，每读一个 dirent 的 inode 都可能多一次 SD 读。xv6 的 buffer cache 是 30 块 LRU，这里刻意省掉了。

读路径（无 dcache、无 icache，每次 lookup 全程走磁盘）：

```
namei("/bin/sh")
+-- 逐分量 directory_lookup()
    +-- read_inode(dir_inum)                块 = inodestart + inum/16
    +-- inode_read() 逐 dirent              inode_block(): 前 12 块直接，之后查间接块
    +-- names_equal(disk_name, component)   名字 ≤30 字节，NUL/长度双判
```

2.7 bootfs：/boot 的 FAT32 交接

笔记 现在只是后备路径：正常情况下 /boot 由 fat32.c 挂载，见第 3 章。

背景：XR829 三个固件文件现在放在 FAT32 分区里（方便换固件不用重烧 bootpack），但内核的 minifs 是只读小文件系统、rootdev 只认分区 2。解法是分工：**SPL 读 FAT32（fat32.c，MBR + BPB + 簇链，支持长文件名），内核只认 DDR 里已经就位的镜像**。

SPL 把固件装到 DDR 固定地址后，向 bootpack 头补写三个条目（offset=0 表示"常驻镜像，非 pack 内偏移"），重算头 CRC：

文件	bootpack 条目	DDR 地址	容量上限
boot_xr829.bin	boot_xr829	0x40100000	64 KiB
fw_xr829.bin	fw_xr829	0x40110000	192 KiB
sdd_xr829.bin	sdd_xr829	0x40140000	64 KiB

内核侧 bootfs.c 把这三条暴露成普通文件：

- lookup("/boot/fw_xr829.bin") → t113_bootpack_find("fw_xr829")，node.id = load_addr（物理地址当文件 id）；
- read() 就是 memcpy——数据已在 DDR，零拷贝语义。

XR829 驱动的 need_image() 因此从硬编码 bootpack 查询改成了标准 VFS 调用（t113_vfs_open("/boot/fw_xr829.bin") →读到页分配的缓冲），驱动不再依赖 bootloader 私有的地址常量。若 FAT32 分区缺失，SPL 回退使用 bootpack 内打包的固件（老路径仍保留）。

2.8 devfs：/dev 设备文件

src/kernel/fs/devfs.c 对应 Linux 早期的 devfs（现代等价物是 devtmpfs）：

- t113_devfs_init()：挂到 /dev，16 个节点槽，设备名 ⩽ 30 字节且不含 /；
- 驱动 probe 时做两件事：t113_chrdev_register(major, minor, ops) 把设备号和驱动操作登记进 chrdev.c 的 chrdev_table[32]（ops 至少要有 read 或 write，设备号去重）；t113_devfs_create_chrdev(name, major, minor) 在 /dev 下建名字（名字去重）。这和 Linux 的 register_chrdev_region + cdev_add、再由 devtmpfs 建节点是同一个分工；
- lookup("/dev/console") 建立字符设备 inode：i_rdev = (major<<16)|minor，i_fop = def_chr_fops；readdir("/dev", i) 枚举已注册设备；
- def_chr_fops.open() 按 i_rdev 查 chrdev_table，把本次打开的 file->f_op 换成驱动 file_operations。之后 read/write 直接进入驱动，VFS 对字符设备不推进 f_pos。

当前唯一注册者是 UART 平台驱动（uart.c probe）：console（major 1, minor 0）和 ttyS0（major 3, minor 0），major 取值沿 Linux 惯例（1 = 内存/终端类，3 = 第一个 8250 串口）。注册时序有要求：kernel_boot() 里的 t113_devfs_init() 必须早于驱动绑定 t113_driver_core_init()，否则 probe 里的注册会失败。

用户态 stdio：/init 第一件事是 open("/dev/console", O_RDWR) 得到 fd 0，再 dup(0) 两次得到 fd 1、2；syscall.c 不再特判这三个 fd。shell 继承的 stdin/stdout 就是这个 devfs 文件，三个 fd 共享同一个 struct file。另一个注册者是 drivers/input/gpio_keys.c 的 /dev/key0。

2.9 用户态视角：从 /init 到 shell

系统调用到 VFS 的映射（src/kernel/syscall.c，每进程 files[T113_MAX_FDS=16]，fd 0/1/2 由 /init 打开 /dev/console 建立）：

系统调用	内核路径	备注
open(path, flags)	sys_open → t113_vfs_open	用户指针逐字节经 user_vm_range_mapped 校验拷入
read(fd, buf, n)	t113_vfs_read	控制台经 devfs → chrdev → UART
write(fd, buf, n)	t113_vfs_write	单次 ⩽ 256 B
dup(fd)	t113_vfs_file_get	新 fd 共享同一个 struct file
close(fd)	t113_vfs_close	引用计数归零才释放槽位
readdir(path, i)	t113_vfs_readdir	ls 的实现基础
exec(path, argv)	sys_exec → t113_vfs_open + ELF 加载	镜像 ⩽ 16 页（64 KiB）
getcwd	常量 "/"	尚无每进程 cwd，pwd 是假的

内核加载 /init 本身（kernel_main 尾部）就是一次普通 VFS 读：open →按文件大小 alloc_pages → read → create_init_user_process →释放临时页。/init 再 fork() + exec("/bin/sh")，shell 的每次命令执行又是一次 exec → VFS → minifs 读块。整条用户态生命线都吊在 rootdev 的 CMD17 上。

2.10 与 Linux / xv6 的对照

miniOS	xv6	Linux
vfs_mount 树 + 前缀匹配	无（只有一个 fs）	挂载树 + namespace
super_block / inode / dentry 对象池	itable inode 缓存，无 dentry	super_block、icache、dcache
原生 inode_operations / file_operations	无，直接调 fs 函数	inode_operations + file_operations + super_operations
rootdev（MBR 分区）	ide 驱动	块设备层 + 分区扫描（read_partition）
minifs 只读	xv6 fs（含 log 写入）	ext4 等
单块缓存	30 块 buffer cache	page cache
/boot FAT32（失败时退回 bootfs）	—	/boot 分区 + firmware loader
devfs + chrdev_register	mknod + devsw[]	devtmpfs + device_create（+ udev 用户态改名）
kernel_main 读 /init	第一个进程硬编码在 initcode.S	initramfs 的 /init → exec /sbin/init
PANIC + wfi	panic()	panic()

2.11 构建与烧写

```
make t113                    # 内核 + minifs.img（mkfs_minifs 打包 init/sh/ls/...）
make install T113_DISK=/dev/diskX
# 内部依次：
```

```
#   dd minios-t113.bin  @ 8 KiB    (Boot0)
#   dd bootpack.bin     @ 128 KiB  (kernel)
#   cp firmware/xr829/*.bin → /Volumes/bootfs   (FAT32, install-firmware)
#   dd minifs.img → /dev/diskXs2                (install-rootfs)
```

换固件只动 FAT32 分区；改用户程序（`user-t113/*.c`）只需重跑 `install-rootfs`，内核和 bootpack 不用重烧。

2.12 预期串口输出

（LBA/大小以实际卡为准，下面是格式示例）

```
SPL: SDHC card ready
SPL: load kernel -> 0x40200000 bytes=0x?????? ok
SPL: FAT32 partition LBA=0x00002000
SPL: FAT32 load boot_xr829.bin bytes=0x888
SPL: FAT32 load fw_xr829.bin bytes=0x2aa20
SPL: FAT32 load sdd_xr829.bin bytes=0x2e8
...
bootpack: kernel @0x40200000 bytes=0x????????
bootpack: boot_xr829 @0x40100000 bytes=0x00000888
bootpack: fw_xr829 @0x40110000 bytes=0x0002aa20
bootpack: sdd_xr829 @0x40140000 bytes=0x000002e8
rootdev: MBR partition 2 type=0x83 lba=0x?????? sectors=0x??????
minifs: mounted block rootfs blocks=0x8000 inodes=0xc8
fat32: /boot mounted rw, partition lba=0x???????? clusters=0x???????? cluster-bytes=0x????????
...
init: starting shell on console
```

`rootdev:`、`minifs:` 先出现，然后才是 `fat32:`：`kernel_boot()` 先挂 `/`（它顺带初始化 SD 卡），再挂 `/boot`。FAT32 挂载失败时会多一行 `fat32: mount failed, using SPL handoff images`，接着是 `bootfs: FAT32 handoff mounted at /boot`。devfs 挂载没有打印，驱动绑定后 `platform_dev_info` 会打印 uart 绑定信息。

第三章 /boot：从 bootpack 视图到可读写 FAT32

本章说明 /boot 的两代实现：改动前的 bootfs（bootpack 交接视图）和现在的 FAT32 文件系统（xv6 风格的 bio + fat32 分层），以及它们和 SPL、XR829 固件加载、tftp 的关系。根文件系统 minifs、VFS 挂载表、devfs 的整体结构见第 2 章。

一句话总结：

- **改动前**：读 FAT32 的是 SPL，内核里的 /boot 只是把 SPL 预先搬进 DDR 的三份固件包装成文件，只读、只认三个写死的路径、不能列目录。
- **改动后**：内核自己读写 SD 卡上的 FAT32 分区，/boot 是真正的文件系统，可以按文件名查找、列目录、创建和写入文件；tftp 下载的文件直接落盘到卡上。老的 bootfs 只在 FAT32 挂载失败时作为后备。

3.1 改动前：bootfs 只是 bootpack 的视图

3.1.1 加载流程

```
miniBoot SPL  load_bootpack()
+-- 读 SD 128 KiB bootpack，装 kernel → 0x40200000
+-- bootpack header 拷到 0x401ff000
+-- load_fat32_firmware()                SPL 里的只读 FAT32 解析器
    +-- fat32_mount()                    MBR 找 FAT32 分区，解析 BPB
    +-- fat32_load_root() × 3            只在根目录按名字找，沿簇链读入 DDR
    |   +-- boot_xr829.bin → 0x40100000
    |   +-- fw_xr829.bin   → 0x40110000
    |   +-- sdd_xr829.bin  → 0x40140000
    +-- bootpack_slot()                  往 header 补 3 个条目（offset=0 表示常驻镜像），重算 CRC

miniOS 内核  bootfs.c（63 行）挂在 /boot
+-- lookup("/boot/fw_xr829.bin")
|   +-- 路径写死映射 → t113_bootpack_find("fw_xr829")
|       → node.id = load_addr（DDR 物理地址当文件 id）
+-- read() = 从 node.id + offset 处 memcpy
```

3.1.2 本质

bootfs 本质上是 **bootpack 解析器**，挂到 /boot 只是借用 VFS 的接口，让 XR829 驱动可以用 t113_vfs_open("/boot/fw_xr829.bin") 拿固件，而不用知道 bootloader 私有的地址常量。它不是文件系统：

能力	bootfs	原因
按文件名查找	×	entry_name() 只认三个硬编码路径，映射到 bootpack 条目名
列目录（ls /boot）	×	readdir = 0
读任意文件	×	只有 SPL 预加载的三份固件
写入 / 创建	×	没有 write、create
看到运行期卡上的变化	×	数据是启动时的 DDR 快照，内核从不访问 FAT32 分区

所以当时 tftp 之类“把文件存进 /boot”的功能无从实现。

3.2 改动后：分层结构

照 xv6 的 bio.c / fs.c 分层：

```
用户态  open(O_CREAT|O_TRUNC) / read / write / readdir / tftp
   |
VFS     src/kernel/fs/vfs.c        挂载表最长前缀匹配，/boot → fat32_ops
   |
fat32   src/kernel/fs/fat32.c      路径 → 目录项，文件偏移 → 簇链；inode 表
   |
bio     src/kernel/fs/bio.c        512 B 扇区缓存，24 块 LRU，写直达
   |
SD      src/drivers/mmc/sunxi-sd.c t113_sd_read / t113_sd_write（CMD17/CMD24，轮询）
```

3.2.1 挂载顺序

src/kernel/init/main.c 的 kernel_boot()：

```
t113_vfs_init();
/* The root device initialises the SD card that /boot shares. */
if (t113_minifs_mount_rootdev() < 0)        /* 先挂 /，顺带初始化 SD 卡 */
    kernel_platform_panic("cannot mount root filesystem");
if (t113_fat32_mount_vfs() < 0) {           /* 再挂 /boot（FAT32 读写） */
    t113_uart_puts("fat32: mount failed, using SPL handoff images\n");
    if (t113_bootfs_mount_vfs() < 0)        /* 后备：老的 bootpack 视图 */
        kernel_platform_panic("cannot mount boot filesystem at /boot");
}
if (t113_devfs_init() < 0) ...
```

与之前相比，/ 改为先挂（rootdev 里会做 t113_sd_init()，/boot 和 / 共用同一张卡、同一个 SMHC0 控制器）。两者都在调度器启动前完成，此时互斥锁实际不生效，SD 访问是串行的。

3.3 bio：扇区缓存（bio.c）

项	值
块大小 BSIZE	512 B（= SD 扇区，块号就是卡上的绝对 LBA）
缓冲数 NBUF	24
替换策略	双向链表 LRU，brelse 后放到表头
写策略	**写直达**：bwrite() 立刻 t113_sd_write()
对齐	data[] 4 字节对齐（SD 驱动用 CPU 搬 FIFO，按字访问）

接口和 xv6 一致，多了一个 bget_blank()：

函数	作用
bread(lba)	返回带引用的缓冲，未命中时从卡上读
bget_blank(lba)	返回带引用的缓冲但**不读卡**，给“整块覆盖”的调用者（写满一个扇区、清零新簇）省一次读
bwrite(b)	写回卡上
brelse(b)	释放引用，缓冲留在缓存里

同一块的并发访问由上层串行化（fat32 每个操作都持有自己的锁），所以 bio 不像 xv6 那样给每个 buf 配 sleeplock。

因为是写直达，卡上的内容永远不比缓存旧，fat32 读大段数据时可以绕开缓存直接读卡（见下文“读路径”）。

3.4 fat32（fat32.c）

3.4.1 挂载 t113_fat32_mount_vfs()

1. binit()，bread(0) 读 MBR，验 55 AA，取**第一个**类型为 0x0B/0x0C/0x1B/0x1C（FAT32 CHS/LBA，含隐藏变体）的分区表项。
2. 读分区首扇区（BPB），校验：55 AA、每扇区 512 字节、每簇扇区数非零、FAT 个数非零、根目录项数为 0（FAT32 特征）、每 FAT 扇区数和总扇区数非零。
3. 算出布局：

```
part_lba
+-- 保留区（含 BPB、FSInfo）          fat_lba  = part + BPB[14] 保留扇区数
+-- FAT × fats                       data_lba = fat_lba + fats × sectors_per_fat
+-- 数据区                           簇 n 的 LBA = data_lba + (n-2) × spc
```

4. 簇数 = (总扇区 − 元数据扇区) / spc，并用 FAT 表能描述的条目数封顶；根目录簇号必须合法。
5. 挂到 /boot，打印 fat32: /boot mounted rw, partition lba=…clusters=…cluster-bytes=…。

3.4.2 内存中的 inode 表

FAT32 没有 inode，文件的元数据（首簇、大小）存在目录项里。为了对齐 xv6 的 icache，fat32.c 维护一个 16 项的 fat_inode 表：

```
struct fat_inode {
    uint32_t ref;
    uint32_t dir;          /* 1 for a directory */
    uint32_t first;        /* first cluster, 0 for an empty file */
    uint32_t size;
    uint32_t ent_lba;      /* sector of the short entry, 0 for the root */
    uint32_t ent_off;      /* byte offset of that entry in the sector */
    uint32_t dirty;        /* first/size changed: rewrite the entry */
    uint32_t cur_index;    /* last cluster looked up: index in chain... */
    uint32_t cur_cluster;  /* ...and its number (0 = none) */
};
```

- **身份** = 短目录项所在的 (ent_lba, ent_off)，同一个文件被打开两次会拿到同一个 inode（iget 按位置查找，引用计数 +1）。

- vfs_node.private_data 指向这个 inode，VFS 后续的 read/write/close 都通过它找到文件。
- cur_index / cur_cluster 缓存上一次查到的簇，顺序读写时沿簇链只往前走一步，不用每次从首簇开始数（簇链是单向链表）。
- dirty：写入导致首簇或大小变化时置位，**最后一次 close 时** iupdate() 把首簇（高 16 位 @20、低 16 位 @26）和大小（@28）写回目录项。

3.4.3 路径解析

resolve(path) 先用 mount_relative() 去掉 /boot 前缀，然后逐个分量在目录簇链里用 dir_find() 查找：

- 目录遍历器 dir_next() 同时拼接长文件名（LFN）片段，并用短名校验和（short_checksum()）确认 LFN 和后面的短目录项属于同一个文件，否则退回用 8.3 短名。
- 名字比较不区分大小写；短名会按 NT 保留字节（byte 12）的大小写标志还原成小写。
- 支持任意层子目录；.. 指向根目录时首簇为 0，按根目录簇处理。
- 根目录本身用 lba = 0 表示（它没有目录项）。

3.4.4 读路径

```
fat32_read(node, offset, dst, count)
+-- 循环
    +-- ibmap(ip, pos / 簇大小, alloc=0)   沿簇链找到所在簇
    +-- 扇区对齐 + 剩余 ⩾ 512 + dst 4 字节对齐
    |   +-- t113_sd_read(lba, n, dst) 直接读进调用者缓冲（一次读完簇内剩余整扇区）
    +-- 否则 bread(lba) → copy 部分扇区 → brelse
```

读大文件（固件、内核镜像）时绝大多数数据走直读，不占缓存也不多拷一次。

3.4.5 写路径

```
fat32_write(node, offset, src, count)
+-- offset > size → 拒绝（不支持稀疏文件）
+-- 循环
    +-- ibmap(ip, pos / 簇大小, alloc=1)   不够就 fat_alloc() 接到链尾
    +-- 写满一个扇区 → bget_blank(); 否则 bread()（读改写）
    +-- copy → bwrite()（写直达）→ brelse
    +-- 文件变长 → ip->size 更新, dirty = 1
```

- fat_alloc(prev)：从 free_hint 开始线性扫 FAT 找值为 0 的簇，标成链尾 0x0FFFFFFF，再把 prev 指向它。
- fat_set() 同时写**所有 FAT 副本**（通常 2 份），保持一致，Windows 和 macOS 的 fsck 不会报 FAT 不一致。
- 写 FAT 项只改低 28 位，保留高 4 位（规范要求）。
- **FSInfo**：第一次分配簇之前把 FSInfo 里的“空闲簇数 / 下一个空闲簇”标成未知（0xFFFFFFFF），让其它系统下次挂载时自己重新计算。这样内核就不需要维护精确的空闲计数。

3.4.6 创建 fat32_create()（open 带 O_CREAT）

1. resolve(path, parent=1) 找到父目录和最后一个分量。
2. 文件已存在就直接打开；否则 create_entry()：

- 名字能直接放进 8.3（`fit_short()`，大写字母、合法字符、基名 ≤ 8、扩展名 ≤ 3；全小写的名字用 NT 大小写标志记录）→只写一个短目录项；
- 否则生成别名 `NAME~N.EXT`（`make_alias()`，N 从 1 递增直到不冲突），再写 ⌈ 长度/13⌉ 个 LFN 项 + 1 个短目录项，LFN 项带短名校验和。
- `find_slots()` 在父目录里找足够的连续空位（`0x00` / `0xE5`），不够就给目录追加一个清零的新簇。
- 创建日期写固定的 2026-01-01（`0x5c21`），没有 RTC。

3. 新文件首簇为 0、大小为 0，第一次写入时才分配簇。

3.4.7 截断 `fat32_truncate()`（open 带 `O_TRUNC`）

顺序是**先改目录项、再释放簇链**：首簇和大小清零→ `iupdate()` 写回→ `fat_free_chain()`。如果中途掉电，最坏情况是留下一条无人引用的簇链（fsck 能回收），而不会出现目录项指向已经释放的簇。

3.4.8 列目录 `fat32_readdir()`

跳过隐藏项、卷标、. 和 ..；类型取值和 minifs 一致（1 = 目录，2 = 文件），所以现有的 `ls` 不用改。`vfs_dirent.name` 只有 31 字节，超过 30 个字符的长文件名在 `ls` 里会被截断显示（打开文件时用完整名字没问题）。

3.4.9 不支持

`mkdir`、`unlink`、`rename`、时间戳更新、稀疏文件、多分区（只认 MBR 里第一个 FAT32 分区）、FAT12/16、exFAT。

3.5 VFS 的配套改动

include/fs/vfs.h:

```
#define VFS_O_ACCMODE 3u
#define VFS_O_CREATE 0x200u      /* xv6 fcntl.h values */
#define VFS_O_TRUNC  0x400u

struct vfs_backend_ops {
    ...
    /* Optional: create an empty regular file (O_CREATE). */
    int (*create)(const char *path, struct vfs_node *node);
    /* Optional: discard a file's contents (O_TRUNC). */
    int (*truncate)(struct vfs_node *node);
    /* Optional: the last reference to an open node is gone. */
    void (*close)(struct vfs_node *node);
};
```

`t113_vfs_open()` 的流程:

```
find_mount(path)                    最长前缀匹配
+-- lookup(path) 失败
|   +-- 带 O_CREATE 且后端有 create → create(path)
+-- 带 O_TRUNC (必须是 O_RDWR) → truncate(node)
+-- 分配 open_files 槽位
t113_vfs_write()   成功后更新 file->node.size
```

```
t113_vfs_close()   引用计数归零 → ops->close(node)   fat32 在这里写回目录项
```

三个新回调都是可选的：minifs、devfs、bootfs 不实现它们，行为不变（对它们带 O_CREATE 会失败）。

用户态：fcntl.h 增加 O_CREATE / O_CREAT（0x200）和 O_TRUNC（0x400），sys_open 接受这两个标志。

3.6 谁在用 /boot

3.6.1 XR829 固件

驱动代码没有改：need_image()（现在在 xradio/fwio.c）一直是

```
struct vfs_file *file = t113_vfs_open(
    "/boot/fw_xr829.bin", VFS_O_RDONLY);
```

变的是 VFS 下面的后端。现在这个调用由 fat32 处理：在 FAT32 根目录里按名字查找，沿簇链从卡上读出。也就是说固件加载现在走的是真正的文件系统查找；只有 FAT32 挂载失败时才会退回 bootfs 的 DDR 视图。

注：SPL 里的 load_fat32_firmware() 仍然会把三份固件预读进 DDR，内核正常路径已经不依赖它，只给 bootfs 后备路径用。上板确认 FAT32 稳定后可以删掉这段，启动会快一点。

3.6.2 tftp

src/net/tftp/tftp_udp.c 的 tftp_get()：先把整个文件收进内存（最多 8 MiB），传输完成后才 t113_vfs_open(local, O_RDWR | O_CREATE | O_TRUNC) 写入，关闭后再重新打开读回、比对 CRC32。传输中断不会在 /boot 留下写了一半的文件。用户命令 tftp < 服务器 IP> < 远端文件 > [本地路径]，本地路径默认 /boot/< 文件名 >。

3.6.3 WiFi 配置

supplicant（src/net/wpa/supplicant.c 的 wpa_supplicant_connect()）从 /boot/wifi.txt 读 ssid=、psk=（以及可选的 ip=、netmask=、gateway=）。密码只放在 FAT32 分区上，不进源码；仓库里对应的 rpi_bm/firmware/wifi.txt 被 .gitignore 忽略。

3.6.4 SPL 从 FAT32 加载内核

miniBoot spl.c 新增 load_fat32_kernel()：SD 初始化后先在 FAT32 根目录找 kernel_minios.img（最大 8 MiB），读到 0x40200000，打印大小和 CRC32；找到了就跳过 bootpack 里的 kernel 条目，找不到就按原来的 bootpack 流程走。配合 tftp，可以在板子上直接更新内核：

```
tftp 192.168.0.100                    # 写入 /boot/kernel_minios.img
# 重启后 SPL 加载新内核
```

想回到 bootpack 里的内核，删掉 FAT32 分区上的 kernel_minios.img 即可。

3.7 FAT32 图解（参照 xv6 book 第 8 章）

xv6 book 讲文件系统时，先给一张分层图，再给磁盘布局，然后逐层讲 buffer cache、inode、目录、路径名，最后串起 open/read/write。本节按同样的顺序讲 miniOS 的 FAT32，每一层都和 xv6 的对应层并排比较。

3.7.1 分层

```
       xv6 文件系统 (7 层)                    miniOS /boot (T113)
+-------------------------------+      +--------------------------------------+
| 文件描述符  file.c sysfile.c  |      | task->files[4] → struct vfs_file     | vfs.c
+-------------------------------+      +--------------------------------------+
| 路径名      namei / namex     |      | find_mount() 选后端 (最长前缀)       | vfs.c
+-------------------------------+      | resolve() 逐级 dir_find()           | fat32.c
| 目录        dirlookup/dirlink |      +--------------------------------------+
+-------------------------------+      | 目录项 32 B 短名 + LFN 长名          | fat32.c
| inode       iget/ilock/bmap   |      | fat_inode (内存) + ibmap() 走簇链    | fat32.c
+-------------------------------+      +--------------------------------------+
| 日志        begin_op/end_op   |      | (无日志: 直写 + 写入顺序约定)        |
+-------------------------------+      +--------------------------------------+
| buffer cache  bread/bwrite    |      | bio.c  24 个 512 B 扇区缓冲, LRU     | bio.c
+-------------------------------+      +--------------------------------------+
| 磁盘        virtio_disk.c     |      | t113_sd_read/write  SMHC0 CMD17/CMD24 | sunxi-sd.c
+-------------------------------+      +--------------------------------------+
```

xv6 的 inode 层同时负责“文件数据在哪些块”（`addrs[]` 加间接块）和“文件元数据存在哪”（磁盘 inode 区）。FAT32 把这两件事拆开了：

- 数据块的位置记在 FAT 表里，一簇接一簇连成链表；
- 文件大小和首簇号记在父目录的 32 字节目录项里。

所以 miniOS 的 `fat_inode` 只是内存里的一个句柄，记住“我的目录项在磁盘哪个位置”，关闭文件时把首簇和大小写回去。

3.7.2 磁盘布局

整张卡和 FAT32 分区内部：

```
整张 SD 卡 (512 B 扇区, LBA = 扇区号)
LBA 0          MBR: 446 B 引导码 | 4 × 16 B 分区表 | 55 AA
               表项 1  type 0x0b/0x0c   → FAT32  → 挂到 /boot
               表项 2  type 0x83        → minifs → 挂到 /
LBA 16         Boot0 SPL (8 KiB 处)
LBA 256        bootpack (128 KiB 处)
part_lba       FAT32 分区开始 -------------+
                                           v
part_lba   +---------------------------------+ +
           | 扇区 0  引导扇区 (BPB)           | |
           | 扇区 1  FSInfo                  | | 保留区: BPB_RsvdSecCnt 个扇区
           | 扇区 6  备份引导扇区             | | (常见 32)
           | ...                             | |
fat_lba    +---------------------------------+ + = part_lba + RsvdSecCnt
           | FAT #1                          |   sectors_per_fat 个扇区
           | 每个簇一项, 4 字节, 低 28 位有效 |   一个扇区放 128 项
           +---------------------------------+
           | FAT #2 (镜像)                    |   fat_set() 两份都写
data_lba   +---------------------------------+ = fat_lba + fats × sectors_per_fat
           | 簇 2   根目录通常从这里开始      |   cluster_lba(c) = data_lba + (c - 2) × spc
           | 簇 3                            |
           | ...                             |
           | 簇 clusters+1                   |
           +---------------------------------+
```

xv6 的 mkfs 布局是连续的几段，块号由超级块给出：

```
xv6 (BSIZE = 1024)
[ 0 boot | 1 super | 2.. log | inodestart.. inode 块 | bmapstart.. 位图 | 数据块 ... ]
```

两者对照：

xv6	FAT32	miniOS 里在哪
超级块 `struct superblock`	引导扇区里的 BPB	`t113_fat32_mount_vfs()` 解析到全局 `fs`
空闲块位图	FAT 表项为 0 表示空闲	`fat_alloc()` 从 `free_hint` 线性扫描
磁盘 inode 区	没有，元数据在目录项里	目录项的 `ent_lba/ent_off`
`addrs[12]` + 1 个间接块	FAT 链表，长度不限	`ibmap()`
日志区	没有	写入顺序约定（见后文）

`t113_fat32_mount_vfs()` 用到的 BPB 字段：

偏移	长度	字段	校验 / 用途
11	2	BytsPerSec	必须等于 512（BSIZE）
13	1	SecPerClus	`fs.spc`，非 0
14	2	RsvdSecCnt	`fat_lba = part_lba + RsvdSecCnt`
16	1	NumFATs	`fs.fats`，非 0
17	2	RootEntCnt	必须为 0，这是 FAT32 的标志
32	4	TotSec32	算数据簇数 `clusters`
36	4	FATSz32	`sectors_per_fat`
44	4	RootClus	`root_cluster`，必须是合法簇号
48	2	FSInfo	FSInfo 扇区号，0 表示没有
510	2	签名	55 AA

簇数还会再被 FAT 表的容量截一次：`clusters ⩽ sectors_per_fat × 128 - 2`。有些格式化工具算出来的 FAT 偏小，截一下能防止 `fat_get()` 越过 FAT 表。

3.7.3 FAT 表与簇链

FAT 表就是一个大数组，下标是簇号，值是“下一个簇号”：

```
目录项: name = "fw_xr829.bin"  first = 5  size = 174624

FAT[]  下标:  2    3    4    5    6    7    8    9    10   11
       值:   EOC  EOC  0    6    7    9    0    10   EOC  0
                       空闲  |    |    |    空闲  |    |    空闲
                             +-→ 6 +-→ 7 +---------→ 9 +-→ 10 → EOC

文件第 k 个簇 = 从 first 出发沿链走 k 步
偏移 off 所在的簇序号 = off / (spc × 512)
扇区 = cluster_lba(簇号) + (off % 簇字节) / 512
```

（簇号只是示意。）表项取值：

值（低 28 位）	含义
0	空闲
2 …clusters+1	下一个簇
⩾ 0x0ffffff8	链尾 EOC，miniOS 写 0x0fffffff

ibmap(ip, index, alloc) 把“文件第 index 个簇”翻译成簇号，相当于 xv6 的 bmap()。差别在于：xv6 用下标直接查 addrs[]，O(1)；FAT 要从头走链，O(n)。miniOS 在 fat_inode 里放了一个游标 cur_index/cur_cluster，顺序读写时从上一次的位置接着走，所以整文件顺序读还是 O(n) 而不是 $O(n^2)$：

```
ibmap(ip, index, alloc)
+-- ip->first == 0（空文件）
|   +-- alloc ? first = fat_alloc(0) : 返回 0
+-- 起点：cur_index ⩽ index 时从游标出发，否则从 first 出发
+-- while i < index:
|   +-- next = fat_get(c)              bread(FAT 扇区)，取 4 字节
|   +-- next 不是合法簇
|   |   +-- 是 EOC 且 alloc → next = fat_alloc(c)   扩链
|   |   +-- 否则返回 0（越过文件末尾）
|   +-- c = next, i++
+-- 更新游标，返回 c

fat_alloc(prev)
+-- fsinfo_invalidate()              第一次分配时把 FSInfo 计数写成“未知”
+-- 从 free_hint 起环形扫描 FAT，找值为 0 的项
+-- fat_set(c, EOC)                   所有 FAT 副本
+-- prev ? fat_set(prev, c)          挂到链尾
+-- free_hint = c + 1
```

3.7.4 目录项

FAT32 的目录也是文件，内容是一串 32 字节目录项。一个 512 B 扇区放 16 项，一个簇放 spc × 16 项。短目录项（8.3）的格式：

```
偏移  长度  字段
 0     8   主名，空格填充      e[0] = 0x00 目录结束；0xE5 已删除；0x05 表示首字符是 0xE5
 8     3   扩展名
11     1   属性   0x01 只读 0x02 隐藏 0x04 系统 0x08 卷标 0x10 目录 0x20 归档；0x0F = LFN
12     1   NT 保留  bit3 主名全小写，bit4 扩展名全小写
13     1   创建时间（10 ms）
14     2   创建时间
16     2   创建日期              miniOS 新建文件写 2026-01-01
18     2   访问日期
20     2   首簇号高 16 位   +
22     2   修改时间         |
24     2   修改日期         |
26     2   首簇号低 16 位   + first = (hi << 16) | lo
28     4   文件大小（目录为 0）
```

长文件名（LFN）放在短目录项前面，每项 13 个 UTF-16 字符，按逆序存放：

```
磁盘顺序 →
+--------------+--------------+-----------------------+
| LFN  0x42    | LFN  0x01    | 短项 "MINIOS~1BIN"    |
| 字符 14..17  | 字符 1..13   | attr 0x20 first size  |
```

```
| attr 0x0F     | attr 0x0F     |                        |
| chksum = S    | chksum = S    | S = short_checksum(名) |
+---------------+---------------+------------------------+
  0x40 标记“最后一片”（也就是磁盘上第一项）

LFN 项内 13 个字符的字节位置：lfn_pos[] = {1,3,5,7,9, 14,16,18,20,22,24, 28,30}
校验和：sum = ((sum & 1) << 7) + (sum >> 1) + name11[i], i = 0..10
```

读目录时，`dir_next()` 是一个小状态机：

```
dir_next(it, d)
+-- 本簇的项读完 → it->cluster = fat_get(it->cluster)
+-- bread(cluster_lba(c) + index / 16)，拷出 32 字节，brelse
+-- e[0] == 0x00   → 目录结束，返回 0
+-- e[0] == 0xE5   → 已删除，丢弃攒了一半的长名
+-- attr == 0x0F   → lfn_piece()：0x40 项清空缓冲并记下 chksum，按 ord 填 13 个字符
+-- attr & VOLUME  → 卷标，跳过
+-- 短项
    +-- 长名完整且 chksum 匹配 → d->name = 长名
    +-- 否则 short_name()     → "FW_XR829.BIN"，按 NT 位转小写
    +-- d->first / size / lba / off，返回 1
```

新建文件时，名字能精确表示成 8.3 就只写一个短项。例如 `fw_xr829.bin` 存成 `FW_XR829BIN`，NT 字节写 0x18 表示主名和扩展名都是小写。表示不了时（太长、多个点、大小写混合等）才生成 `NAME~N.EXT` 别名并写 LFN 项：

```
create_entry(dir_cluster, "minios-kernel.bin")
+-- long_char_ok() 逐字符检查          不允许 \ / : * ? " < > |
+-- fit_short() 失败（主名 13 字符 > 8）
+-- make_alias()                       "MINIOS~1BIN"，~1 已存在就试 ~2 ...
+-- pieces = ceil(17 / 13) = 2
+-- find_slots(dir, 3)                 找 3 个连续空项；目录满了就 fat_alloc + zero_cluster 扩一簇
+-- write_entry() × 2                  LFN 0x42、0x01
+-- write_entry()                      短项：attr ARCHIVE，first = 0，size = 0
```

xv6 的目录项是 16 字节的 `struct dirent { ushort inum; char name[14]; }`，`inum` 指向 inode，长度和数据都在 inode 里。FAT 的目录项本身就是 inode。

3.7.5 内存中的数据结构

```
用户 fd / 内核指针
      |
      v
task->files[i] --→ struct vfs_file             open_files[32]，vfs.c
                   +-- references / flags / offset
                   +-- ops --→ fat32_ops
                   +-- node { id = first, size, type,
                              private_data } --+
                                               v
                             fs.inode[16]：struct fat_inode      fat32.c
                             +-- ref                    同一目录项只占一个槽（iget 去重）
                             +-- dir / first / size
                             +-- ent_lba / ent_off --→ 磁盘上短目录项的位置（根目录为 0）
                             +-- dirty                  first/size 改过，close 时 iupdate()
                             +-- cur_index / cur_cluster  簇链游标
                                              | fat_get / bread / bwrite
                                              v
```

```
bcache: struct buf[24]                bio.c
head <-> 最近释放 <-> ... <-> 最久未用 <-> head
+-- bget: 正向找命中（valid && blockno 相同）
|         反向找 refcnt == 0 的回收
+-- bread: 未命中时 t113_sd_read(blockno, 1)
+-- bwrite: 立刻 t113_sd_write（直写）
+-- brelse: refcnt 归 0 时移到 head.next
                  |
                  v
sunxi-sd.c: SMHC0, PF0–PF5, 1-bit, 12 MHz
CMD17 单块读 / CMD24 单块写, CPU 搬 FIFO, sd_lock 互斥
```

与 xv6 的 bio.c 相比：

方面	xv6 bio.c	miniOS bio.c
块大小	1024 B（BSIZE）	512 B，正好一个 SD 扇区
缓冲数	NBUF = MAXOPBLOCKS × 3 = 30	24
每块锁	每个 buf 一把 sleeplock，bread 返回时已上锁	没有每块锁；FAT32 层的 fs.lock 一把互斥锁串行所有操作
写	bwrite 由日志层决定何时写，log_write 钉住缓冲	直写，bwrite 立刻写卡；失败时把缓冲标成无效，下次重读
大块读	总是经过缓存	fat32_read 对齐的整扇区直接 t113_sd_read 进调用者缓冲，绕过缓存
设备	virtio，中断完成，进程 sleep	SMHC0 轮询，忙等

绕过缓存是安全的，因为缓存是直写：卡上的内容永远不比缓存旧。

3.7.6 调用链：打开并读一个文件

XR829 驱动装固件时的路径（fwio.c 的 need_image()）：

```
need_image("/boot/fw_xr829.bin")
+-- t113_vfs_open(path, VFS_O_RDONLY)                    vfs.c
|   +-- mutex_lock(&vfs_lock)
|   +-- find_mount(path)            "/" 和 "/boot" 都匹配，取更长的 "/boot"
|   +-- fat32_lookup(path, &node)                        fat32.c
|   |   +-- mutex_lock(&fs.lock)
|   |   +-- resolve(path, &d, parent = 0)
|   |   |   +-- mount_relative()    "/boot/fw_xr829.bin" → "fw_xr829.bin"
|   |   |   +-- d = 根目录（first = root_cluster, lba = 0）
|   |   |   +-- dir_find(root_cluster, "fw_xr829.bin")
|   |   |       +-- while dir_next() > 0: name_equal()     不区分大小写
|   |   |           +-- bread → 命中或 t113_sd_read(lba, 1)
|   |   +-- iget(dir = 0, first, size, lba, off)          按目录项位置去重
|   |   +-- fill_node()             node.private_data = fat_inode
|   +-- 取一个空闲 open_files[], offset = 0
+-- t113_vfs_file_size(file)        174624
+-- t113_alloc_pages(43)
+-- t113_vfs_read(file, data, 174624)
|   +-- fat32_read(&node, 0, data, 174624)
|       +-- while done < count
|           +-- c = ibmap(ip, pos / 簇字节, 0)    游标让顺序读每簇只走一步
|           +-- 扇区对齐、剩余 ⩾ 512、dst 4 字节对齐
|           |   +-- t113_sd_read(lba, n, dst + done)   一次读到本簇末尾
```

```
|          +-- 否则 bread(lba) → copy → brelse        首尾不对齐的零头
+-- t113_vfs_close(file)
    +-- fat32_close()              未修改，只做 ref--
```

3.7.7 调用链：创建并写一个文件

tftp 把下载的文件写进 /boot 时（src/net/tftp/tftp_udp.c 的 write_file()）：

```
t113_vfs_open("/boot/minios-kernel.bin", RDWR | CREATE | TRUNC)
+-- find_mount → fat32_ops
+-- fat32_lookup() 失败（文件还不存在）
+-- fat32_create(path, &node)
|   +-- resolve(path, &d, parent = 1, &name, &len)    停在父目录
|   +-- dir_find() 再确认不存在
|   +-- create_entry()                                LFN + 短项，first = 0
|   +-- iget(0, 0, 0, lba, off)
+-- fat32_truncate(&node)          新文件也走一遍
    +-- first = 0, size = 0, 清游标
    +-- iupdate(ip)                先改目录项...
    +-- fat_free_chain(旧 first)   ...再释放旧簇链

t113_vfs_write(f, data, 4096) × N
+-- fat32_write(&node, offset, src, count)
    +-- offset > size → -1          不支持稀疏文件
    +-- while done < count
        +-- c = ibmap(ip, pos / 簇字节, alloc = 1)
        |   +-- 走到链尾 → fat_alloc(c)
        |       +-- fat_set(new, EOC)   FAT1、FAT2 各 bread + bwrite
        |       +-- fat_set(c, new)
        +-- 整扇区覆盖 → bget_blank(lba)   不必先读
        |   部分扇区   → bread(lba)
        +-- copy → bwrite → brelse       数据扇区立刻落盘
        +-- size 变大 → dirty = 1

t113_vfs_close(f)
+-- fat32_close()
    +-- dirty → iupdate()           目录项写回 first 和 size
    +-- ref--
```

3.7.8 没有日志时的写入顺序

xv6 的每个文件系统调用包在 begin_op()/end_op() 里。日志提交前，所有修改只存在于缓存和日志区，崩溃后要么全部重放，要么全部丢弃。miniOS 的 FAT32 没有日志，靠写入顺序把崩溃的后果限制在“丢空间，不丢一致性”：

操作	顺序	中途断电的后果
追加写	先 fat_alloc 写 FAT，再写数据扇区，close 时最后写目录项	目录项还是旧的 first/size，新簇成为没人引用的“丢失簇”；fsck 能回收
截断	先 iupdate 把目录项清零，再 fat_free_chain	旧链没释放完，同样只是丢失簇
新建	先写 LFN 项，再写短项	只有 LFN 没有短项时，校验和对不上，读目录会忽略这几项
FSInfo	第一次分配时把空闲计数写成 0xffffffff	其他系统会重新数空闲簇，不会相信过期的计数

FAT 的两份副本是先后写的，中途断电时两份可能不一致。Windows 和 `fsck.vfat` 默认以第一份为准。

3.7.9 T113 实现上的区别

- 块设备是 SD 卡本身：`bio.c` 的块号就是整卡 LBA，和 xv6 的“文件系统内块号”不同。FAT32 挂载时从 MBR 找第一个类型为 0x0b/0x0c/0x1b/0x1c 的分区，所有地址都加上 `part_lba`。
- `sunxi-sd.c` 和 SPL 共用：SMHC0 走 PF0–PF5，1-bit 总线，12 MHz，每次一块（CMD17/CMD24），CPU 轮询 FIFO。内核里加了 `sd_lock` 互斥锁，根文件系统（`rootdev.c`）和 `/boot` 共享这一个控制器。
- 一把锁：`fs.lock` 串行 FAT32 的所有操作，外面还有 VFS 的 `vfs_lock`。没有 xv6 那种每个 inode、每个缓冲一把 sleeplock 的细粒度并发。
- 不支持：mkdir、unlink、rename、时间戳（固定写 2026-01-01）、稀疏写。目录只能读和在已有目录里新建文件。
- 失败可回退：`t113_fat32_mount_vfs()` 失败时，`kernel_boot()` 改挂 `bootfs`，用 SPL 预加载到 DDR 的固件镜像（见第 2 章）。

3.8 一致性与限制

- **没有日志**。写直达保证每次 `write` 返回时数据已经在卡上，但目录项里的大小和首簇只在最后一次 `close` 时写回。如果写到一半掉电，新分配的簇在 FAT 里已经占用，目录项却还是旧的大小，表现为文件内容没变、分区里多出一些丢失的簇，fsck（`fsck.fat` 或 macOS 磁盘工具）能回收。
- **锁**：整个 fat32 一把互斥锁（类似早期 xv6 的大锁），每个 VFS 回调从头锁到尾。互斥锁不可重入，fat32 内部函数之间不会重复加锁。
- **inode 表 16 项**：同时打开的不同 FAT32 文件最多 16 个（VFS 的 `open_files` 也只有 32 个槽）。
- **SD 访问**：单块 CMD17/CMD24 轮询，没有 DMA、没有多块传输，大文件写入速度受此限制。

3.9 改动前后对照

	改动前（bootfs）	改动后（fat32）
谁解析 FAT32	SPL	内核（SPL 也保留自己的只读解析器）
数据来源	启动时 DDR 快照	SD 卡，运行期实时读写
查找	三个硬编码路径	任意路径、长文件名、子目录
`ls /boot`	空	列出分区内容
写入 / 创建 / 截断	×	✓
缓存	无（数据本来就在内存）	bio 24 块 LRU，写直达
失败时	PANIC	退回 bootfs，再失败才 PANIC
与 Linux 对照	firmware loader 读 initramfs 里的文件	vfat 驱动挂载 `/boot`

3.10 测试

主机测试（`gcc -fsanitize=address,undefined`，链接内核原样的 `vfs.c`、`bio.c`、`fat32.c`，SD 读写换成对镜像文件的读写）：

- `mkfs.fat -F 32` 生成分区（每簇扇区数作为参数，测过不同簇大小）；
- 读 mtools 写入的长文件名文件和子目录里的文件，逐字节比对；
- `O_CREAT|O_TRUNC` 新建文件，用 512 字节和 300 字节交替写入（覆盖对齐和不对齐、跨扇区、跨簇），读回比对；
- 截断已有的 `fw_xr829.bin` 并重写；
- 在子目录里连续创建 40 个长文件名文件，触发目录追加新簇；
- 用 `fsck.fat -n` 检查分区无错误，再用 `mcopy` 读回内核写的文件比对。

上板预期串口输出：

```
SPL: FAT32 kernel kernel_minios.img bytes=0x... crc32=0x...   # FAT32 上有内核时
SPL: skip bootpack kernel (FAT32 kernel used)
...
fat32: /boot mounted rw, partition lba=0x00002000 clusters=0x... cluster-bytes=0x...
mmc1:0001:1: loaded /boot/boot_xr829.bin bytes 2184
...
$ ls /boot
$ tftp 192.168.0.100 test.bin
tftp: received ... bytes in ... ms, crc32 0x...
tftp: saved /boot/test.bin
```

如果看到 `fat32: mount failed, using SPL handoff images`，说明 MBR 里没有 FAT32 分区或 BPB 校验失败，此时 `/boot` 退回老的只读视图。

第四章 UART3 控制台：TX 轮询、RX 中断与 /dev/console

本章对应当前 miniOS（100ASK T113-S3）的实现，说明 UART3 从初始化到收发数据的完整工作流程：发送侧为什么用轮询、接收侧如何用中断 + 256 字节环形缓冲把按键交给 shell、`/dev/console` 这个设备文件是怎么注册和关联到 fd 0/1/2 的。涉及的代码集中在 `src/boards/100ask_t113/uart.c`、`timer.c`（IRQ 分派）、`src/kernel/fs/devfs.c`、`src/kernel/syscall.c`。

当前已打通的链路：

- `t113_uart_init()` 是 `kernel_main()` 做的第一件事，串口立即可用，内核所有 `t113_uart_puts` 调试用打印都走它（early console）；
- TX 纯轮询：等 `USR.TFNF` 有空槽就写 THR，带 10 ms 停转看门狗；
- RX 双模式：中断使能前用轮询，使能后走 IRQ 37 →环形缓冲→等待队列唤醒阻塞读；
- 驱动模型 probe 把同一对读写函数注册成 devfs 的 `console`(1,0) 和 `ttyS0`(3,0)，`open("/dev/console")` 可用；
- 用户 stdio 不走 VFS：`sys_read` 对 fd 0、`sys_write` 对 fd 1/2 直接特判到 UART。

尚未实现：TX 软件环形缓冲 + 发送中断（现在写是忙等）、RX 流控（无 RTS/CTS，环形缓冲满只能丢字节）、行规约（echo/退格/canonical 模式在 sh 的 readline 里自己做）、串口参数 ioctl（波特率固定 115200）、多个串口。

4.1 硬件与初始化

UART3 是 DesignWare APB UART（16550 兼容 + 扩展寄存器）：

引脚/时钟	配置
PB6 = TX，PB7 = RX	GPIO PB CFG0 复用 7（`gpio_set_uart3`，uart.c:77）
时钟	CCU `UART_BGR` bit3 = UART3 gate，bit19 = 复位释放（uart.c:87-92，先关 gate 再开，中间 100 次空转等待）
波特率	24 MHz / (16 × 13) = 115200（`LCR.DLAB=1` 写 `DLL=13`，uart.c:97-98）
帧格式	8N1（`LCR=0x03`，uart.c:100）
FIFO	`FCR=0x07`：使能 + 收发 FIFO 复位，触发级默认（uart.c:101）

比普通 16550 多出来的两个状态位贯穿全文：

- `USR`（UART+0x7C）：bit0 BUSY、bit1 TFNF（TX FIFO 未满）、bit3 RFNE（RX FIFO 非空）——8250 驱动里通常轮 `LSR.THRE`，DW 变体要看 `USR.TFNF`；
- `IIR` 的 0x07 = Busy detect：在 `USR.BUSY` 期间写 LCR 会挂起这个中断，**只有读一次 USR 才清**，不清的话电平中断会一直挂着（uart.c:242-243 专门处理）。

4.2 启动时序：early console 与驱动 probe 的分工

```
kernel_main (platform_kernel.c:32)
 ├─── t113_uart_init()          ← 第一件事：裸寄存器初始化，串口立刻能打
 ├─── t113_mmu_init()
 └─── kernel_boot (src/kernel/init/main.c:27)
     ├─── 挂载 /boot / /dev
     ├─── t113_driver_core_init()   → t113_uart_probe (uart.c:349)
```

```
    |                                  只校验 mem/IRQ 与硬编码值一致，
    |                                  注册 devfs "console"/"ttyS0"
    ├─── t113_gic_init()
    ├─── t113_sched_init()
    ├─── kernel_platform_console_irq_init()  → t113_uart_irq_init (uart.c:194)
    ├─── start_init_process()      ← 加载 /init
    └─── timer0_init + cpsie i     ← 从此 IRQ 37 (UART RX) 开始到达
```

注意 t113_uart_probe 里那句注释：硬件在 kernel_main() 开头就已经初始化好了，probe **不做任何硬件操作**，只把 “这块板子的 UART3 资源确实对应 early console” 这件事校验一遍，然后向 devfs 登记节点（uart.c:343-364）。这是把 “启动早期就要用的裸驱动” 嫁接到驱动模型上的典型做法：硬件归 boot 代码，注册归 probe。

4.3 TX 路径：轮询 + 停转看门狗

```
write(1, buf, n)  或内核 t113_uart_puts
  → sys_write (syscall.c:405)  fd==1||fd==2 特判
    → t113_uart_putc (uart.c:141)  逐字节:
        '\n' → 先递归发 '\r'                (终端要 CRLF)
        uart_tx_wait_slot()                 等 USR.TFNF
        uart3->rbr_thr_dll = c              写 THR
        uart_tx_report_stall()              有停转则补打诊断
```

为什么 TX 用轮询不用中断 + 缓冲：这个串口的第一职责是内核调试打印（panic、exception dump、驱动 probe 信息），语义上要求 “打完这句才继续”，而且要能在 IRQ 关掉的上下文里用。环形缓冲 + 发送中断适合吞吐量场景，对 printk 反而是负担——xv6 的 uartputc 同样是关中断轮询 LSR.THRE，Linux 则只在 console_unlock 的 atomic copy 阶段用轮询。miniOS 现在连 TX 软件缓冲都没有，console_write（uart.c:307）就是逐字节 putc 循环。

轮询的代价由 uart_tx_wait_slot（uart.c:108）兜底：

- 正常路径：USR.TFNF 为 1 就返回，一个字节约 87 µs（115200 8N1）；
- 停转路径：连续 10 ms 没有空槽（UART_TX_STALL_US），认为发送器卡死——快照 USR/LSR、计数、**只复位 TX FIFO**（FCR=0x05，保住 RX FIFO 里没读走的输入），把诊断推迟到 putc 末尾再打（uart_tx_report_stall，避免在持锁/IRQ 上下文里递归打印）。

4.4 RX 路径：中断如何拿到数据

4.4.1 使能

t113_uart_irq_init()（uart.c:194）在调度器就绪后、第一个用户进程创建前调用（main.c:45）：

1. rx_irq_enabled = 1（持锁写，读侧据此在 “轮询直读” 和 “睡等待队列” 之间二选一）；
2. IER = ERBFI：只开 “接收数据可用” 中断；
3. dsb sy + t113_gic_enable_irq(T113_IRQ_UART3)——IRQ 37 = GIC SPI 5（platform.h:21）。

4.4.2 中断分派链

```
硬件 IRQ 37 电平触发
  → vector → kernel_irq_entry (kernel_entry.S:122)
      srsdb sp!, #0x13            IRQ→SVC, 保存 LR+SPSR
      cps #0x13                   切 SVC 栈
      stmdb sp!, {r0-r12, lr}
```

```
bl t113_irq_handler          (timer.c:96)
  ├─── GIC IAR 取号 + EOI
  ├─── irq == T113_IRQ_UART3 → t113_uart_irq_handler() (uart.c:235)
  └─── UART3: t113_sched_yield()   ← 让被唤醒的读任务立即跑
ldmia / rfeia                恢复原上下文(可能已是新任务)
```

4.4.3 中断里做什么

`t113_uart_irq_handler`（uart.c:235）很短，三个动作：

1. **清 Busy detect**：`IIR == 0x07` 时读一次 `USR`（唯一清除方式，不清则电平中断永久挂起）；
2. **drain FIFO** → `uart_rx_drain`（uart.c:204）：
   - 不信任 IIR 的瞬态 `NO_INT`——DW 版在第一次读 RBR 后可能立即报 “无中断”，但 `LSR.DR` 还挂着同一串按键的后续字节。所以按 **FIFO 状态**（`LSR.DR`）循环抽干，而不是按中断原因数字节；
   - 每字节：持 `rx_wait.lock` →写 `rx_buffer[rx_head]` → `rx_head = (head+1) & 255`，`next == rx_tail` 说明 256 字节环形缓冲**满了，丢弃**（无流控，丢字节好过覆盖）；
   - 收尾重写 `IER = ERBFI`（timeout/line-status 中断服务后防丢使能的保险）；
3. **唤醒读方**：收到字节就 `wake_up_all(&rx_wait)`，把睡在队列上的任务置为 RUNNING；返回 `t113_irq_handler` 后立刻 `t113_sched_yield()`，不等下一秒 Timer0——这是交互延迟的关键：shell 按一键，UART IRQ 在同一条中断路径里完成 “存字符→唤醒→调度走”，read 系统调用毫秒级返回。

4.4.4 阻塞读：锁内入队防丢唤醒

```
read(0, buf, n) → sys_read (syscall.c:63)  fd==0 特判
  → t113_uart_read (uart.c:161):
      持 rx_wait.lock 查环形缓冲:
        有字符 → 拷贝并立刻返回(终端语义: 有 1 字节就返回)
        空 + IRQ 模式 → wait_queue_sleep_locked(&rx_wait, flags)
                        持锁状态下把自己入队再睡 —— 与 IRQ 生产者
                        共用一把锁, "查空→入队" 与 "存字节→唤醒"
                        不可能交错丢唤醒
        空 + 轮询模式 → 解锁, 直接读硬件 LSR.DR, 否则 yield 自旋
```

`wait_queue_sleep_locked`（`src/kernel/sync.c:43`）= 持锁入队→解锁并恢复 IRQ → `t113_sched_yield()`。拿锁用 `spin_lock_irqsave`，因为生产者那半在 IRQ 上下文里跑。

4.4.5 RX 看门狗：漏中断恢复

UART 中断毕竟刚调通，留了一道保险：Timer0 每秒 tick 里调用 `t113_uart_rx_watchdog`（uart.c:247，挂在 timer.c:104）——若 `LSR.DR` 显示有数据而 IRQ 没把它们搬走（漏中断/使能丢失），就主动 `uart_rx_drain()` 一次，并打印一次 “recovered missed RX interrupt”。能自愈，又能暴露问题。

4.5 /dev/console 如何关联

4.5.1 注册：probe → devfs 节点

uart probe（uart.c:359-360）：

```
t113_devfs_register_chrdev("console", 1u, 0u, &console_fops);
t113_devfs_register_chrdev("ttyS0",   3u, 0u, &console_fops);
```

console_fops 只有两个成员 .read 与 .write，都是 t113_uart_read / 逐字节 t113_uart_putc 的薄封装（uart.c:302-318）——devfs 的 read/write 本就不带 offset（字符设备无位置概念），正好匹配串口语义。

devfs 侧（devfs.c）：节点表 16 项，名字 ≤30 字符；devfs_basename 剥掉 /dev 前缀后线性查名；lookup 命中后把 id = major<<16 | minor、private_data = &nodes[i] 填进 VFS 节点；devfs_read/write 转发到 node->ops->read/write。VFS 层负责“哪个挂载点接这个路径”（最长前缀匹配，/dev → devfs），这部分详见第 2 章。

4.5.2 使用：fd 0/1/2 特判，不走 VFS

这是与 xv6/ Linux 差异最大的一处。当前 sys_read/sys_write 对 stdio 是硬编码特判（syscall.c:65、syscall.c:411）：

fd	系统调用	去向
0 (stdin)	sys_read	直接 t113_uart_read
1/2 (stdout/stderr)	sys_write	直接逐字节 t113_uart_putc
≥3	open/read/write/close	经 VFS → devfs → console_fops（open("/dev/console", O_RDONLY) 可读串口）

用户进程创建时不打开任何文件，files[] 全空；sh 的 read(0,...) / write(1,...) 全靠 fd 特判落到 UART。/dev/console 节点现在主要给将来的用户程序用（以及证明 devfs 通路完整），init 并未像 xv6 那样 open("/dev/console") 后 dup 成 0/1/2。

4.5.3 全景数据流

```
键盘
 | USB 转串口
 ▼
UART3 RX FIFO（硬件，16 深）
 | IRQ 37 电平触发，IER.ERBFI
 ▼
t113_uart_irq_handler → uart_rx_drain
 | 持锁写 256B 环形缓冲 rx_buffer[head]   （满则丢）
 | wake_up_all(&rx_wait)  → sched_yield
 ▼
sh: read(0, &c, 1)
 | svc #0, r7=9 (T113_SYS_READ)
 ▼
sys_read (fd==0 特判) → t113_uart_read
 | 环形缓冲空 → wait_queue_sleep_locked 阻塞
 | 有字节 → 拷给用户 buf，返回 1
 ▼
sh readline 回显: putchar(c)
 | write(1, &c, 1) → svc #0, r7=1 (T113_SYS_WRITE)
 ▼
sys_write (fd==1 特判) → t113_uart_putc
 | '\n'→'\r\n'; 等 USR.TFNF; 写 THR
 ▼
UART3 TX FIFO → PB6 → USB 转串口 → 终端
```

内核打印（t113_uart_puts）与用户 write(1,...) 共用同一个 putc，**没有互斥**：多任务并发打印会逐字节交错，这是有意保留的调试行为（调度器 trace 就是靠它看交错的）。xv6 用 Console.lock 保证整行不穿插；Linux 靠 console_sem + per-console lock。

4.6 双模式读的设计意图

rx_irq_enabled 把读路径切成两副面孔，是为了 bring-up 顺序：

- 阶段一（t113_uart_irq_init 之前，含 SPL 交接后的早期内核）：GIC 未初始化，IRQ 到不了，t113_uart_read/t113_uart_getc 退化为轮询 + yield 自旋。kernel_shell（kernel_shell.c:63，当前未接入启动流程，留作调试）用的就是这套；
- 阶段二（调度器 + GIC 就绪后）：开 IER、开 GIC，读路径切到 “环形缓冲 + 睡眠唤醒”，CPU 空转代价归零。

getc_nonblock（uart.c:150）也保留双模式：IRQ 模式下只查环形缓冲（硬件 FIFO 由中断独占搬运，避免与 IRQ 竞争读 RBR），轮询模式下才直读硬件。

4.7 调试设施

调通这条链路留下的开关，全部默认 0（除 Kconfig 项）：

开关	位置	作用
CONFIG_DEBUG_TIMER_TICK	menuconfig / autoconf.h	每秒打印 tick 计数
SCHED_STATUS_TRACE	timer.c:20	每秒一行：任务状态 + IER/LSR/USR + GIC 中 IRQ37 状态 + 等待队列
HEARTBEAT_TRACE	timer.c:28	每 5 秒 ‘.’, 有新 UART IRQ 时打 <qN>
STDIN_TRACE	syscall.c:23	首次 stdin 等待/返回时打 LSR/USR/收到数
EXEC_TRACE	syscall.c:16	exec 返回用户前经真实 MMU 校验 entry/sp/argc
运行时计数	uart.c:282	t113_uart_rx_irq_count()；t113_uart_debug_print() 打 q<irq 数 > r<head>/<tail> w< 等待者 pid>
TX 停转	uart.c:42	tx_stall_count/usr/lsr，发生时打印 uart3: TX stall #N USR=.. LSR=..
RX 看门狗	uart.c:247	漏中断自愈 + 一次性提示

排障口诀：st 行里的 g（GIC 视角 IRQ37 状态）为 1 而 r<head/tail> 不动→中断到了但 drain 没跑；q 计数不动而 LSR 有 DR → IRQ 没进来，查 IER/GIC enable；q 涨、r 动、shell 仍卡→问题在唤醒/调度，不在串口。

4.8 与 xv6 / Linux 对照

话题	miniOS	xv6	Linux
TX	轮询 `USR.TFNF`，无缓冲	`uartputc` 关中断轮 `LSR.THRE`，无缓冲	serial core：环形 xmit_buf + 发送中断/PIO 兜底
RX	IRQ → 256B 环形缓冲→等待队列	IRQ → `uartrecv` 直存 console 输入队列（同样睡眠唤醒）	IRQ → `tty_port` flip buffer → line discipline → read
阻塞读	`wait_queue_sleep_locked` 持锁入队	`sleep(&cons.r, ...)` 同样要求持锁	wait_event + tty 互斥
行规约	无，sh 的 readline 自己 echo/退格	无（echo 在 console 输入回显里做了一半）	tty ldisc：echo、canonical、信号
/dev/console	devfs 节点已注册，但 stdio 走 fd 特判	init 显式 open + dup 到 0/1/2	内核 console（printk）与用户 tty 分离又共存
丢数据	RX 满丢字节，无流控	输入队列满丢字节	有 RTS/CTS 流控、可设 termios

演进方向（如果要做）：照 xv6 的样子把 init 改成 `open("/dev/console")` + dup 到 0/1/2，撤掉 syscall 的 fd 特判；TX 加软件环形缓冲 + `IER.ETBEI` 发送中断，把长打印从忙等里解放出来；这两点做完，UART 子系统就和 xv6 的 console/ 基本同构了。

第五章 WiFi 驱动：XR829 / SDIO 枚举与固件下载

本章对应当前 miniOS 的实现，说明 100ASK T113-Pro 板上 XR829 WiFi/BT 模组驱动的分层结构、SDIO 枚举与 CMD53 数据通路、SMHC 主机控制器、HIF 寄存器/间接寻址、固件下载全流程，以及固件镜像的打包与加载。

当前已打通的链路：

- SDIO 枚举（CMD0/5/3/7 + CMD52 读 CCCR/FBR/CIS）；
- CMD53 双向数据通路（PIO，非 DMA）与 HIF 寄存器自检；
- 总线提速（400 kHz → 12 MHz 默认速度、1-bit）并在高速下复测；
- HIF 间接访问（SRAM/AHB 端口）与 DPLL/唤醒序列；
- boot_xr829 载入 + fw_xr829 通过 32 KiB 下载 FIFO 传输；
- 切到 queue（消息）模式，解析 WSM startup indication；
- WSM 命令/confirm 串行化、RX indication 分派、TX confirm、MIB 读写；
- WSM CONFIGURATION、2.4 GHz 定向主动扫描，以及 beacon/probe response 解析；
- WPA2-PSK 所需 PBKDF2/PTK/MIC/AES unwrap 通用密码学模块。

主机侧没有接 SDIO 中断（IRQ 73 已声明但全部轮询），整条 WiFi 链路是轮询驱动的。

更新：STA JOIN、WPA2 四次握手（EAPOL）和 IPv4/UDP/DHCP/tftp 都已实现，网络栈和驱动已按 Linux 分层重新组织（cfg80211 / mac80211 / supplicant / xradio 驱动拆分），网络栈细节见第 6 章；`/boot` 固件加载已改为内核直接读写 FAT32，见第 3 章。本章硬件、SDIO 和固件下载部分的说明不变，“文件组成”“分层结构” 两节已按新文件名更新。

5.1 硬件连接

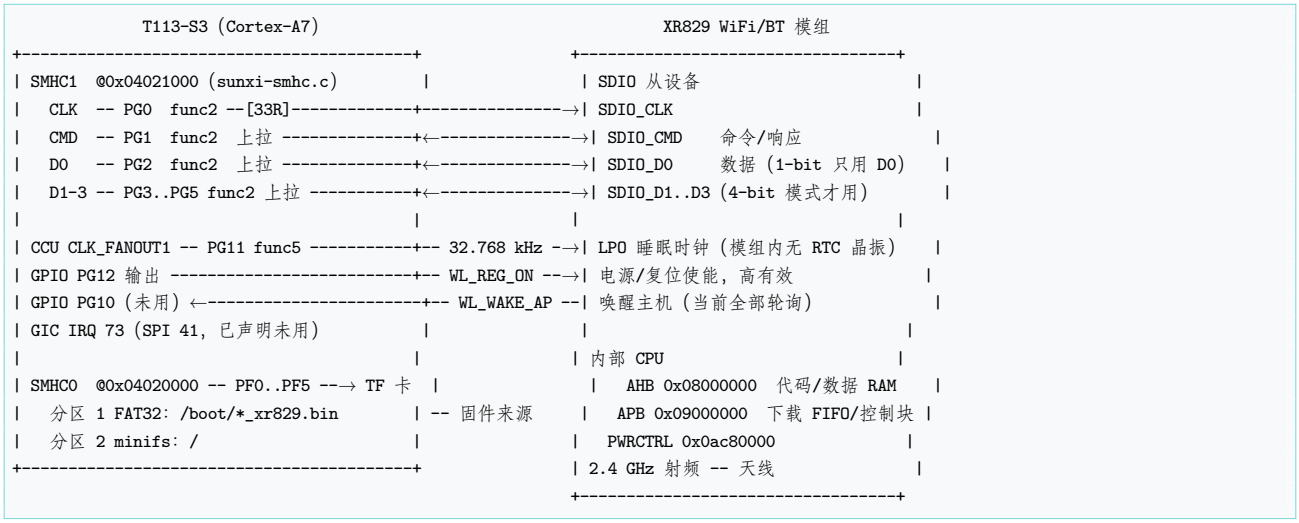
XR829 WiFi/BT 合封模组挂在 SMHC1 上，原理图参考 `100ASK_T113-Pro_Base-SCH_V1.2` p4/p9：

信号	引脚	功能号	说明
SDC1-CLK	PG0	2	串 33R 电阻
SDC1-CMD	PG1	2	上拉
SDC1-D0..D3	PG2..PG5	2	上拉
CLK-FANOUT1	PG11	5	给模组喂 32.768 kHz 时钟
WL_WAKE_AP	PG10	-	未使用（当前轮询，无中断）
WL_REG_ON	PG12	GPIO 输出	模组使能，高有效

- SMHC1 寄存器基址 `0x04021000`（`T113_SMHC_BASE(1)`），长度 4 KiB；
- IRQ 73（GIC SPI 41，`T113_IRQ_SMHC1`）在平台资源里声明了，但驱动纯轮询，目前没有使用；
- 32.768 kHz 由 CCU 的 CLK_FANOUT1 输出：XR829 没有内部 RTC 时钟，睡眠时钟必须由 SoC 提供，否则模组上电后不响应 CMD5。

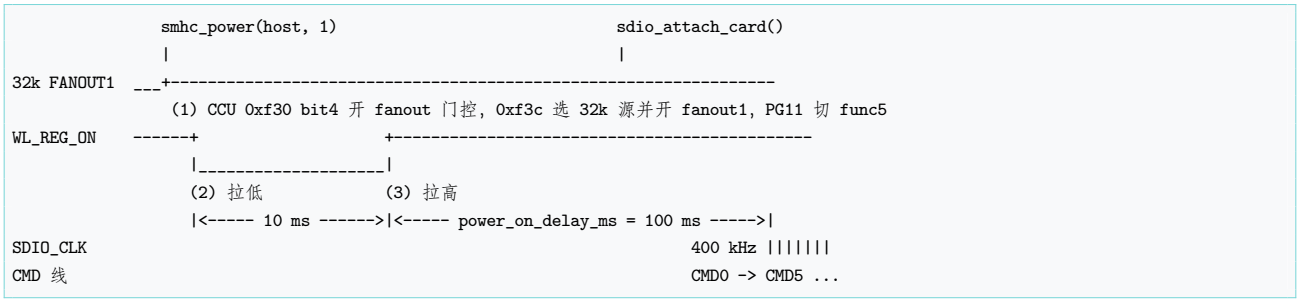
板级引脚和电源时序描述在 `src/ include/ t113/ wifi.h` 的 `t113_mmc_platform_data`，具体数值在 `board.c` 的 `sdc1_pdata`。

5.2 图解：硬件连接与上电时序



两个 SD 控制器分工明确：SMHC0 接 TF 卡，负责根文件系统和 `/boot`；SMHC1 接 XR829。WiFi 驱动启动时，要先经 SMHC0 从 FAT32 读出三个固件文件，再经 SMHC1 灌进模组。

上电时序在 `smhc_power()`（`sunxi-smhc.c`）里，由 `sdio_attach_card()` 在发 CMD0 之前调用：



32k 时钟必须先于 WL_REG_ON 拉高：XR829 的内部电源管理靠它计时，没有它模组不会响应 CMD5。`PLL_PERIPH0` 没开时 fanout 也出不来，`smhc_power()` 会打印提示。

5.3 文件组成

文件	角色
src/drivers/mmc/host/sunxi-smhc.c	SMHC（sunxi-mmc）主机控制器驱动：命令 + PIO 数据阶段，platform 总线驱动，轮询
src/drivers/mmc/core.c	轻量 MMC/SDIO core：CMD52/CMD53、CCCR/CIS 解析、sdio 总线、卡枚举、mmc_set_clock()
src/ drivers/ net/ wireless/ xradio/ sdio.c	SDIO function 驱动 xradio_sdio_driver：probe/ remove、CMD53 自检、提速
.../xradio/hwio.c	HIF 寄存器、APB/AHB 间接访问
.../xradio/fwio.c	DPLL、bootloader 与固件下载、SDD
.../xradio/bh.c	底半部：HIF 队列读写、序号、固件输入缓冲计数
.../xradio/wsm.c	WSM 命令编码、confirm、indication 分派、MIB、TX
.../xradio/main.c	核心 probe：固件 → WSM 启动 → CONFIGURATION → 注册到 mac80211
.../xradio/sta.c	ieee80211_ops：start、JOIN/关联参数、set_key、poll
.../xradio/scan.c、txrx.c	硬件扫描；802.11 帧 ↔ WSM TX/RX
.../xradio/{xradio,wsm,hwio}.h	驱动私有头文件（WSM wire-format、HIF 定义）
src/ net/（core、ethernet、ipv4、wireless、mac80211、wpa、tftp）	网络栈，见第 6 章
src/ lib/ crypto/ wpa_crypto.c、src/ include/crypto/wpa_crypto.h	通用 WPA2-PSK 密码学
src/boards/100ask_t113/bootpack.c	内核侧 bootpack 查询（SPL 把 header 拷到 DDR 固定地址）
src/include/t113/bootpack.h	bootpack 容器格式，与 miniBoot、mkbootpack 共用
src/include/t113/mmc.h	mmc_host/sdio_card/sdio_func/sdio_driver、CMD53 API、CCCR/FBR 定义
src/include/t113/wifi.h	t113_mmc_platform_data：sdcN 平台数据（总线引脚 + 模组电源时序）
src/boards/100ask_t113/board.c	sdc1_device 平台设备定义、驱动注册顺序
src/boards/100ask_t113/driver_model.c	初始化顺序：设备 → 驱动 → mmc_rescan_all()
miniBoot/tools/mkbootpack.c	主机侧打包工具：kernel + 固件镜像 → bootpack.bin
rpi_bm/Makefile	检测 firmware/ xr829/ *.bin 并自动填入 BOOTPACK_EXTRA

固件镜像本身（boot/fw/sdd）是 Tina SDK 的有版权文件，不进入 git（rpi_bm/.gitignore 忽略 /firmware/），构建时从本地目录打入 bootpack。

5.4 分层结构

整体照 Linux 的形状搭（名字和回调都尽量对齐），但实现极小：

```
platform 总线                    sdio 总线
---------------------            ---------------------
```

```
sdc1_device  --match-->  t113_smhc_driver
    |                        | probe: 开 CCU 时钟/复位, pinmux,
    |                        |        复位控制器, 注册 mmc_host
    v                        v
  struct device           struct mmc_host (mmc1)
                                |  mmc_rescan_all() 枚举
                                v
                           struct sdio_card
                                |  sdio_add_funcs()
                                v
                           struct sdio_func (func 1)
                                |  device_register() 后作为
                                |  /bus/sdio/devices/mmc1:0001:1
                                v
   mmc1:0001:1 --match-->  xradio_sdio_driver ("xradio_wlan")
                                | probe (sdio.c):
                                |   使能 function, 设块大小 512
                                |   CMD53 自检 -> 提速到 12 MHz
                                | xradio_core_probe (main.c):
                                |   fwio.c 下载固件 -> WSM startup
                                |   CONFIGURATION -> ieee80211_register_hw
                                v
                      struct xradio_common -> mac80211 -> wlan0
```

代码目录按 Linux 的 bus/core/protocol/MLME 边界组织：

```
src/drivers/mmc/
+-- host/sunxi-smhc.c      SoC host controller (硬件层)
+-- core.c                 MMC/SDIO core 与 sdio_bus (总线层)

src/drivers/net/wireless/xradio/
+-- sdio.c                 sdio_driver
+-- hwio.c / fwio.c        HIF 访问 / 固件下载
+-- bh.c / wsm.c           消息传输 / WSM 协议
+-- main.c                 核心 probe, 注册到 mac80211
+-- sta.c scan.c txrx.c    ieee80211_ops 实现

src/net/mac80211/ src/net/wireless/ src/net/wpa/   802.11 状态机、cfg80211、supplicant
src/lib/crypto/wpa_crypto.c    与具体网卡无关的 WPA2 primitive
```

依赖方向保持单向：板级 SMHC → SDIO core → xradio 驱动 → mac80211 → cfg80211 → supplicant；wpa_crypto 只提供算法，不反向依赖驱动。

数据面走的是 SDIO function 1 的 CMD53：

```
hwio.c
+-- xradio_reg_read_32/write_32()   CMD53 4 字节, 访问 HIF 寄存器窗口
+-- xradio_apb_write/apb_read_32()  SRAM_BASE_ADDR + SRAM_DPORT 间接访问 APB
+-- xradio_ahb_write_32/read_32()   SRAM_BASE_ADDR + AHB_DPORT 间接访问 AHB
+-- sdio_memcpy_fromio/toio()       CMD53 块/字节传输
        +-- sdio_io_rw_extended()
                +-- mmc_command + mmc_data
                        +-- smhc_request() -> smhc_pio() FIFO 搬运
```

对应 Linux 里的映射关系：

miniOS	Linux / xradio
`mmc_host` + `mmc_host_ops`	`struct mmc_host` + `mmc_host_ops`
`sdio_card` / `sdio_func`	`mmc_card` / `sdio_func`
`sdio_bus_type` + `sdio_driver`	`sdio_bus_type` + `sdio_driver`
`sdio_driver_register()`	`sdio_register_driver()`
`sdio_io_rw_extended()`	`mmc_io_rw_extended()`
`mmc_rescan_all()`	`mmc_rescan()`（Linux 里在 workqueue 中跑）
`xradio_reg_*` / HIF id	xradio `hwio.c` 的 `xradio_reg_read_32()` 等
固件下载流程	xradio `fwio.c` 的 `xradio_load_firmware()`

5.5 总调用链

从驱动核心初始化到 `wlan0` 注册：

```
kernel_boot()
+-- t113_driver_core_init()                          boards/100ask_t113/driver_model.c
    +-- platform_bus_init() / i2c_bus_init() / sdio_bus_init()
    +-- t113_board_register_devices()
    |   +-- platform_device_register(&sdc1_device)   只登记，不碰硬件
    +-- t113_board_register_drivers()
    |   +-- platform_driver_register(&t113_smhc_driver)
    |   |   +-- t113_smhc_probe(sdc1)                drivers/mmc/host/sunxi-smhc.c
    |   |       +-- platform_get_resource()          SMHC1 基址 0x04021000
    |   |       +-- CCU 0x84c：复位 → 开门控；0x834：mod clock = HOSC (/2 后 12 MHz)
    |   |       +-- t113_pinmux_apply(sdc1_pins)     PG0..PG5 func 2
    |   |       +-- smhc_reset(); HWRST 拉低 10 us 再拉高
    |   |       +-- NTSR |= NEW_MODE; TMOUT; IMASK = 0 (不用中断)
    |   |       +-- smhc_set_ios(400 kHz, 1-bit)
    |   |       +-- mmc_add_host()                   卡扫描推迟
    |   +-- sdio_driver_register(&xradio_sdio_driver)    总线上还没有设备
    +-- mmc_rescan_all()                             不在任何 probe 里，避免持锁注册设备
        +-- sdio_attach_card(host) → sdio_add_funcs(host)    (上一节的状态机)
            +-- device_register(func) → xradio_sdio_probe(func)      xradio/sdio.c
                +-- sdio_enable_func()、sdio_set_block_size(512)
                +-- xradio_cmd53_selftest(verbose = 1)
                |   +-- CMD53 读 HIF_CONFIG 两次，必须一致
                |   +-- CMD53 写/读 HIF_TSET_GEN 三组模式，再恢复原值
                +-- mmc_set_clock(12 MHz) + 自检；失败 → mmc_set_clock(0) 回 400 kHz
                +-- xradio_core_probe(func)                      xradio/main.c
                    +-- xradio_load_firmware(hw_priv)            xradio/fwio.c
                    |   +-- need_image("/boot/boot_xr829.bin")   经 VFS → FAT32 → SMHC0
                    |   +-- need_image("/boot/fw_xr829.bin")
                    |   +-- need_image("/boot/sdd_xr829.bin")    SDD 留在内存给 CONFIGURATION
                    |   +-- CONFIG 的 hw-type 位检查
                    |   +-- parse_sdd() → DPLL；写 TSET_GEN 并回读
                    |   +-- CONTROL.WUP，等 RDY；CONFIG |= ACCESS_MODE
                    |   +-- update_dpllctrl()
                    |   +-- download_bootloader()   AHB 0x08000000，逐字写
                    |   +-- download_firmware()     释放 CPU，握手，1 KiB 块写下载 FIFO
                    |   +-- CONFIG：开 IRQ 位，清 ACCESS_MODE → queue 模式
                    +-- wsm_init(func, 2000)                     xradio/wsm.c
                    |   +-- xradio_bh_init(func)                 xradio/bh.c
                    |   +-- 循环 xradio_bh_poll(10)，直到 STARTUP 0x0801
                    |   |   +-- parse_startup()：num_bufs、buf_size、固件版本
                    |   |       +-- xradio_bh_set_caps()          流控上限
                    |   +-- CONTROL = WUP                        保持唤醒
                    +-- xradio_setup_mac()
                    |   +-- wsm_configuration(MAC, SDD)          CONFIGURATION 0x0009
                    |   +-- wsm_read_mib(CURRENT_TX_POWER)       诊断
```

```
+-- ieee80211_alloc_hw(&xradio_ops, hw_priv)     net/mac80211/main.c
+-- ieee80211_register_hw(hw, mac, xradio_rates, 12)
    +-- wiphy_new() / wiphy_register()           cfg80211
    +-- xradio_start()                           ops->start
    |   +-- OPERATIONAL_POWER_MODE MIB
    |   +-- wsm_set_edca_default()               4 个 AC 的 EDCA
    |   +-- wsm_set_tx_policies()                速率重试表
    +-- register_netdev(wlan0)
        +-- NETDEV_REGISTER 通知 → wpa_netdev_event()   自动连接
```

5.6 初始化顺序与驱动核心约束

`device_register()` 会拿驱动核心锁并触发匹配绑定（probe 也在同一把锁下运行）。卡枚举要注册 `sdio_func` 设备，如果放在 SMHC 的 probe 里就会在持锁状态下再次注册设备——死锁。所以 `t113_driver_core_init()` 先把所有设备、驱动注册完，最后统一调 `mmc_rescan_all()`，对应 Linux 把 `mmc_rescan` 扔进 workqueue 的做法。

同理，注册顺序也有要求：`xradio_sdio_driver` 必须在 `mmc_rescan_all()` 之前注册，否则 function 设备注册时找不到驱动（本模型没有 deferred-probe 重试；未绑定时会打印 `no sdio driver matched`）。

另一个后果是：**整段固件下载都在 xradio_sdio_probe() 里、在驱动核心锁下忙等完成**。这在 Linux 里不可接受（probe 必须快、可睡眠），这里因为只有这一个 SDIO 设备、且下载在 12 MHz 下只需约 0.2 s 量级（失败回退到 400 kHz 时约数秒），所以先保持最简单的同步实现。后续接中断/工作队列时应把固件下载移出 probe。

`xradio_sdio_probe()` 即使 CMD53 或固件下载失败也返回 0，保持绑定关系，方便驱动模型打印出设备状态并在串口留下诊断信息（若 probe 返回负值，绑定不会建立，现场更难分析）。

5.7 图解：SDIO 协议与初始化

5.7.1 命令与响应的格式

SDIO 沿用 SD 卡的物理层：主机在 CMD 线上发 48 位命令，卡在 CMD 线上回响应，数据走 D0（4-bit 模式下走 D0–D3）。

```
命令（主机 → 卡，48 位）
 47   46   45 ..... 40   39 ................. 8   7 ... 1   0
+---+----+------------+----------------------+---------+---+
| 0 | 1  | 命令号 6 位 | 参数 32 位            | CRC7    | 1 |
+---+----+------------+----------------------+---------+---+
 起始 方向=主机                                        结束

SMHC 只要软件给出“命令号 + 参数 + 期望的响应类型”，起始位、CRC7 和结束位由硬件生成：
  SMHC_ARG = 参数
  SMHC_CMD = START | 命令号 | RESP_EXPIRE [| LONG_RESP] [| CHK_CRC] [| DATA | WAIT_PRE [| WRITE]]
  轮询 SMHC_RINT 等 CMD_DONE（有数据时再等 DATA_OVER）
  SMHC_RESP0 = 响应里的 32 位内容
```

SDIO 初始化用到的命令和响应：

命令	名称	参数	响应	miniOS 用它做什么
CMD0	GO_IDLE_STATE	0	无	复位卡的状态机；带 `INIT_SEQ` 位，让 SMHC 先发初始化时钟（规范要求至少 74 个）
CMD5	IO_SEND_OP_COND	0，之后是 OCR	R4	先问电压窗口，再反复协商直到就绪
CMD3	SEND_RELATIVE_ADDR	0	R6	卡给出 16 位 RCA
CMD7	SELECT_CARD	RCA « 16	R1b	选中卡，进入可传输状态
CMD52	IO_RW_DIRECT	见下	R5	读写一个字节：CCCR、FBR、CIS
CMD53	IO_RW_EXTENDED	见下	R5 + 数据	批量读写：HIF 寄存器、下载 FIFO、消息队列

响应里用到的位：

```
R4 (CMD5, 无 CRC)
 31   30..28      27   26..24   23 ............... 0
+---+----------+----+--------+-----------------+
| C | 功能数     | MP | 保留     | OCR 电压窗口      |   C = 1: 卡已就绪
+---+----------+----+--------+-----------------+   miniOS 取 OCR & 0x00ff8000 (2.7–3.6 V)

R6 (CMD3)
 31 ......... 16   15 ......... 0
+--------------+--------------+
| RCA          | 卡状态         |
+--------------+--------------+

R5 (CMD52/CMD53)
 31 ... 16   15        14        13..12    11     10   9        8      7 ...... 0
+---------+---------+---------+---------+------+----+--------+------+----------+
| 保留      | CRC 错  | 非法命令 | IO 状态  | 错误 | 0  | 功能号错| 越界  | 读到的数据 |
+---------+---------+---------+---------+------+----+--------+------+----------+
R5_ERROR_MASK = 0xcb00 (位 15、14、11、9、8)，命中就返回 -EIO
```

5.7.2 初始化状态机

```
sdio_attach_card(host)                          drivers/mmc/core.c
|
+-- smhc_power(host, 1)                         32k + WL_REG_ON 时序
+-- set_ios(400 kHz, 1-bit)
|
+-- CMD0                        卡: 任何状态 → Idle
+-- CMD5  arg = 0       → R4    读 OCR 和功能数，不改变状态
|     OCR & 0x00ff8000 == 0 → "card OCR has no 2.7-3.6V window"
+-- CMD5  arg = ocr     → R4    循环，每次间隔 10 ms，最多 100 次
|     R4.C == 1  →  卡: Idle → Ready
+-- CMD3                → R6    RCA; 卡: Ready → Stand-by
+-- CMD7  RCA << 16     → R1b   卡: Stand-by → Command (可以收发数据)
|
+-- CMD52 读 CCCR 0x00          CCCR/SDIO 版本
+-- CMD52 读 CCCR 0x08          能力位
+-- CMD52 读 CCCR 0x09..0x0b    公共 CIS 指针
|     +-- parse_cis_manfid()    沿元组链表找 CISTPL_MANFID(0x20)
|                               → vendor 0x0a9e, device 0x2282
+-- sdio_add_funcs(host)        对每个 function n = 1..功能数
```

```
      +-- CMD52 读 FBR n×0x100 + 0x00     标准接口类别
      +-- CMD52 读 FBR n×0x100 + 0x09     本 function 的 CIS 指针 → MANFID
      +-- device_register(&func->dev)     名字 "mmc1:0001:1"
            +-- sdio_bus_match()          按 xradio_sdio_ids[] 匹配
                  +-- xradio_sdio_probe(func)           xradio/sdio.c
                        +-- sdio_enable_func()
                        |   +-- CMD52 读改写 CCCR 0x02 IOEx |= 1 << 1
                        |   +-- CMD52 轮询 CCCR 0x03 IORx bit1   function 1 就绪
                        +-- sdio_set_block_size(512)
                        |   +-- CMD52 写 FBR1 + 0x10 = 0x00, + 0x11 = 0x02
                        +-- xradio_cmd53_selftest(verbose)      CMD53 能不能用
                        +-- mmc_set_clock(12 MHz) → 再自检一次，不行退回 400 kHz
                        +-- xradio_core_probe(func)           进入固件阶段
```

miniOS 没有发 CMD52 去写 CCCR 0x07（总线宽度），所以卡一直停在默认的 1-bit 模式；也没有用 CMD52 写 CCCR 0x04（IENx）打开卡中断。这两项是 Linux 的 `mmc_attach_sdio()` 会做、而 miniOS 刻意省掉的。

5.8 SDIO 枚举

5.8.1 CMD52：单字节读写

所有 CCCR/FBR 寄存器访问走 `sdio_io_rw_direct()`，命令参数位域：

```
bit 31     R/W        1 = 写
bit 30:28  function   0 = CCCR/FBR 空间，1..7 = 各 function 寄存器
bit 27     RAW        写后立即回读
bit 26     保留 0
bit 25:9   地址       17 位
bit  7:0   数据       写方向是数据，读方向是 stuff
```

响应 R5 的错误位掩码取 0xcb00（与 Linux `mmc_io_rw_direct()` 相同）：

```
bit 15     COM_CRC_ERROR     CRC 校验失败
bit 14     ILLEGAL_COMMAND
bit 13:12  IO_CURRENT_STATE  卡的 I/O 状态，不算错误
bit 11     ERROR             通用错误
bit  9     FUNCTION_NUMBER   function 号超范围
bit  8     OUT_OF_RANGE      地址超范围
```

封装（drivers/mmc/core.c）：

- `sdio_readb/writeb`：指定 function 的单字节读写；
- `sdio_f0_readb/writeb`：固定 function 0，即 CCCR/FBR 配置空间；
- `sdio_enable_func`：IOEx 置位 function 位后轮询 IORx 对应位，最长 1 秒；
- `sdio_set_block_size`：写 FBR+0x10/0x11，并同步记录到 `sdio_func.blksz`（CMD53 块模式要用）。

5.8.2 CCCR / FBR / CIS

CCCR（function 0 公共配置）：

地址	名称	作用
0x00	CCCR	低 4 位 CCCR 版本, 高 4 位 SD 版本
0x02	IOEx	各 function I/O 使能（写）
0x03	IORx	各 function 就绪状态（只读）
0x08	CAPS	能力（低功耗、DMA、高速支持）
0x09..0x0b	CIS	指向公共 CIS 元组链表的 24 位指针（小端）

FBR（每个 function n 一份，基址 `n*0x100`）：

偏移	名称	作用
+0x00	STD_IF	低 4 位 standard interface code；XR829 不用它匹配（驱动按 CIS 的 vendor/device 匹配）
+0x09..	CIS	该 function 自己的 CIS 指针
+0x10/0x11	BLKSIZE	块大小低/高字节

CIS 是元组链表：每项一字节 code + 一字节 link（长度），跟着内容。`parse_cis_manfid()` 沿链表走，找 `CISTPL_MANFID(0x20)`，取前 4 字节得到 vendor/device。

实测这块 100ASK T113-Pro 上的 XR829 报 **0x0a9e:0x2282**；开源 xradio 驱动里的老 XR819 是 0x0020:0x2281（ST-Ericsson 历史遗留 ID）。`xradio_sdio_ids[]` 两个都匹配，另外各留一条只限 vendor 的兜底项：

```
{ SDIO_ANY_ID, 0x0a9e, 0x2282 },        /* XR829, 本板实测 */
{ SDIO_ANY_ID, 0x0020, 0x2281 },        /* XR819 per xradio */
{ SDIO_ANY_ID, 0x0a9e, SDIO_ANY_ID },
{ SDIO_ANY_ID, 0x0020, SDIO_ANY_ID },
{ 0, 0, 0 },
```

5.9 CMD53 与数据通路

`sdio_io_rw_extended()` 实现 CMD53，参数位域：

```
bit 31     R/W        1 = 写
bit 30:28  function   function 1
bit 27     block      1 = 块模式, 0 = 字节模式
bit 26     incr       1 = 地址自增（内存型）, 0 = 固定地址（FIFO 口）
bit 25:9   地址        17 位
bit  8:0   count       字节模式 = 字节数（512 编码为 0）
                       块模式 = 块数
```

约束与选择逻辑（都强制 4 字节对齐）：

条件	模式	说明
`len <= 512`	字节模式	单次传输，`data.blksz = len, blocks = 1`
`len > 512`	块模式	`len` 必须是 `f->blksz` 的整数倍，块数 ≤ 511

对外接口：

- `sdio_memcpy_fromio(func, dst, addr, len)` / `sdio_memcpy_toio()`：地址自增的 CMD53，HIF 的寄存器、FIFO、队列都走它；
- HIF 寄存器访问封装成 `xradio_reg_read_32/write_32()`（4 字节，hwio.c），声明在 `mmc.h` 末尾；
- 1 KiB 固件块 = 2 × 512 字节块传输。

5.9.1 SMHC PIO 数据阶段

数据阶段不用 DMA、不用中断，CPU 直接搬 FIFO：

```
smhc_request() 带 mmc_data 时：
+-- CMD 寄存器置 DATA(b9) | WAIT_PRE(b13) | WRITE(b10)
+-- GCTRL |= AHB(b31) | FIFO_RST(b1)，等 FIFO_RST 自清
+-- BLKSZ = blksz，BYTECNT = blksz * blocks
+-- 启动命令后进 smhc_pio()
|   +-- 写：STATUS.FIFO_FULL(b3) 为 0 时写 SMHC_FIFO(0x200)
|   +-- 读：STATUS.FIFO_EMPTY(b2) 为 0 时读 SMHC_FIFO
|   +-- 每搬 4 字节检查一次 RINT 错误位，有错立即 -EIO
|   +-- 连续 ~200 ms 没有进展 -> -ETIMEDOUT
+-- 等 RINT.CMD_DONE(b2) | DATA_OVER(b3)
```

`SMHC_FIFO` 在偏移 `0x200`（D1/T113 的 NCAT2 布局）。出错时和纯命令路径一样：`smhc_reset()` + 重新应用时钟，避免状态机卡死。

5.10 SMHC 主机控制器

`sunxi-smhc.c` 是命令级轮询驱动：命令 + PIO 数据，不接中断。

用到的寄存器（偏移）：

偏移	名称	用途
0x000	GCTRL	软/FIFO/DMA 复位（`0x7`），bit1 FIFO 复位，bit31 CPU 访存 FIFO
0x004	CLKCR	bit16 时钟使能, bit17 省电, bit31 MASK_D0, [7:0] 分频
0x008	TMOUT	超时配置（初始化写 `0xffffff40`）
0x00c	WIDTH	0 = 1-bit, 1 = 4-bit
0x010	BLKSZ	数据阶段块大小
0x014	BYTECNT	数据阶段总字节数
0x018	CMD	bit31 启动，b21 UPCLK_ONLY，b15 INIT_SEQ，b13 WAIT_PRE, b10 WRITE, b9 DATA, b8/7/6 CRC/长响应/响应
0x01c	ARG	命令参数
0x020	RESP0	32 位响应（136 位响应在 RESP0..3）
0x030	IMASK	全 0：不用中断
0x038	RINT	b2 命令完成, b3 数据完成, b8 响应超时, 错误掩码 0xbfc2
0x03c	STATUS	b2 FIFO 空, b3 FIFO 满, b9 卡忙（R1b 判忙用）
0x05c	NTSR	bit31 new timing 模式
0x078	HWRST	0 = 复位, 1 = 释放
0x200	FIFO	PIO 数据端口（NCAT2）

时钟（CCU，ccu-sun20i-d1 布局）：

- 模块时钟 0x830 + 4N：bit31 gate，bit24-26 mux（0 = HOSC 24 MHz），P b8-9、M b0-3，**固定后分频 2**，即有效源时钟 12 MHz；
- 总线 gate/复位 0x84c：gate bit N，reset bit 16+N；
- fanout gate 0xf30 bit4、fanout 配置 0xf3c：每通道 3 位源选择（0 = 32k）+ bit21+n gate。

smhc_set_ios() 的换频序列是 D1/T113 的坑点，Linux sunxi-mmc 同款：

1. 关 CLKCR_ENABLE、置 MASK_D0，发一次 UPCLK_ONLY 命令；
2. 写新分频 + ENABLE + MASK_D0，再发 UPCLK_ONLY；
3. 清掉 MASK_D0。

换频期间必须把 DATA0 屏蔽掉，否则卡时钟抖动会让总线上出现非法电平。分频公式 div = ceil(12M / (2*hz)) 保证实际时钟只会更低：

目标	分频	实际时钟
400 kHz（识别/下载回退）	15	400 kHz
12 MHz（源时钟上限，提速目标）	0	12 MHz

XR829 驱动用 mmc_set_clock(host, 12 MHz) 提速，mmc_set_clock(host, 0) 回退到 host 的 f_init（400 kHz）。12 MHz 是本板 SMHC 源时钟（HOSC 24 MHz 固定 /2）能给出的上限，配合 1-bit 总线；没有做 high-speed 模式（那需要 CMD6 切时序）。

命令超时/错误处理：轮询 RINT，命中错误掩码就按 RESP_TMO 区分 -ETIMEDOUT / -EIO；出错后控制器状态机可能卡死，所以每次错误都做 smhc_reset() + 重设时钟再返回。

5.11 HIF 寄存器与间接寻址

XR829 的 host 接口（HIF）固定在 SDIO function 1 的地址空间，遵循 xradio hwio.h 的寻址方式：

```
SDIO 地址 = (buf_id << 7) | (reg_id << 2)
```

即 32 个 128 字节的 buffer 窗口；寄存器 id 在窗口内按 4 字节步进（本驱动用到 0..6）。direct 模式用 buf_id 0 访问寄存器：

reg_id	名称	作用
0	HIF_CONFIG	全局配置：访问模式、prefetch、CPU 复位/时钟、IRQ 使能、hw-type
1	HIF_CONTROL	WUP/RDY 握手、NEXT_LEN（待收消息长度）
2	HIF_IN_OUT_QUEUE	queue 模式的数据窗口
3	HIF_AHB_DPORT	AHB 间接访问端口
4	HIF_SRAM_BASE_ADDR	间接访问的目标地址
5	HIF_SRAM_DPORT	APB/SRAM 间接访问端口
6	HIF_TSET_GEN_R_W	DPLL 配置寄存器（写完由硬件读取）

CONFIG / CONTROL 关键位：

```
CONFIG b10 ACCESS_MODE      1 = direct（端口）模式，0 = queue（消息）模式
CONFIG b11 AHB_PFETCH       AHB 读预取中
CONFIG b12 CPU_CLK_DIS      CPU 时钟关
```

```
CONFIG b13 PFETCH          SRAM 读预取中
CONFIG b14 CPU_RESET       CPU 复位
CONFIG b16|b17 IRQ 使能
CONFIG b26 hw-type 标识    固件下载前必须为 1
CONTROL b11:0 NEXT_LEN     下一条消息长度（16-bit 字为单位）
CONTROL b12 WUP            写 0 唤醒设备
CONTROL b13 RDY            设备就绪
```

设备内存不在 SDIO 地址空间里，要通过间接窗口访问：

```
写：  SRAM_BASE_ADDR = 目标地址
      CMD53 写 SRAM_DPORT（APB 空间）或 AHB_DPORT（AHB 空间）

读：  SRAM_BASE_ADDR = 目标地址
      CONFIG |= PFETCH（SRAM）或 AHB_PFETCH（AHB）
      轮询 CONFIG 直到该位清零（退避等待，最多约 190 ms）
      CMD53 读对应端口
```

地址空间：

- AHB 0x08000000 起：设备 CPU 的内存，bootloader 镜像写到这里；
- APB 0x09000000 起：设备外设空间，下载 FIFO/控制块在这里；
- AHB 0x0ac80000：WLAN 电源控制（PWRCTRL）寄存器组。

所有 HIF 传输都是地址自增的 CMD53（与 xradio 的 hwio 一致），寄存器访问是 4 字节字节模式传输。

5.12 固件下载

5.12.1 镜像

bootpack 名称	文件	大小	加载地址	作用
boot_xr829	boot_xr829.bin	2184 B	0x40100000	设备侧下载器（写 AHB 内存后由 CPU 执行）
fw_xr829	fw_xr829.bin	174624 B	0x40110000	主固件，经下载 FIFO 灌入
sdd_xr829	sdd_xr829.bin	744 B	0x40140000	RF/时钟校准数据，取参考时钟频率

目录里另有 etf_xr829.bin、fw_xr829_bt.bin（蓝牙固件），当前不打包。

5.12.2 流程

xradio_load_firmware() 按 xradio fwio.c xradio_load_firmware() 的顺序执行：

1. need_image() 经 VFS 从 /boot 读三个镜像（正常是 FAT32；FAT32 挂载失败时是 bootfs 的 DDR 视图）。缺任何一个就打印 cannot load < 路径 > 并放弃，xradio_sdio_probe() 仍返回 0，保持绑定。
2. CMD53 读 CONFIG，检查 hw-type 位（bit26）：不是 XRadio HIF 就退出。
3. 解析 SDD：遍历 {u8 id, u8 len, data} 元素，取 0xc5（reference frequency，kHz，小端 16 位），按表换算 DPLL 值：

参考时钟	DPLL 值
13 MHz	0x1d89d241
16 MHz	0x000001e1
16.8 MHz	0x124931c1
19.2 MHz	0x00000191
24 MHz	0x00000141
26 MHz	0x0ec4f121
33.6 MHz	0x092490e1
38.4 MHz	0x100010c1
40 MHz	0x000000c1
48 MHz	0x000000a1
52 MHz	0x07627091

4. CMD53 把 DPLL 写进 TSET_GEN_R_W，延迟 5 ms 后回读校验。
5. CONTROL.WUP 写 0 唤醒设备，退避轮询 CONTROL.RDY（累计约 0.6 s）。
6. CONFIG 置 ACCESS_MODE（direct 端口模式）。
7. `update_dpllctrl()`：在 AHB 0x0ac80000 的 PWRCTRL 上等 WLAN 仲裁（ARBT 且非 BT），把 DPLL 值写进 DPLL_CTRL（带使能 bit0），再置 COMMON_CFG.DONE；最后清 START_CFG。
8. `download_bootloader()`：把 boot_xr829 以 32 位字逐个写入 AHB 0x08000000（每字一次 CMD53 AHB 间接写）。
9. 清 CONFIG.CPU_RESET 和 CPU_CLK_DIS，释放设备 CPU。
10. 轮询 APB 的 IMAGE_SIZE 寄存器直到读到 `I_AM_HERE`（100 × 10 ms）：设备侧 downloader 已跑起来。

然后 `download_firmware()`：

1. 初始化控制块：IMAGE_SIZE = `ARE_YOU_HERE`，PUT/GET = 0，STATUS = `PENDING`，FLAGS = 0。
2. 再次检查块大小：1024 必须能被 `f->blksz` 整除且 `blksz ≤ 1024`，否则设成 512。
3. IMAGE_SIZE = 实际镜像字节数。
4. 以 1 KiB 为块循环（镜像 171 块）：
   - 读 STATUS，必须仍是 PENDING；
   - 数据拷进 1 KiB 对齐缓冲，最后一块补零；
   - 等 FIFO 空间：`put - get <= 32 KiB - 1 KiB`（留一块余量）；
   - CMD53 把 1 KiB 写到 FIFO 环内偏移 `put & 0x7fff`；
   - `put +=` 实际字节数，写回 PUT 寄存器；
   - 每 32 块打印一次进度（32/64/96/128/160 KiB）。
5. 退避轮询 STATUS 直到非 PENDING；必须等于 `SUCCESS`（0），否则打印 STATUS 和 TRACE_PC。
6. CONFIG 使能设备中断位（b16|b17），再清 ACCESS_MODE 切到 queue（消息）模式，延迟 10 ms。

5.12.3 下载 FIFO 协议

```
APB 0x09004000  32 KiB 下载 FIFO (put/get 环形窗口, 掩码 0x7fff)
APB 0x0900ff80  控制块:
  +0x00 IMAGE_SIZE    握手: ARE_YOU_HERE -> I_AM_HERE, 随后写镜像大小
  +0x04 FLAGS
  +0x08 PUT           生产者字节计数 (host 更新)
  +0x0c TRACE_PC      失败时设备侧 PC
  +0x10 GET           消费者字节计数 (设备更新)
  +0x14 STATUS        PENDING / SUCCESS / 错误码
```

常量：ARE_YOU_HERE=0x87654321、I_AM_HERE=0x12345678、PENDING=0xffffffff、SUCCESS=0。

这是典型的单生产者/单消费者环形队列：host 只推进 PUT，设备只推进 GET；host 靠 put - get 判断剩余空间。最后一块补零写入但 PUT 只按实际镜像长度推进，设备按 IMAGE_SIZE 截断。

5.12.4 进入 queue 模式

固件跑起来后，数据通路从“端口 + 间接地址”切换为“消息队列”：

- 读：HIF_IN_OUT_QUEUE 端口，RX buffer id 从 1 开始按 +1 & 3 轮转（buf_id_rx）；
- 写：同一端口用 TX buffer id（buf_id_tx 在 0..31 间轮转，见“通信协议”一节）；
- 消息长度由 CONTROL.NEXT_LEN 给出。

5.13 通信协议：从 SDIO 到 WSM

5.13.1 协议分层

```
+-------------------------------------------------------------+
| 802.11 帧（mac80211 组好的管理帧、数据帧）                    | net/mac80211
+-------------------------------------------------------------+
| WSM 消息：{u16 len, u16 id} + 负载                           | xradio/wsm.c
|   请求 0x00xx → confirm 0x04xx；固件主动发 indication 0x08xx |
+-------------------------------------------------------------+
| HIF 队列：IN_OUT_QUEUE 端口、buf_id 轮转、3 位序号、NEXT_LEN、 | xradio/bh.c
|          固件输入缓冲计数                                    |
+-------------------------------------------------------------+
| HIF 寄存器窗口：SDIO 地址 = HIF_ADDR(buf_id, reg_id)         | xradio/hwio.c
+-------------------------------------------------------------+
| SDIO：CMD52 一个字节（function 0 配置空间）                  | drivers/mmc/core.c
|       CMD53 多字节（function 1，字节模式 ⩽ 512 B，或 512 B 块模式） |
+-------------------------------------------------------------+
| SMHC1：命令寄存器 + FIFO，CPU 轮询搬运                       | mmc/host/sunxi-smhc.c
+-------------------------------------------------------------+
| CLK / CMD / D0                                              |
+-------------------------------------------------------------+
```

XR829 的 HIF 有两种工作模式，CONFIG.ACCESS_MODE（bit10）切换：

模式	何时	怎么访问设备
direct（ACCESS_MODE = 1）	固件下载期间	寄存器窗口 + SRAM_BASE_ADDR 间接访问 AHB/APB 内存
queue（ACCESS_MODE = 0）	固件跑起来以后	只通过 IN_OUT_QUEUE 端口收发 WSM 消息

5.13.2 HIF 地址

```
CMD53 参数里的 17 位地址（function 1）
 16 ... 12   11 ..... 7    6 ..... 2    1  0
+----------+------------+----------+------+
|    0     | buf_id 5 位 | reg_id   | 0  0 |
+----------+------------+----------+------+
HIF_ADDR(buf_id, reg_id) = (buf_id & 0x1f) << 7 | reg_id << 2

寄存器访问：buf_id = 0，reg_id = 0..6（CONFIG、CONTROL、IN_OUT_QUEUE、AHB_DPORT、
                                  SRAM_BASE_ADDR、SRAM_DPORT、TSET_GEN）
队列写：    buf_id = tx 序号 0..31，reg_id = 2      每写一条消息 +1
```

```
队列读：    buf_id = rx 序号 + 1 (1..4) , reg_id = 2  每读一次 +1, 模 4
```

5.13.3 WSM 消息格式

```
字节  0 ..... 1     2 ............... 3     4 ...
   +-------------+---------------------+-------------+
   | len (LE)    | id (LE)             | 负载        |   len 含这 4 字节头
   +-------------+---------------------+-------------+

id 的位
 15 ... 13   12   11   10    9 ..... 6     5 ..... 0
+---------+----+----+----+-----------+--------------+
| 序号     |保留| IND| CFM| 接口/链路号| 消息号       |
+---------+----+----+----+-----------+--------------+
WSM_MSG_ID(id) = id & 0x0c3f     去掉序号和链路号
0x00xx 请求    0x04xx confirm (= 请求 | 0x0400)    0x08xx indication
```

TX 请求（`wsm_tx_frame()`，0x0004）在 4 字节头之后还有 20 字节，一共 24 字节：

偏移	字段	miniOS 填什么
4	packetID（u32）	mac80211 的 cookie，TX confirm 原样带回
8	queueId	链路号左移 2 位再加队列号；STA 的链路号是 0，队列 VO 3、VI 2、BE 0、BK 1
9	more	0
10	DataOffset	24，802.11 帧从这里开始
11	EPTA 优先级	0
12	TxRateEntry（u16）	管理帧 0x000f，数据帧 0x00a4
14	TxPolicyIndex	管理帧 0，数据帧 1
15	ExpireTimeSetting	0
16	expireTime（u32）+ 保留	0，用固件默认
24	802.11 帧	头 + 数据，不带 FCS

RX indication（0x0804）的 16 字节头：

偏移（相对负载）	字段
0	status（u32），非 0 表示出错或没有钥匙，驱动丢弃
4	channel（u16）
6	收到的速率表项（u16）
11	RCPI/RSSI
12	flags（u32），低 3 位是加密类型：0 无、2 TKIP、3 AES
16	802.11 帧（固件已解密，但 IV 和 MIC 还在）

用到的消息：

方向	id	名字	谁处理
请求	0x0004	TX	`wsm_tx_frame()`
请求	0x0005 / 0x0006	READ_MIB / WRITE_MIB	`wsm_read_mib()` / `wsm_write_mib()`
请求	0x0007	START_SCAN	`wsm_scan()`
请求	0x0009	CONFIGURATION（带 SDD）	`wsm_configuration()`
请求	0x000b	JOIN	`wsm_join()`
请求	0x000c / 0x000d	ADD_KEY / REMOVE_KEY	`wsm_add_key_*()` / `wsm_remove_key()`
请求	0x0010 / 0x0011 / 0x0013	SET_PM / SET_BSS_PARAMS / SET_EDCA	`sta.c`
confirm	0x0404 / 0x041e	TX confirm / 多个 TX confirm	`xradio_tx_confirm_cb()`
indication	0x0800	EXCEPTION	打印并 `xradio_bh_halt()`
indication	0x0801	STARTUP	`parse_startup()`
indication	0x0804	RX	`xradio_rx_cb()`
indication	0x0805	EVENT（BSS_LOST 等）	`xradio_event_cb()`
indication	0x0806	SCAN_COMPLETE	`xradio_scan_complete_cb()`
indication	0x0809	SET_PM 完成	记标志

5.13.4 一次队列传输

```
读 (rx_one)
+-- 有上次捎带的 CONTROL 就用它，否则 CMD53 读 HIF_CONTROL (4 字节)
+-- len = NEXT_LEN × 2 (NEXT_LEN 以 16 位字为单位)；为 0 表示没有消息
+-- rd = align_len(len + 2)          ⩽ 512 → 4 字节对齐的字节模式
|                                     > 512 → 512 B 的整数块
+-- CMD53 读 HIF_ADDR(buf_id_rx + 1, IN_OUT_QUEUE), rd 字节
+-- 末尾 2 字节 = 下一个 CONTROL 值 (捎带，省一次寄存器读)
+-- 校验 3 位序号 (期望 rx_seq，之后 +1 模 8)
+-- 一次传输里可能有多条消息 (multi-rx)，逐条:
    +-- release_buffers()           confirm 归还固件输入缓冲
    +-- wsm_handle_rx()

写 (xradio_bh_tx)
+-- while hw_bufs_used ⩾ num_bufs: xradio_bh_poll(10)   等 confirm 还缓冲，最多 2 s
+-- 写 len，把 tx_seq 填进 id 的 15..13 位
+-- wlen = align_len(len)，短消息至少 16 字节，补零
+-- CMD53 写 HIF_ADDR(buf_id_tx, IN_OUT_QUEUE)
+-- buf_id_tx = (buf_id_tx + 1) & 31; tx_seq++; hw_bufs_used++
```

5.13.5 流控：固件输入缓冲

STARTUP indication 告诉主机固件有几个输入缓冲（`num_bufs`）、每个多大（`buf_size`）。每个请求（包括 TX）占一个，每个 confirm 还一个：

```
主机 hw_bufs_used                          固件输入缓冲 (num_bufs 个)
0 -- wsm_cmd(JOIN) - CMD53 写 ----------→ [J][ ][ ][ ]...        1
1 -- wsm_tx_frame(EAPOL M2) -------------→ [J][T][ ][ ]...       2
2 ←-- JOIN confirm 0x040b - CMD53 读 ------ [ ][T][ ][ ]...       1   release_buffers()
1 ←-- TX confirm 0x0404 (packetID) -------- [ ][ ][ ][ ]...       0
满 → xradio_bh_tx() 先轮询收 confirm，腾出缓冲再写
```

5.13.6 命令和 confirm 的配对

```
wsm_cmd(req, payload, len, cfm, cap, timeout)
+-- mutex_lock(&command_mutex)                一次只有一个命令在等
+-- wait_cfm = req | 0x0400, cfm_done = 0
+-- xradio_bh_tx(cmd_buf)
+-- while !cfm_done: xradio_bh_poll(5)         自己驱动底半部
|   +-- rx_one → wsm_handle_rx
|       +-- id == wait_cfm → 拷负载, cfm_status = 负载前 4 字节, cfm_done = 1
|       +-- TX confirm → ieee80211_tx_status()   照常分派，不影响等待
|       +-- indication → 各自回调（只入队或记标志）
+-- 返回 cfm_status（0 = WSM_STATUS_SUCCESS）
```

Linux 的 xradio 用 `wait_event_timeout()` 睡眠，由 bh 线程收到 confirm 后唤醒；miniOS 没有中断也没有 bh 线程，等 confirm 的人自己调 `xradio_bh_poll()`。为防止回调里再次进入底半部，`bh.in_rx` 会让嵌套的 `xradio_bh_poll()` 返回 `-EBUSY`。

5.13.7 启动握手

固件下载完、切到 queue 模式后，固件先主动发 STARTUP indication。驱动在它之前不能发任何命令：

```
xradio_core_probe()
+-- xradio_load_firmware()        结束时 ACCESS_MODE = 0（queue 模式）
+-- wsm_init(func, 2000)
|   +-- xradio_bh_init()          num_bufs = 1，序号和 buf_id 清零
|   +-- 循环 xradio_bh_poll(10)
|       +-- wsm_handle_rx(0x0801) → parse_startup()
|           +-- num_bufs、buf_size → xradio_bh_set_caps()
|           +-- hw_id、fw_type、fw_api、固件标签
|           +-- status == 0 → wsm.ready = 1
+-- CONTROL = WUP                 一直保持唤醒（不做省电）
+-- wsm_configuration(MAC, SDD)   第一条命令必须是 CONFIGURATION：把 SDD 交给固件
    +-- confirm 带回固件实际使用的 MAC、频段和速率掩码
```

5.14 固件打包与加载（bootpack，现为后备路径）

注：现在固件的正常来源是内核直接读写 SD 卡 FAT32 分区上的 `/boot/*.bin`（见第 3 章）；本节的 bootpack 预载 + DDR 交接是 FAT32 挂载失败时的后备路径。

固件比 SRAM 大得多，必须放在 DDR 里由 SPL 预加载。miniOS 用一个小容器把 kernel 和附加镜像一起放到 SD 卡：

```
SD 8 KiB      Boot0（miniBoot SPL）
SD 128 KiB    bootpack: 512 B header + 镜像，每项 512 对齐
```

bootpack header（`src/include/t113/bootpack.h`，miniBoot、miniOS、mkbootpack 三方共用）：

字段	说明
magic / version	"MPAK" / 1
count / total_size	条目数（≤8）/ 总字节数
entry[]	name[16]、offset、size、load_addr、crc32
header_crc32	除本字段外整个 header 的 CRC32

SPL 按条目把镜像拷到 load_addr 并校验 CRC32，再把 header 复制到 BOOTPACK_DDR_ADDR = 0x401ff000，最后跳到 kernel。内核通过 t113_bootpack_find(name) 查条目，启动时 t113_bootpack_dump() 打印全部条目。

DDR 布局（AUX 窗口是 kernel 下方的 2 MiB）：

```
0x40000000  AUX_BASE
0x40100000  boot_xr829.bin   (2184 B)
0x40110000  fw_xr829.bin     (174624 B, 结束于 0x4013aa20)
0x40140000  sdd_xr829.bin    (744 B)
0x401ff000  bootpack header 副本
0x40200000  kernel（页分配器从这里往后）
```

打包由 miniBoot/tools/mkbootpack.c 完成：

```
mkbootpack bootpack.bin kernel=minios-kernel.bin@0x40200000 \
    boot_xr829=.../boot_xr829.bin@0x40100000 \
    fw_xr829=.../fw_xr829.bin@0x40110000 \
    sdd_xr829=.../sdd_xr829.bin@0x40140000
```

顶层 rpi_bm/ Makefile 会自动处理：firmware/ xr829/ 下 boot/ fw/ sdd 三个文件齐全时填入 BOOTPACK_EXTRA，否则打印 WiFi firmware not packed 并只打包 kernel。安装时 Boot0 写 8 KiB、bootpack 写 128 KiB：

```
make t113
make install T113_DISK=/dev/目标设备
```

5.15 图解：数据流

5.15.1 发送：一个 UDP 包到空口

```
用户态 tftp / ping
  | svc: T113_SYS_TFTP / T113_SYS_NETCTL
  v
udp_sendto(s, ip, port, data, len)          net/ipv4/udp.c, 加 rtnl_lock
  | skb = alloc_skb(len)                     头部预留 NET_SKB_PAD = 128 B
  | skb_push(8)                              UDP 头
  v
ip_queue_xmit(skb, daddr, IPPROTO_UDP)      net/ipv4/ip_output.c
  | skb_push(20)                             IP 头 + 校验和
  | arp_resolve(dev, 下一跳, mac)            表里没有就发 ARP 请求并返回 -EBUSY
  | eth_header(): skb_push(14)               以太网头
  v
dev_queue_xmit(skb)                         net/core/dev.c, 检查 carrier
  | dev->netdev_ops->ndo_start_xmit
```

```
  v
ieee80211_subif_start_xmit(skb, wlan0)        net/mac80211/tx.c
  | 取出目的 MAC 和 ethertype，skb_pull(14)
  | ieee80211_build_data_hdr()
  |   +-- LLC/SNAP 8 B (AA AA 03 00 00 00 + ethertype)
  |   +-- 有 PTK 时头部预留 CCMP IV 8 B、尾部预留 MIC 8 B，置 Protected 位
  |   +-- 802.11 头 24 B：ToDS，A1 = BSSID，A2 = 本机，A3 = 目的
  v
ieee80211_tx_skb(local, skb, AC_BE, ...)
  | cookie = ++next_cookie，登记 tx_status[cookie % N] = PENDING
  | local->hw.ops->tx
  v
xradio_tx(hw, skb)                            xradio/txrx.c
  | AC → WSM 队列；数据帧用策略 1、速率表 0x00a4
  v
wsm_tx_frame(cookie, frame, len, queue, ...)    xradio/wsm.c
  | 拷进 tx_buf：24 B wsm_tx 头 + 802.11 帧         (之后 mac80211 释放 skb)
  v
xradio_bh_tx(tx_buf, len, cap)                xradio/bh.c
  | 等固件输入缓冲，写 len 和序号，对齐
  v
sdio_memcpy_toio(func, HIF_ADDR(buf_id_tx, IN_OUT_QUEUE), buf, wlen)
  | sdio_io_rw_extended()：CMD53，写，function 1，地址自增
  v
smhc_request() → smhc_pio()                    SMHC1：CPU 逐字写 FIFO (0x200)，1-bit 12 MHz
  v
XR829 固件：用 TK 做 CCMP 加密、算 MIC 和 FCS，按速率表发送和重试
  |
  +-- TX confirm 0x0404 {packetID = cookie, status}
        → 下一次 xradio_bh_poll() 收到 → xradio_tx_confirm_cb()
        → ieee80211_tx_status(hw, cookie, status)      记到 tx_status 槽
```

一路上 skb 只在两个地方被拷贝：用户数据进 skb 一次，skb 进 tx_buf 一次（因为 CMD53 要求 4 字节对齐的连续缓冲，而且长度要按块对齐）。

5.15.2 接收：从空口到 socket

```
XR829 固件：收帧、查 BSSID 过滤、用 TK/GTK 解密（保留 IV 和 MIC）
  | RX indication 0x0804 放进输出队列，CONTROL.NEXT_LEN ≠ 0
  v
（谁在轮询）udp_recvfrom / icmp_ping / wpa 握手 / net_poll
  +-- __net_poll(dev, 10)                      net/core/dev.c
      +-- ndo_poll = ieee80211_poll()          net/mac80211/main.c
          +-- ops->poll = xradio_poll() → xradio_bh_poll(10)      【底半部】
          |   +-- rx_one()
          |       +-- CMD53 读 IN_OUT_QUEUE      SMHC1 PIO 读 FIFO
          |       +-- wsm_handle_rx(msg)
          |           +-- xradio_rx_cb(msg, len)                  xradio/txrx.c
          |               +-- status ≠ 0 → 丢弃
          |               +-- alloc_skb，拷贝 802.11 帧
          |               +-- AES：头部后移 8 B 盖住 IV，尾部去掉 8 B MIC
          |               +-- rx_status：channel、signal、DECRYPTED | IV_STRIPPED | MIC_STRIPPED
          |               +-- ieee80211_rx_irqsafe(hw, skb)        只入 local->rx_queue
          +-- ieee80211_rx_tasklet(local)                          【tasklet】
              +-- ieee80211_rx_one(skb)
                  +-- 管理帧
                  |   +-- 扫描中 → ieee80211_scan_rx() → cfg80211 BSS 表
                  |   +-- 已加入 → ieee80211_rx_mgmt()：auth/assoc 响应、deauth
                  +-- 数据帧 → ieee80211_rx_data()
```

```
                    +-- 只收 FromDS 且来自本 BSSID 的
                    +-- 加密帧必须已被硬件解密
                    +-- ieee80211_data_to_8023()    802.11 + LLC → 以太网帧
                    +-- EAPOL (0x888e) → cfg80211_rx_control_port() → wpa_rx_eapol()
                    +-- 其他：端口已授权才收
                        +-- eth_type_trans() → netif_rx(skb)        入 backlog
    +-- net_rx_action()                                             【软中断】
        +-- deliver_skb()：按 skb->protocol 查 packet_type
            +-- ETH_P_ARP → arp_rcv()
            +-- ETH_P_IP  → ip_rcv()
                +-- ICMP → icmp_rcv()
                +-- UDP  → udp_rcv() → 按端口挂到 udp_sock.rx_queue
    +-- run_net_work()                                              【工作队列】
        +-- 例如 supplicant 的 key_work：M4 的 TX status 回来后装钥

udp_recvfrom() 看到 rx_queue 非空 → 出队拷给调用者
```

三个层次对应 Linux 的三种上下文。没有中断时，它们由同一次 `__net_poll()` 按顺序跑完：

层次	Linux	miniOS 里的位置	允许做什么
底半部	SDIO 中断→ xradio bh 线程	`xradio_bh_poll()`	读队列、分派；回调只能入队或记标志
tasklet	mac80211 RX tasklet	`ieee80211_rx_tasklet()`	处理 802.11 帧，可以发帧、下命令
软中断	NET_RX softirq	`net_rx_action()`	交给 IP/ARP，转到 socket
工作队列	workqueue	`run_net_work()`	需要等待的事，例如装钥

5.16 调用链：连接一个 WPA2 AP

```
register_netdev(wlan0) → NETDEV_REGISTER → wpa_netdev_event()
 (或用户命令 wifi → T113_SYS_NETCTL → NETCTL_WIFI)
+-- wpa_supplicant_connect(dev)                                  net/wpa/supplicant.c
    +-- rtnl_lock()
    +-- __wpa_connect(dev)
    |   +-- cfg80211_disconnect(); 清 IP
    |   +-- load_config()              读 /boot/wifi.txt：ssid、psk、可选静态 IP
    |   +-- cfg80211_scan(dev, 信道 1-13, 50-110 ms)
    |   |   +-- ieee80211_scan() → xradio_hw_scan()            xradio/scan.c
    |   |       +-- wsm_scan() → START_SCAN 0x0007
    |   |           beacon / probe resp → RX → ieee80211_scan_rx() → BSS 表
    |   |           SCAN_COMPLETE 0x0806 → ieee80211_scan_completed()
    |   +-- select_bss()               按 wifi.txt 的 SSID 查 BSS 表，没配置时试内置默认 SSID
    |   +-- build_rsn()                CCMP 单播，CCMP/TKIP 组播，PSK
    |   +-- wpa_pbkdf2(psk, ssid) → PMK; eapol_init()
    |   +-- cfg80211_connect(dev, {bss, RSN IE})                net/wireless/sme.c
    |   |   +-- ieee80211_mgd_auth()                           net/mac80211/mlme.c
    |   |   |   +-- bss_info_changed(BSSID) → xradio_do_join()
    |   |   |   |   +-- 预加入 MIB：电源模式、BA 策略、PMF 策略
    |   |   |   |   +-- wsm_join()            JOIN 0x000b：信道、BSSID、SSID、基本速率
    |   |   |   |   +-- 过滤 MIB：RX 过滤、beacon 过滤表/开关、BSSID 过滤
    |   |   |   +-- 发 Open System auth (管理帧，最低速率)，等 auth 响应
    |   |   +-- ieee80211_mgd_assoc()
    |   |       +-- 发 assoc request (带 RSN IE)，等 assoc 响应
    |   |       +-- bss_info_changed(ASSOC) → xradio_set_assoc()
    |   |           +-- 关联模式、keep-alive、BSS 参数 (AID)、beacon 唤醒周期、PM、slot
    |   +-- 等四次握手 (__net_poll 循环，最长 HANDSHAKE_TIMEOUT)
    |       M1 ← EAPOL 经 control port → eapol.c 生成 SNonce，算 PTK
    |       M2 → cfg80211_tx_control_port() → ieee80211_tx_control_port()
    |       M3 ← 校验 MIC，解出 GTK
    |       M4 → 发送；TX status 回来后 net_schedule_work(key_work)
```

```
|       key_work → install_keys()
|           +-- cfg80211_add_key() → ieee80211_cfg_add_key() → xradio_set_key()
|               +-- wsm_add_key_pairwise()   ADD_KEY 0x000c, 槽 0, AES 成对钥
|               +-- wsm_add_key_group()      槽 1, GTK + RSC
+-- cfg80211_port_authorized()     打开数据端口, carrier on, 打印 "wlan0: link up"
+-- rtnl_unlock()
+-- wifi.txt 有静态 IP → inet_set_addr(); 否则 ipconfig_dhcp()
```

5.17 T113 实现与 Linux xradio 驱动的区别

方面	Linux（Allwinner xradio + mac80211）	T113 miniOS
事件来源	SDIO 卡中断（DAT1）或 `WL_WAKE_AP` GPIO 中断，唤醒 bh 线程	没有中断。bh 在 `ieee80211_poll()`、等 confirm、等缓冲时被调用
总线	4-bit，可到 50 MHz 高速模式	1-bit，12 MHz（HOSC 24 MHz 固定 /2，是 SMHC 源时钟上限）
数据搬运	SMHC 内部 DMA（IDMAC）	CPU 读写 FIFO
固件来源	`request_firmware()`，从 `/lib/firmware`	`t113_vfs_open("/boot/*.bin")`，从 SD 卡 FAT32 分区；失败时用 SPL 预加载的 bootfs
固件下载时机	probe 很快返回，下载放在工作队列	在 `xradio_sdio_probe()` 里同步完成，此时持有驱动核心锁
bh	内核线程，等待队列 + 原子计数	普通函数 `xradio_bh_poll()`，单线程，`in_rx` 防重入
命令等待	`wait_event_timeout()` 睡眠	循环 `xradio_bh_poll(5)` 忙等
TX 队列	每个 AC 一个队列，按 link 流控、可重排	不排队，直接写；只用 `hw_bufs_used` 节流
虚拟接口	最多两个（STA + AP/P2P）	只有 STA，接口号 0
省电	PS-Poll、动态休眠、SDIO 睡眠唤醒	一直保持 WUP，PM 关闭
加密	硬件加解密	同样交给固件；驱动在 RX 时剥 IV/MIC，TX 时 mac80211 预留 IV 和 MIC 的位置
802.11n	HT、A-MPDU、40 MHz	只做 11b/g 关联，BA 策略写 0

5.18 预期串口输出

（地址、CRC、HIF 寄存器值和 WSM 数据以实测为准，下面是格式示例）

```
bootpack: kernel @0x40200000 bytes=0x????????
bootpack: boot_xr829 @0x40100000 bytes=0x00000888
bootpack: fw_xr829 @0x40110000 bytes=0x0002aa20
bootpack: sdd_xr829 @0x40140000 bytes=0x000002e8
sdc1: SMHC host ready (card scan deferred to rescan)
mmc1: SDIO card rca 0x0001 OCR 0xffff0000 funcs 1 vendor 0x0a9e device 0x2282
mmc1: func 1 class 0x0000 vendor 0x0a9e device 0x2282
mmc1:0001:1: XRadio WiFi 0x0a9e:0x2282 enabled (IOEx 0x0002 IORx 0x0002 blksz 0x0200)
mmc1:0001:1: HIF_CONFIG 0x???????? (stable) HIF_CONTROL 0x????????
mmc1:0001:1: CMD53 write/read-back OK (HIF_TSET_GEN, 3 patterns)
mmc1:0001:1: SDIO clock 12000 kHz
mmc1:0001:1: SDD ref clock 26000 kHz, DPLL 0x0ec4f121
mmc1:0001:1: device awake
mmc1:0001:1: bootloader loaded (2184 bytes)
mmc1:0001:1: bootloader answered I_AM_HERE
mmc1:0001:1: firmware 32 KiB
```

```
mmc1:0001:1: firmware 64 KiB
mmc1:0001:1: firmware 96 KiB
mmc1:0001:1: firmware 128 KiB
mmc1:0001:1: firmware 160 KiB
mmc1:0001:1: firmware download complete (174624 bytes)
mmc1:0001:1: firmware running
wsm: firmware startup: ? x ???? B input buffers, hw ?.?, type ?, api 0x????????
wsm: firmware label "..."
mmc1:0001:1: WSM command layer ready
mmc1:0001:1: configure firmware, SDD bytes=0x000002e8
mmc1:0001:1: configured MAC 02:0a:9e:22:82:01 bands=0x???????? rates=0x????????
```

5.19 排障提示

启动阶段：

- `mmc1: no SDIO card (CMD5 timeout)`——先查 WL_REG_ON 电平和 32k fanout 是否真的在输出（`smhc_dump()` 会打印 FANOUT_GATE/CFG 和 PG 组引脚配置）；
- `card OCR has no 2.7-3.6V window`——R4 回来了但 OCR 位图异常，多半是总线信号质量（CLK 上没串 33R 就容易出这个）；
- `card never became ready (CMD5)`——模组在初始化，可适当加大 `power_on_delay_ms`；
- `no sdio driver matched (N registered)`——vendor/device 不在 `xradio_sdio_ids[]` 里，看上一行打印的实测 ID；
- `cannot load /boot/..._xr829.bin`——`/boot` 下缺固件：把三个 `*_xr829.bin` 拷到 SD 卡的 FAT32 分区（`make install-firmware`），启动后用 `ls /boot` 确认；如果串口打印了 `fat32: mount failed`，再检查 bootpack 是否打包了固件。

CMD53 与固件下载：

打印	含义
`CMD53 not working; firmware download impossible`	HIF 寄存器读/写失败；查 pinmux、PIO FIFO 和总线时钟
`CMD53 fails at the fast clock, back to the init clock`	12 MHz 下总线不稳，已回退 400 kHz，下载变慢但仍可完成
`not an XRadio HIF (CONFIG)`	CONFIG 的 hw-type 位不是 1：芯片/固件不匹配或访问错窗口
`DPLL register did not stick`	TSET_GEN 写入没生效，检查间接/直连访问模式
`device did not wake up, CONTROL`	WUP 后 RDY 未置位：DPLL 或 32k 时钟问题
`DPLL access timeout, COMMON_CFG`	WLAN PWRCTRL 仲裁一直没拿到（BT 占用或时钟未起）
`bootloader not ready, IMAGE_SIZE reads`	boot_xr829 没跑起来：AHB 载入地址/CPU 释放/镜像损坏
`bootloader reported error`	设备侧 downloader 拒收，看 STATUS 值
`timeout waiting for download FIFO`	GET 长时间不推进，设备侧卡住
`firmware download failed, status ... trace pc ...`	下载失败，TRACE_PC 是设备侧出错位置
`wsm: no startup indication from firmware`	固件下载成功但没发 STARTUP indication，通常是 fw/sdd 版本不匹配

5.20 当前边界与下一步

硬件与固件链路（本章主线）已实现：

- SDIO 枚举、CCCR/FBR/CIS、CMD52；
- CMD53 字节/块模式 + SMHC PIO 数据阶段；
- HIF 寄存器、SRAM/AHB 间接寻址、prefetch；
- SDD 参考时钟解析、DPLL 配置、设备唤醒；
- bootloader 载入、1 KiB 分块固件下载、下载 FIFO 协议；
- queue 模式切换、HIF 队列收发（bh.c）与 WSM 协议（命令/confirm/indication、MIB）。

协议与数据面（见第 6 章）也已实现：硬件扫描、WSM JOIN、认证/关联、WPA2-PSK 四次握手与装钥、802.11 ↔ 以太网数据帧转换、ARP/IPv4/ICMP/UDP/DHCP、tftp 客户端与用户命令。

未实现（按优先级）：

1. SMHC/SDIO 中断（现在整条链路轮询，吞吐受限）；
2. 可选：SMHC DMA、high-speed 时序、蓝牙固件（fw_xr829_bt.bin）与 ETF 测试镜像。

其他刻意简化：卡/函数结构用静态池（`card_pool[3]`、`func_pool[21]`，外加 1 KiB 的静态下载缓冲）而不是分配器；所有等待用 `mdelay/udelay` 忙等；固件下载在 probe 内同步完成，尚未搬到工作队列。

第六章 网络栈与 WiFi 驱动分层

本章说明 miniOS 的网络代码按 Linux 驱动分层重新组织后的结构：每一层放在哪个目录、对应 Linux 的哪个模块、层与层之间通过什么接口说话，以及一个数据包从 UDP 走到 XR829 固件、再从空口回到 socket 的完整路径。

XR829 的 SDIO 枚举、HIF 寄存器、固件下载等硬件细节见第 5 章；`/boot`（FAT32）和 tftp 写文件见第 3 章。

6.1 分层总览

```
用户态  ifconfig / ping / tftp / wifi        (user-t113/*.c)
          | T113_SYS_NETCTL / T113_SYS_TFTP
------------------ src/kernel/syscall.c ------------------------------------------------------
          |
net/tftp   tftp.c (协议状态机)  tftp_udp.c (收进内存 → 写 /boot)
          | udp_open/sendto/recvfrom
net/ipv4   udp.c  icmp.c  ip_input.c  ip_output.c  arp.c
          devinet.c (地址)  ipconfig.c (DHCP)  af_inet.c (注册协议)
          | ip_rcv / arp_rcv  <--- packet_type ----  dev_queue_xmit
net/ethernet  eth.c (eth_header / eth_type_trans)
net/core   skbuff.c  dev.c (net_device, netif_rx, backlog, notifier, rtnl)
          | ndo_start_xmit / ndo_poll
============== struct net_device "wlan0" ========================================
net/wpa    supplicant.c (wpa_supplicant 的活)  eapol.c (四次握手)
          | cfg80211_scan / cfg80211_connect / control port / add_key
net/wireless (cfg80211)  core.c  scan.c (BSS 表)  sme.c (连接状态)  util.c
          | struct cfg80211_ops
net/mac80211  cfg.c  mlme.c (auth/assoc/deauth)  rx.c  tx.c  key.c  scan.c  main.c
          | struct ieee80211_ops            ↑ ieee80211_rx_irqsafe / tx_status
============== drivers/net/wireless/xradio ======================================
          sta.c (ops)  txrx.c  scan.c  main.c (probe/注册)
          wsm.c (WSM 命令/confirm/indication)
          bh.c  (HIF 队列收发)  fwio.c (固件下载)  hwio.c (HIF 寄存器)
          sdio.c (sdio_driver)
          | sdio_memcpy_toio / fromio (CMD53)
drivers/mmc  core.c (SDIO core)  host/sunxi-smhc.c
```

依赖只向下：驱动不知道 IP，IP 不知道 802.11，supplicant 只通过 cfg80211 接口操作无线设备。每一层的头文件放在 `src/include/net/` 下，名字与 Linux 一致（`skbuff.h`、`netdevice.h`、`cfg80211.h`、`mac80211.h` ...），驱动私有头文件留在驱动目录里（`xradio.h`、`wsm.h`、`hwio.h`），和 Linux 的 `drivers/net/wireless/xradio/` 一样。

6.2 目录与 Linux 对照

miniOS	Linux 对应	作用
src/net/core/skbuff.c	net/core/skbuff.c	固定池（32 个）的 sk_buff；skb_push/pull/put 让每层原地加减头部
src/net/core/dev.c	net/core/dev.c	register_netdev、notifier、netif_rx backlog、dev_queue_xmit、rtnl_lock、轮询入口 net_poll
src/net/ethernet/eth.c	net/ethernet/eth.c	以太网头的构造与解析
src/net/ipv4/*	net/ipv4/*	ARP、IPv4 收发、ICMP echo、UDP socket、地址配置、DHCP（ipconfig.c 对应内核 IP autoconfig）
src/net/wireless/*	net/wireless/（cfg80211）	wiphy、BSS 表、SME 连接状态、EAPOL control port、802.11↔802.3 转换
src/net/mac80211/*	net/mac80211/	SoftMAC：组 auth/assoc/deauth 帧、数据帧封装、密钥、扫描、RX tasklet
src/net/wpa/*	用户态 wpa_supplicant	读 /boot/wifi.txt、选 AP、RSN IE、PBKDF2、四次握手、装钥、开端口、DHCP
.../crypto/wpa_crypto.c	lib/crypto/	PBKDF2/ PRF/ HMAC-SHA1/ AES unwrap
src/net/tftp/*	无（应用协议）	TFTP 客户端
.../xradio/sdio.c	xradio/sdio.c	xradio_sdio_driver：probe、CMD53 自检、提速
.../hwio.c	xradio/hwio.c	HIF 寄存器、APB/AHB 间接访问
.../fwio.c	xradio/fwio.c	DPLL、bootloader、固件 FIFO 下载、SDD
.../bh.c	xradio/bh.c	底半部：HIF 队列读写、序号、固件输入缓冲计数
.../wsm.c	xradio/wsm.c	WSM 请求编码、confirm 等待、indication 分派
.../main.c	xradio/main.c	核心 probe：装固件 → WSM 启动 → CONFIGURATION → 注册到 mac80211
.../sta.c	xradio/sta.c	ieee80211_ops：start、JOIN 与关联参数、set_key、poll
.../scan.c	xradio/scan.c	hw_scan → WSM START_SCAN；完成通知
.../txrx.c	xradio/txrx.c	802.11 帧 ↔ WSM TX/RX；剥 IV/MIC；TX confirm

6.3 接口边界

驱动 ↔ mac80211（include/ net/ mac80211.h）：驱动实现 ieee80211_ops（start、tx、hw_scan、bss_info_changed、set_key、poll），在 probe 里 ieee80211_alloc_hw() + ieee80211_register_hw()。驱动向上只用四个函数：ieee80211_rx_irqsafe()、ieee80211_tx_status()、ieee80211_scan_completed()、ieee80211_beacon_loss()。驱动只搬帧和下命令，不构造任何 802.11 管理帧。

mac80211 ↔ cfg80211（include/net/cfg80211.h）：mac80211 实现 cfg80211_ops（scan、auth、assoc、deauth、add_key、tx_control_port）。cfg80211 保存扫描得到的 BSS 表，cfg80211_connect() 依次调 auth、assoc。

cfg80211 ↔ supplicant：supplicant 调 cfg80211_scan()、cfg80211_get_bss()、cfg80211_connect()、cfg80211_add_key()，EAPOL 帧经 control port 收发（Linux 里是 nl80211），握手完成后调 cfg80211_port_authorized() 打开数据端口、置 carrier。

net_device：mac80211 注册 wlan0，ndo_start_xmit 把以太网帧转成 802.11 数据帧，ndo_poll 驱动整个无线栈。IP 层只看到一个以太网设备。

6.4 一个包怎么走

发送（ping 192.168.1.1）：

```
icmp_ping → ip_queue_xmit (加 IP 头, arp_resolve 下一跳)
  → eth_header (加以太网头) → dev_queue_xmit
  → ieee80211_subif_start_xmit (去掉以太网头, 加 LLC/SNAP、CCMP 预留、802.11 头 ToDS)
  → ieee80211_tx_skb (cookie = WSM packet id) → xradio_tx
  → wsm_tx_frame (24 字节 wsm_tx 头, 队列/速率策略) → xradio_bh_tx → CMD53
```

接收：

```
xradio_bh_poll → wsm_handle_rx → xradio_rx_cb
  (WSM RX indication → skb, 剥 CCMP IV/MIC, 填 rx_status)
  → ieee80211_rx_irqsafe  [只入队]
mac80211 RX tasklet (ieee80211_poll 在驱动 poll 之后跑):
  管理帧 → 扫描中: cfg80211_inform_bss_frame; 已加入: mlme
  数据帧 → ieee80211_data_to_8023 → EAPOL: control port → supplicant
                                  → 其他: eth_type_trans → netif_rx
net_rx_action: backlog → ip_rcv / arp_rcv → icmp_rcv / udp_rcv → socket 队列
```

一个 skb 从 UDP payload 到 802.11 帧都是原地 skb_push，不复制；NET_SKB_PAD 128 字节的头部余量足够放 UDP+IP+ 以太网/802.11+CCMP+LLC。

6.5 连接流程

wlan0 注册时 supplicant 收到 NETDEV_REGISTER 通知，自动连接；wifi 命令走的是同一个 wpa_supplicant_connect()。

```
wpa: 读 /boot/wifi.txt
cfg80211_scan (信道 1-13 被动扫描) → mac80211 hw_scan → xradio scan.c → WSM START_SCAN
   beacon → BSS 表; SCAN_COMPLETE → ieee80211_scan_completed
wpa: 选 SSID, 校验 RSN (CCMP 单播, CCMP/TKIP 组播, PSK) , PBKDF2 算 PMK
cfg80211_connect
   → ieee80211_mgd_auth: bss_info_changed(BSSID) → xradio: 预加入 MIB + WSM JOIN + 过滤 MIB
                         发 Open System auth, 等 AP 回复
   → ieee80211_mgd_assoc: 发 assoc req (带 RSN IE) , 等 assoc resp
                          bss_info_changed(ASSOC) → xradio: 关联模式/BSS 参数/PM/slot MIB
EAPOL M1..M4 (control port) → eapol.c
   M4 的 TX status 回来后装钥: cfg80211_add_key → set_key → WSM ADD_KEY
```

```
cfg80211_port_authorized → carrier on → "wlan0: link up"
ipconfig_dhcp 或 wifi.txt 里的静态地址
```

6.6 执行上下文（没有中断时的规则）

miniOS 没有接 SDIO 中断，全部轮询，但仍按 Linux 的上下文划分来写：

- **底半部**：`xradio_bh_poll()` 读 HIF 队列并分派。它调用的回调（`ieee80211_rx_irqsafe`、`ieee80211_tx_status`、`ieee80211_scan_completed`、`ieee80211_beacon_loss`）只能入队或记录状态，不能发帧、不能下 WSM 命令。bh 用静态接收缓冲，`in_rx` 守卫禁止重入。
- **tasklet**：`ieee80211_poll()` 在驱动 poll 之后运行 mac80211 RX tasklet，这里可以发帧、下命令（例如装钥匙）。`in_tasklet` 防止嵌套。
- **工作项**：`net_schedule_work()` 相当于 workqueue，在 `__net_poll()` 末尾执行；supplicant 用它在 M4 发送完成后装钥。
- **rtnl_lock**：一把不可递归的大锁。对外入口（`udp_*`、`icmp_ping`、`wpa_supplicant_connect`、`net_poll`）自己加锁；带 `__` 前缀的函数要求调用者已持锁。`ipconfig_dhcp` 走加锁的 UDP 接口，所以不能在持锁时调用。
- 在 `net_rx_action` 里需要 ARP 时，`arp_resolve` 只发请求并返回 `-EBUSY`，不在接收路径里轮询。

6.7 Kconfig

```
Networking support              src/net/Kconfig
  NET                           core + ethernet + ipv4 + DHCP
  TFTP_CLIENT
  CFG80211                      src/net/wireless/Kconfig
  MAC80211                      src/net/mac80211/Kconfig
  WPA2_PSK                      src/net/wpa/Kconfig (依赖 CFG80211)
Device Drivers
  Network device support        src/drivers/net/Kconfig
    WLAN
      WLAN_XRADIO               依赖 WLAN && SDIO && MAC80211
        WLAN_XRADIO_XR829
```

Makefile 按同样的开关挑目标文件；树莓派（kernel8.img）构建排除整个 `src/net/` 和 `src/lib/crypto/`。

6.8 加一块新网卡要做什么

1. 在 `src/drivers/net/wireless/< 厂商 >/` 实现 `ieee80211_ops`；
2. probe 里 `ieee80211_alloc_hw()`、`ieee80211_register_hw(hw, mac, rates, n)`；
3. 底半部收帧时构造 skb、填 `IEEE80211_SKB_RXCB`，调 `ieee80211_rx_irqsafe()`；TX 完成调 `ieee80211_tx_status()`（带上 cookie 与 status）。

有线网卡则直接 `register_netdev()` 一个 `net_device`，实现 `ndo_start_xmit/ndo_poll`，收包时 `eth_type_trans()` + `netif_rx()`，IP 层和 tftp 不用改。

6.9 从旧结构的对应关系

旧文件	现在
src/net/net.c	net/core/*、net/ethernet/eth.c、net/ipv4/*
src/drivers/net/wireless/xradio/sta.c（扫描、JOIN、auth/assoc、EAPOL、数据帧转换）	802.11 状态机 → net/mac80211/；BSS 表与连接 → net/wireless/；wifi.txt/RSN/握手/装钥 → net/wpa/supplicant.c；固件 MIB 设置 → xradio/sta.c
.../xradio/xr829_core.c	sdio.c、hwio.c、fwio.c、main.c
.../xradio/wsm.c	传输 → bh.c；命令编码与分派 → wsm.c
src/net/eapol.c	src/net/wpa/eapol.c
src/net/tftp.c、tftp_udp.c	src/net/tftp/
src/lib/wpa_crypto.c、include/net/wpa_crypto.h	src/lib/crypto/、include/crypto/
include/net/net.h、include/net/xradio/*	include/net/ 下的 skbuff、netdevice、if_ether、ip、arp、icmp、udp、ipconfig、ieee80211、cfg80211、mac80211、wpa 等头文件；WSM 头移到驱动目录
xr829_driver	xradio_sdio_driver
xradio_sta_connect()	wpa_supplicant_connect(dev)
net_dhcp / net_ping / net_set_addr / net_parse_ip	ipconfig_dhcp / icmp_ping / inet_set_addr / in4_pton

固件交互的顺序和参数（预加入 MIB、JOIN、过滤 MIB、关联 MIB、钥匙槽位 0/1、速率策略）与旧实现逐项保持一致。

6.10 验证

- make t113（-Wall -Wextra）零警告，内核链接通过。
- 主机端联调：把 src/net、lib/crypto 和 xradio 的 wsm/main/sta/scan/txrx 在 x86 上用 ASan/UBSan 编译，bh.c 换成假固件，另一端是 Python 写的 WPA2 AP。跑通：扫描 → JOIN → 认证 → 关联 → 四次握手（双方 MIC 校验、装进固件的 TK/GTK 与 AP 一致）→ DHCP → ping 网关 → TFTP 下载 1000 字节并校验 → 组播密钥更新 → AP 踢下线（链路断开）→ 重新连接 → ping。
- 尚未上板：SDIO、固件真实行为需要在 T113 板子上确认。

上板时预期的串口日志（顺序）：

```
mmc1:0001:1: firmware running
wsm: firmware startup: ...
mmc1:0001:1: configured MAC 02:0a:9e:22:82:01 ...
cfg80211: phy0 registered, 12 bitrates
net: wlan0 registered, hwaddr 02:0a:9e:22:82:01
wpa: /boot/wifi.txt ssid="TP-Link_B114" psk=<set>
cfg80211: passive scan, 13 channels
cfg80211: AP ssid="..." bssid=... channel=... rcpi=... secured
wpa: selected SSID="TP-Link_B114" ...
wlan0: joining BSS
wlan0: open-system authentication
wlan0: association request
```

```
wlan0: associated, AID ...
wpa: WPA2 4-way handshake
wpa: WPA2 keys installed
wlan0: link up
dhcp: bound 192.168.x.x gateway ...
```

第七章 驱动：模型、网络驱动分层与 WiFi / UART 实现

前面各章从协议、文件系统或单条链路的角度描述过驱动的各个侧面。本章把它们收拢到 *驱动模型*这条主线上：miniOS 仿照 Linux 搭建了一个轻量的总线–设备–驱动核心（bus / device / driver），网络驱动按 Linux 的分层重新组织（第 6 章），XR829 挂在 SDIO 总线上（第 5 章），UART 作为最早的设备走 “裸驱动 + probe 登记” 的路线（第 4 章）。

本章内容：

- 驱动架构：kobject/kset、三种总线、probe 绑定与初始化顺序的锁约束；
- 网络驱动分层：`net_device` / `ieee80211_ops` / `cfg80211_ops` 边界，轮询如何顶替中断；
- SDIO WiFi 初始化：卡枚举 → function 绑定 → CMD53 自检 → 固件 → WSM 启动 → 注册 wlan0；
- 数据的接收与发送：HIF 队列、底半部、固件输入缓冲流控、skb 双向转换；
- 无线部分的发射：802.11 帧封装、CCMP 预留、速率重试策略、EDCA、EAPOL 特例；
- UART 串口驱动与 `/dev/console`：驱动模型视角的回顾。

7.1 驱动架构：总线、设备与绑定

7.1.1 kobject/kset：内存里的目录树

驱动核心建立在轻量 `kobject/kset` 之上（`kobject.c`）：kobject 有 `parent`（虚拟目录树中的位置）和 `kset`（成员列表），`kobject_get_path()` 沿 parent 链拼出 “/a/b/c” 风格路径。启动时 `t113_driver_core_dump()` 会把整棵树打印到串口；`CONFIG_SYSFS=y` 时这棵树还会被投影到 `/sys`，用户态可以直接 `ls`、`cat`，详见第 10 章：

```
/devices/platform/gic  /devices/platform/uart3  ...  kset "devices"
/bus/platform  /bus/i2c  /bus/sdio                   kset "bus"
/bus/platform/devices  /bus/platform/drivers/...      总线下的两个目录
/drivers/t113-uart  /drivers/xradio_wlan ...          kset "drivers"
```

miniOS	Linux	说明
`struct kobject/kset`	`kobject/kset`	parent/kset 与引用计数；由 sysfs 投影到 `/sys`
`struct bus_type`	`struct bus_type`	`match/probe/remove`，devices/drivers 两个链表
`struct device`	`struct device`	`platform_data`、`driver_data`
`struct device_driver`	`struct device_driver`	`probe/remove`、已绑定设备链表
`of_device_id.compatible`	device tree 匹配	纯字符串比较，DT 替身
`kref` 引用计数	`kref`	`get_device/put_device`

7.1.2 一把锁规则与初始化顺序

所有 `register/ unregister/ bind/ unbind` 由一**把驱动核心锁**串行化，而且这把锁 跨 *`->probe()/->remove()`* 持有。由此推出两条硬约束：

- probe/remove 里**不能睡眠、不能再调用注册 API**；
- 没有 deferred probe（Linux 用它重试依赖不满足的 probe），依赖只能用 **注册顺序**表达。

两个具体例子。其一是摄像头链：csi（提供 MCLK）和 twi（I2C 适配器）必须先绑定，ov5640 驱动才能后注册，注释原话是 “probe runs under the core lock and there is no deferred-probe retry yet”。其二是 SDIO 卡枚

举：device_register() 会注册设备，所以绝不能在任何 probe 里跑——Linux 把 mmc_rescan 挂在 workqueue 上，miniOS 则把它显式放在所有驱动注册完之后调用一次：

```
void t113_driver_core_init(void)
{
    kset_register(&devices, 0, "devices");      /* 三个顶层 kset */
    kset_register(&buses,   0, "bus");
    kset_register(&drivers, 0, "drivers");
    platform_bus_init();
    i2c_bus_init();
    sdio_bus_init();
    /* Devices first, then drivers: binding happens in driver_register(). */
    t113_board_register_devices();
    t113_board_register_drivers();
    /* Card enumeration registers devices: must run outside any probe. */
    mmc_rescan_all();
    t113_driver_core_dump();
}
```

设备先注册、驱动后注册，绑定发生在 driver_register() 里（遍历总线上未绑定的设备逐个尝试 match）。board.c 的数组顺序同时表达了电气依赖：中断控制器（gic）永远排第一，“和真实 Linux 按设备树顺序 probe 一样”。

7.1.3 平台设备：手写的设备树替身

board.c 用静态结构体描述 SoC 上内核实际驱动的块，地址/中断号取自 sun8i-t113s.dtsi：

设备	compatible	资源与平台数据
gic	arm,cortex-a7-gic	dist + cpu 两段 mem
uart3	allwinner,sun20i-d1-uart	mem + IRQ 37；probe 不碰硬件（见 7.6）
timer0	allwinner,sun20i-d1-timer	mem + IRQ
twi2	allwinner,sun20i-d1-i2c	mem；PE12/13，100 kHz
sdc1	allwinner,sun20i-d1-mmc	mem + IRQ 73；PG0..5 总线、WL_REG_ON、32k fanout

以 uart3 为例，资源用 Linux 风格的宏声明：

```
static const struct resource uart3_resources[] = {
    DEFINE_RES_MEM(T113_UART3_BASE, T113_UART3_SIZE),
    DEFINE_RES_IRQ(T113_IRQ_UART3),
};
```

sdc1 的平台数据最丰富：t113_mmc_platform_data 带总线引脚表（PG0..PG5，功能号 2，CMD/D0..D3 上拉）、power_gpio = PG12（WL_REG_ON）加 100 ms 上电延时，以及 fanout_pin——XR829 没有内部 RTC 时钟，32.768 kHz 由 CCU 的 CLK-FANOUT1 输出到 PG11。SMHC 驱动的 probe 把它注册成 mmc_host，smhc_power() 负责整条上电时序（fanout 门控、引脚复用、延时）。

7.2 网络驱动分层

7.2.1 从 ndo_ops 到 ieee80211_ops

miniOS 的网络代码按 Linux 的边界拆成 `src/net/`（core、ethernet、ipv4、wireless、mac80211、wpa、tftp）和 `src/drivers/net/wireless/xradio/`，协议栈的内部结构见第 6 章；从**驱动**的角度看，关键的只有三个接口层：

```
ipv4 / icmp / udp
   | dev_queue_xmit                    (rtnl_lock 约定, __net_poll)
   ▼
net_device "wlan0"                    net/core/dev.c
   | ndo_start_xmit = ieee80211_subif_start_xmit
   | ndo_poll      = ieee80211_poll      ← 轮询顶替中断 + tasklet
   ▼
mac80211  tx.c rx.c mlme.c            802.11 状态机与帧封装
   | struct ieee80211_ops               驱动必须实现的 6 个回调
   ▼
xradio   sta.c txrx.c bh.c wsm.c      帧搬运、WSM 协议、HIF 队列
   | sdio_memcpy_toio/fromio (CMD53)
   ▼
mmc core  drivers/mmc/core.c          CMD52/53、卡枚举、sdio 总线
   | mmc_host_ops.request/set_ios/power
   ▼
host     drivers/mmc/host/sunxi-smhc.c   SMHC1 寄存器 + PIO FIFO
```

依赖只向下：IP 层只看见一块以太网设备，mac80211 不认识 SDIO，驱动不构造任何管理帧。每层的头文件名也与 Linux 一致（`netdevice.h`、`mac80211.h`、`cfg80211.h`...）。

7.2.2 轮询顶替中断

miniOS 没有接 SDIO 中断（IRQ 73 已声明但未使用），Linux 里“中断 → 底半部 → tasklet”的链条全部由轮询驱动：

```c
/* net/core/dev.c: net_poll() -> __net_poll() */
int __net_poll(struct net_device *dev, int timeout_ms)
{
    int r = dev->netdev_ops->ndo_poll
                ? dev->netdev_ops->ndo_poll(dev, timeout_ms) : 0;
    net_rx_action();                    /* backlog -> ip_rcv / arp_rcv */
    net_run_work();                     /* workqueue 替身 */
    return r;
}

/* mac80211/main.c: wlan0 的 ndo_poll */
static int ieee80211_poll(struct net_device *dev, int timeout_ms)
{
    struct ieee80211_local *local = dev->ml_priv;
    int r = local->hw.ops->poll(&local->hw, timeout_ms);   /* 驱动底半部 */
    if (!local->in_tasklet)
        ieee80211_rx_tasklet(local);                       /* RX tasklet */
    return r;
```

```
}
```

注意顺序：先跑**驱动**的 poll（收 HIF 队列，等价 Linux 的 IRQ 底半部），再跑 mac80211 的 RX tasklet（分派帧、可以发帧下命令）。`ieee80211_poll` 是所有轮询的汇聚点：`wsm_cmd` 等 confirm、supplicant 等握手帧、`net_poll` 处理收包，最终都回到这里。

7.2.3 ieee80211_ops：驱动与 mac80211 的契约

驱动实现 6 个回调，注册时由 mac80211 在 `ieee80211_register_hw()` 里回调 `start` 做一次性 MAC 配置：

```
const struct ieee80211_ops xradio_ops = {
    .start           = xradio_start,           /* 电源模式/probe 模板/EDCA/重试策略 */
    .tx              = xradio_tx,              /* 帧 -> WSM TX 请求 */
    .hw_scan         = xradio_hw_scan,         /* WSM START_SCAN */
    .bss_info_changed= xradio_bss_info_changed,/* BSSID→JOIN, ASSOC→MIB 组 */
    .set_key         = xradio_set_key,         /* WSM ADD_KEY/REMOVE_KEY */
    .poll            = xradio_poll,            /* = xradio_bh_poll */
};
```

驱动向上只用四个函数：`ieee80211_rx_irqsafe()`（RX 入队）、`ieee80211_tx_status()`（TX confirm）、`ieee80211_scan_completed()`、`ieee80211_beacon_loss()`。再往上一层的 `cfg80211_ops` 边界与执行上下文规则（bh 不能发帧、tasklet 可以、`__` 前缀要求持 rtnl 锁）见第 6 章，此处不重复。

7.3 SDIO WiFi 初始化

SDIO 总线在 miniOS 里是三段结构（`src/include/t113/mmc.h`，对应 Linux `drivers/mmc/`）：

- `mmc_host`：控制器，由 SMHC 驱动 probe 时 `mmc_add_host()` 进静态表（最多 3 个）；
- `sdio_card`：枚举出的卡（RCA、OCR、CCCR 版本、function 数、从 CIS MANFID 读到的 vendor/device）；
- `sdio_func`：一个 SDIO function（1..7），内嵌一个 `struct device` 挂在 `sdio_bus_type` 上，名字形如 `"mmc1:0001:1"`（host:rca:func）。

function 驱动按 `id_table` 匹配（class/vendor/device，`SDIO_ANY_ID` 表示不关心）：

```
static const struct sdio_device_id xradio_sdio_ids[] = {
    { SDIO_ANY_ID, 0x0a9e, 0x2282 },        /* XR829（实测） */
    { SDIO_ANY_ID, 0x0020, 0x2281 },        /* XR819 */
    { SDIO_ANY_ID, 0x0a9e, SDIO_ANY_ID },   /* 其它 XRadio 芯片也绑定 */
    { SDIO_ANY_ID, 0x0020, SDIO_ANY_ID },
    { 0, 0, 0 },
};
```

7.3.1 卡枚举：CMD5 到绑定

```
mmc_rescan_all()                        所有驱动注册完之后调用一次
 └──── sdio_attach_card(host)
    ├──── ops->power(host, 1)             PG12 WL_REG_ON + 32k fanout + 100 ms
    ├──── ops->set_ios(400 kHz, 1-bit)    识别期时钟
    ├──── CMD0 GO_IDLE → CMD5 读 OCR (R4)
    |     校验 2.7-3.6 V 窗口，轮询 R4_READY (10 ms × 100)
    |     function 数 = R4[30:28]
```

```
    ├──── CMD3 取 RCA → CMD7 选中 (R1B)
    ├──── CMD52 读 CCCR 版本 / CAPS / CIS 指针 (CCCR 0x09)
    └──── 沿 CIS 元组链找 MANFID (0x20) → card->vendor/device
└──── sdio_add_funcs(host)
    ├──── 每个 function: 读 FBR (class @+0x00, per-func CIS @+0x09)
    └──── device_register(&func->dev)     sdio 总线上立即尝试绑定
```

CMD52/CMD53 的参数位域与 R5 错误掩码见第 5 章；软件层只需要记住 `sdio_io_rw_extended` 的三条约束：长度必须 4 字节对齐、缓冲区 4 字节对齐、长度 > 512 走块模式（须为 `func->blksz` 整数倍，最多 511 块），否则走字节模式。`sdio_enable_func()` 置 CCCR IOEx 位后轮询 IORx 等就绪（1 ms × 1000）；`sdio_set_block_size()` 写 FBR+0x10 的两个字节。

7.3.2 probe 链：从 function 到 wlan0

因为 xradio 驱动在 `mmc_rescan_all()` 之前就已注册，`device_register()` 会在枚举现场直接完成绑定：

```
xradio_sdio_probe(func)                          xradio/sdio.c
├──── sdio_enable_func()              IOEx 置位，轮询 IORx
├──── sdio_set_block_size(512)
├──── CMD53 自检
|     HIF_CONFIG 读两次必须一致（验证 32 位读稳定）
|     HIF_TSET_GEN 写读 3 组模式 0xa5a55a5a / 0x12345678 / 0
├──── mmc_set_clock(12 MHz) + 复测自检（失败回退 400 kHz）
└──── xradio_core_probe()                          xradio/main.c
    ├──── xradio_load_firmware()       固件下载（第~\ref{chap:wifi}~章）
    ├──── wsm_init(func, 2000)         轮询等 startup indication
    |     caps: num_bufs × buf_size, hw_id, fw 版本/label
    ├──── xradio_setup_mac()
    |     WSM CONFIGURATION（必须是其后的第一条普通命令）
    |     交 SDD，得 MAC 02:0a:9e:22:82:01
    └──── ieee80211_alloc_hw / ieee80211_register_hw
        ├──── wiphy_register()
        ├──── wlan0 net_device (ndo_start_xmit / ndo_poll)
        ├──── ops->start()             电源模式/probe 模板/EDCA/重试策略
        └──── register_netdev()
```

两处值得注意的取舍：

- CMD53 自检失败时 probe **返回 0 保持绑定**（“数据面还在 bring-up：报告，但留住绑定”），只有固件下载失败才真正放弃设备——这样串口上的寄存器诊断始终可见；
- 提速后必须**在 12 MHz 下复测自检**，不过关就退回识别时钟。174 KiB 固件在 400 kHz 1-bit 总线上要以秒计。

7.4 数据接收与发送

7.4.1 HIF 队列与底半部

`bh.c` 是驱动的心脏。它维护的队列状态全部是静态的：

```
static struct {
    struct sdio_func *func;
    uint32_t num_bufs;          /* 固件输入缓冲个数（startup 后更新）*/
    uint32_t buf_size;
    int hw_bufs_used;           /* 已占用的固件输入缓冲 */
```

```
    uint8_t tx_seq, rx_seq;      /* 3-bit 消息序号 */
    uint8_t buf_id_tx, buf_id_rx;/* TX buf_id 0..31, RX 0..3 */
    uint32_t next_ctrl;          /* 捎带回的 CONTROL, 0 = 读寄存器 */
    int halted, in_rx;
} bh;
```

队列语义（HIF 寄存器定义见第 5 章）：

- 发送写 HIF_IN_OUT_QUEUE，地址用 buf_id_tx （0..31 循环）；读用 buf_id_rx + 1（RX id 只有 0..3）；
- CONTROL.NEXT_LEN 给出下一条消息长度，单位是 16 位字（代码里 << 1）；
- 每次队列读会多返回 2 字节：**下一条**的 CONTROL 值（piggyback），下次可以不再读寄存器；
- 消息头里有 3-bit 序号，错序只告警不重传；
- 一次队列传输可以携带多条 RX indication（multi-rx），按长度字段切分；
- **流控的核心**：每个 TX 请求/命令占用一个固件输入缓冲，每个 confirm 释放一个（multi-TX-confirm 一次释放 N 个，id 0x041e 的首个字段是计数）。

7.4.2 发送侧流控与接收排水

```
int xradio_bh_tx(uint8_t *b, uint32_t len, uint32_t cap)
{
    uint64_t end = t113_time_us() + 2000000u;
    while (bh.hw_bufs_used >= (int)bh.num_bufs) {      /* 固件缓冲用尽 */
        if (xradio_bh_poll(10) < 0 || t113_time_us() > end)
            return -ETIMEDOUT;                         /* 边轮询边等 confirm */
    }
    if (bh.buf_size && len > bh.buf_size)
        return -EINVAL;
    put_unaligned_le16(len, b);                        /* len 字段 */
    put_unaligned_le16((get_unaligned_le16(b + 2) & 0x1fffu) |
                       ((uint32_t)bh.tx_seq << 13), b + 2);  /* 序号入 id 字段 */
    uint32_t wlen = align_len(len <= 8u ? 16u : len);  /* 4 对齐/块对齐 */
    int ret = sdio_memcpy_toio(f, HIF_ADDR(bh.buf_id_tx,
                                           HIF_IN_OUT_QUEUE_REG_ID), b, wlen);
    bh.buf_id_tx = (bh.buf_id_tx + 1u) & 31u;
    bh.tx_seq = (bh.tx_seq + 1u) & 7u;
    ++bh.hw_bufs_used;
    return ret;
}
```

读方向由 xradio_bh_poll() 驱动，in_rx 守卫保证底半部不可重入（handler 里再 poll 会被拒绝）：按 FIFO 状态反复排水，handled 后立即返回，队列空则每 200 µs 查一次直到超时。Linux 里这是 “SDIO 中断唤醒的内核线程”，这里被 wsm_cmd 的 confirm 等待和 ieee80211_poll 复用。

7.4.3 接收路径：从 CMD53 到 socket

```
xradio_bh_poll() ─── rx_one()
  ├─── CMD53 读 IN_OUT_QUEUE (buf_id_rx+1), 取回 piggyback CONTROL
  ├─── release_buffers()          confirm 释放固件输入缓冲
  └─── wsm_handle_rx()
       ├─── TX confirm 0x0404 → xradio_tx_confirm_cb
```

```
        |                       → ieee80211_tx_status(cookie, status)
        ├─── 命令 confirm       → 命中 wait_cfm, 唤醒 wsm_cmd
        ├─── EVENT              → BSS_LOST → ieee80211_beacon_loss()
        └─── RX indication      → xradio_rx_cb()
             ├─── RX status ≠ 0 (错误/无密钥) → 丢弃
             ├─── 剥 CCMP IV (8B) / MIC (8B)   固件已解密
             ├─── memmove 802.11 头盖回 IV 位置, trim 尾部
             ├─── rx_status = { channel, rcpi, DECRYPTED|IV/MIC_STRIPPED }
             └─── ieee80211_rx_irqsafe()   ← 只入队, 不处理
                                      ▼ 驱动 poll 之后
ieee80211_rx_tasklet()
  ├─── 管理帧: 扫描中 → cfg80211_inform_bss_frame; 已加入 → mlme
  ├─── 数据帧: ieee80211_data_to_8023 → EAPOL? control port : netif_rx
  └─── net_rx_action → ip_rcv / arp_rcv → icmp_rcv / udp_rcv → socket 队列
```

xradio_rx_cb 的剥密钥逻辑是“硬件加密”驱动照 Linux 约定交帧的标准写法：

```
if (enc == WSM_RX_ENC_AES) {
    head = IEEE80211_CCMP_HDR_LEN;      /* 8: IV/PN */
    tail = IEEE80211_CCMP_MIC_LEN;      /* 8: MIC */
} else if (enc == WSM_RX_ENC_TKIP) {
    head = IEEE80211_TKIP_IV_LEN;
    tail = IEEE80211_TKIP_TAIL_LEN;
} else if (enc != WSM_RX_ENC_NONE || ieee80211_has_protected(fc)) {
    return;                             /* 固件没解密的帧不收 */
}
if (head)
    flag = RX_FLAG_DECRYPTED | RX_FLAG_IV_STRIPPED | RX_FLAG_MIC_STRIPPED;
...
net_memcpy(p, frame, frame_len);
if (head) {
    net_memmove(p + head, p, hdrlen);   /* 802.11 头盖过 IV */
    skb_pull(skb, head);
    skb_trim(skb, skb->len - tail);
}
```

7.4.4 发送路径：从 write 到 CMD53

```
write()/sendto → udp/icmp → ip_queue_xmit → eth_header → dev_queue_xmit
 └─── ndo_start_xmit = ieee80211_subif_start_xmit(skb, wlan0)
      ├─── !associated → 丢帧; 剥以太网头 (da, ethertype)
      ├─── ieee80211_build_data_hdr()
      |    push RFC1042 LLC/SNAP + ethertype
      |    ptk_set → push CCMP IV 8B, put MIC 8B, fc |= PROTECTED
      |    push 802.11 头: ToDS, seqno, duration=0(硬件填)
      └─── ieee80211_tx_skb(ac=AC_BE)
           ├─── cookie = ++next_cookie          TX 完成凭据
           ├─── tx_status[cookie % SLOTS] = {cookie, PENDING}
           └─── ops->tx = xradio_tx
                ├─── AC → WSM queue 映射 (VO/VI/BE/BK)
                └─── wsm_tx_frame(): 24B wsm_tx 头 + 原帧
                     └─── xradio_bh_tx() → CMD53 写队列
                                      ..... TX confirm .....
```

```
wsm_handle_rx: 0x0404 → xradio_tx_confirm_cb → ieee80211_tx_status()
           → tx_status[slot].status = 结果 (control_port 帧还要
             cfg80211_control_port_tx_status 唤醒 supplicant)
```

ieee80211_tx_skb 里有一个 tinyOS 风格的决定：ops->tx 返回后立即 kfree_skb(skb)——因为驱动把帧拷进自己的静态 tx_buf 再发，mac80211 不保留 skb。cookie 与 tx_status[] 槽位表顶替了 Linux 的 skb->cb + IEEE80211_TX_STAT；ieee80211_wait_tx_status() 在槽位上自旋（每轮 __net_poll(5)），supplicant 用它等 EAPOL 的 TX confirm。

7.5 无线部分的发射

7.5.1 802.11 帧封装

数据帧从以太网帧原地变换而来，skb_push 逐层加头：

```
static uint8_t *put_hdr(struct ieee80211_local *local, struct sk_buff *skb,
                        uint16_t fc, const uint8_t *a1, const uint8_t *a3)
{
    uint8_t *h = skb_push(skb, IEEE80211_HDR_3ADDR_LEN);
    put_unaligned_le16(fc, h);
    put_unaligned_le16(0, h + 2);            /* duration: 硬件填 */
    net_memcpy(h + 4,  a1, ETH_ALEN);        /* A1 = BSSID (ToDS) */
    net_memcpy(h + 10, local->dev.dev_addr, ETH_ALEN);  /* A2 = SA */
    net_memcpy(h + 16, a3, ETH_ALEN);        /* A3 = DA */
    put_unaligned_le16((uint32_t)(local->seq++ & 0x0fffu) << 4, h + 22);
    return h;
}

int ieee80211_build_data_hdr(struct ieee80211_local *local,
                             struct sk_buff *skb, const uint8_t *da,
                             uint16_t ethertype)
{
    static const uint8_t rfc1042[6] = { 0xaa, 0xaa, 0x03, 0x00, 0x00, 0x00 };
    int protect = local->ptk_set;            /* 装钥后才加密 */
    uint8_t *llc = skb_push(skb, 8);
    net_memcpy(llc, rfc1042, 6);
    put_unaligned_be16(ethertype, llc + 6);
    if (protect) {                           /* 只预留，硬件填 IV/MIC */
        uint8_t *iv  = skb_push(skb, IEEE80211_CCMP_HDR_LEN);
        uint8_t *mic = skb_put(skb, IEEE80211_CCMP_MIC_LEN);
        net_memset(iv, 0, IEEE80211_CCMP_HDR_LEN);
        net_memset(mic, 0, IEEE80211_CCMP_MIC_LEN);
    }
    uint16_t fc = IEEE80211_FTYPE_DATA | IEEE80211_STYPE_DATA |
                  IEEE80211_FCTL_TODS |
                  (protect ? IEEE80211_FCTL_PROTECTED : 0);
    return put_hdr(local, skb, fc, local->bss_conf.bssid, da) ? 0 : -EINVAL;
}
```

要点：

- ToDS 帧三个地址是 A1=BSSID、A2= 源、A3= 目的；
- duration 固定写 0——NAV/重传时间由固件按速率计算；
- 序号由 mac80211 的 `local->seq` 递增，12 位序号 + 4 位分片号；
- `ptk_set` 之后才置 PROTECTED 并**预留** CCMP 头尾：IV/PN 和 MIC 由固件填，主机只负责留出空间——这就是 RX 侧要剥 8+8 字节的镜像操作。

7.5.2 速率与重试策略

发射速率不是逐帧选的：`start` 时通过 MIB 0x1016 装两条重试策略，之后每帧只带一个策略索引：

```
static const uint16_t rates[2][3] = {
    { WSM_RATE_MGMT, 0, 0 },              /* 管理帧: 1 Mbps × 15 */
    { WSM_RATE_DATA, 0x0034u, 0x0007u },   /* 数据: 24M×4, 11M×4, 1M×7 */
};
/* rate entry 位域: [15:14] modem(0=legacy) [13:12] 带宽 [11:8] flags
 *                  [7:4] 速率索引 (1M=0 2M=1 5.5M=2 11M=3 6M=6 .. 54M=13)
 *                  [3:0] 尝试次数 */
e[1] = 14;  e[2] = 14;   /* short / long retry count */
```

管理帧和 EAPOL 走策略 0（最低速率最可靠），数据帧走策略 1 的三级降速链。注册到 mac80211 的 `xradio_rates` 表（12 个 legacy 速率，`hw_value` 是固件速率索引 0..3 / 6..13）就是这套索引的对照表。

7.5.3 EDCA 与队列映射

固件管 QoS，主机只下一组 vendor 默认 EDCA 参数（上线时一次）：

队列 (wire 顺序)	cwMin	cwMax	AIFS	TXOP (μs)
BK	15	1023	7	0
BE	3	7	2	0
VI	7	15	2	3008
VO	3	7	2	1504

每帧的接入类别映射成 WSM 队列号，塞进 wsm_tx 头第 8 字节（`raw_link_id << 2 | queue`，STA 模式 link id 恒 0）：

```
static const uint8_t ac_to_queue[4] = {
    [IEEE80211_AC_VO] = WSM_QUEUE_VOICE,
    [IEEE80211_AC_VI] = WSM_QUEUE_VIDEO,
    [IEEE80211_AC_BE] = WSM_QUEUE_BEST_EFFORT,
    [IEEE80211_AC_BK] = WSM_QUEUE_BACKGROUND,
};
```

7.5.4 WSM TX 请求与 confirm

`wsm_tx_frame()` 把 802.11 帧包进 24 字节 `wsm_tx` 头：

偏移	字段	值
0	len	由 xradio_bh_tx 填（13-bit，[15:13] 是序号）
2	id	WSM_TX_REQ
4	packetID	mac80211 cookie，confirm 原样带回
8	queue	raw_link_id << 2 | queue
10	DataOffset	24（头之后才是帧）
12	TxRateEntry	策略表条目
14	TxPolicyIndex	0= 管理 1= 数据
16	expireTime	0（固件默认）

confirm 回来时 packet_id 就是 cookie，xradio_tx_confirm_cb() 校验后调 ieee80211_tx_status()，槽位表把状态交还等待者。整个环路的驱动方仍是轮询：发送侧若固件缓冲耗尽，xradio_bh_tx 边 poll 边等 confirm 腾缓冲；EAPOL 这类同步发射用 ieee80211_wait_tx_status() 主动转轮子。

7.5.5 EAPOL 发射特例

四次握手的帧与普通数据帧共用 build_data_hdr，但有三点不同（ieee80211_tx_control_port()）：

- 前置条件只要求 local->joined 而非 associated——AP 的 EAPOL M1 常与 assoc response 背靠背到达，等自己走完关联状态机就晚了；
- 队列用 AC_VO 且置 IEEE80211_TX_CTL_USE_MINRATE，驱动侧翻译成 WSM_POLICY_MGMT + WSM_RATE_MGMT（1 Mbps），握手帧宁可慢不可丢；
- control_port = 1：TX confirm 里会额外调 cfg80211_control_port_tx_status()，M4 发送成功才触发装钥（PTK/GTK 下到固件），这是 supplicant 状态机的节拍器。

7.6 UART 串口驱动与 /dev/console

UART3 在驱动模型里走的是另一条路线：它是内核第一个设备，也是唯一一个 “boot 代码先摸硬件、probe 只做登记” 的设备。完整的中断、环形缓冲与 /dev/console 流程见第 4 章，这里只看驱动模型视角：

- t113_uart_init() 在 kernel_main() 最先执行，串口作为 early console 立即可用；此时驱动核心还没起来，谈不上 probe；
- uart3_device 注册进平台总线后，probe **不做任何硬件操作**，只校验资源表（mem/IRQ）与早控制台实际使用的值一致，然后向 devfs 登记两个字符设备节点；
- TX 纯轮询（等 USR.TFNF 写 THR，10 ms 停转看门狗），RX 走 IRQ 37 → 256 字节环形缓冲 → 等待队列唤醒。

```
/* uart probe: 硬件已由 boot 代码初始化, probe 只登记 devfs 节点 */
t113_devfs_register_chrdev("console", 1u, 0u, &console_fops);
t113_devfs_register_chrdev("ttyS0",   3u, 0u, &console_fops);
```

用户态 stdio 目前不走 VFS：sys_read 对 fd 0、sys_write 对 fd 1/2 直接特判到 UART 读写函数；/dev/console（major 1）与 /dev/ttyS0 （major 3，沿 Linux 惯例）作为设备文件可被 open，主要留给将来的用户程序。对照：xv6 的 uart.c 同样是轮询发送 + 中断接收，但 init 显式 open("/dev/console") 再 dup 成 0/1/2；miniOS 把这一步留作演进方向。

7.7 与 Linux 的对照

话题	miniOS	Linux
kobject/kset	有树与路径，sysfs 按路径实时投影（只读）	kernfs 节点，属性可读写
总线	platform + i2c + sdio 三条	同名总线 + 更多（usb/spi…）
匹配	compatible/id_table 字符串比较	device tree / ACPI / id 表
probe 串行化	一把核心锁，跨 probe 持有	per-device 锁，可并发 probe
依赖处理	注册顺序约束	deferred probe / probe retry
卡枚举	`mmc_rescan_all()` 显式一次	`mmc_rescan` workqueue
SDIO 中断	无，全轮询	SDIO IRQ + 底半部线程
TX 状态	cookie + 静态槽位表	`skb->cb` + TX status 回调/队列
帧加密	硬件 CCMP/TKIP，主机预留头尾	同（`mac80211` 硬件加密路径）
速率	静态重试策略 MIB	minstrel 自适应

总结成一句话：miniOS 把 Linux 驱动模型的**形状**（总线/设备/驱动、分层接口、上下文规则）完整保留，把**机制**（并发 probe、deferred probe、中断、自适应速率）简化成单核轮询下的顺序代码——每个简化点都是有意为之的 bring-up 取舍，且在源码注释里标明了对应的 Linux 原型。

第八章 SMHC 主机控制器：SD 卡与 SDIO WiFi

本文说明 T113-S3 的 SMHC（SD/MMC Host Controller）怎样同时服务两类从设备：SMHC0 接 TF 卡（SD 存储协议，块读写），SMHC1 接 XR829（SDIO 协议，CMD52/CMD53 读写模组寄存器和消息队列）。内容依次是芯片内部结构、驱动分层、信号时序，然后和 xv6-aarch64 的块设备、Linux sunxi-mmc 以及树莓派 3 的 WiFi 硬件做对照。

相关源码：

文件	作用
`src/drivers/mmc/sunxi-sd.c`	SMHC0 的 SD 卡块驱动（miniBoot 和 miniOS 共用）
`src/drivers/mmc/host/sunxi-smhc.c`	SMHC 主机驱动（platform_driver），当前只绑定 SMHC1
`src/drivers/mmc/core.c`	MMC/SDIO 核心：SDIO 枚举、CMD52/CMD53、sdio 总线
`src/include/t113/mmc.h`	`mmc_host`、`sdio_card`、`sdio_func`、`sdio_driver`
`src/boards/100ask_t113/board.c`	`sdc1_device`：SMHC1 的资源、引脚、WL_REG_ON、32k fanout
`src/drivers/net/wireless/xradio/sdio.c`	XR829 的 SDIO function 驱动

8.1 SMHC 芯片结构

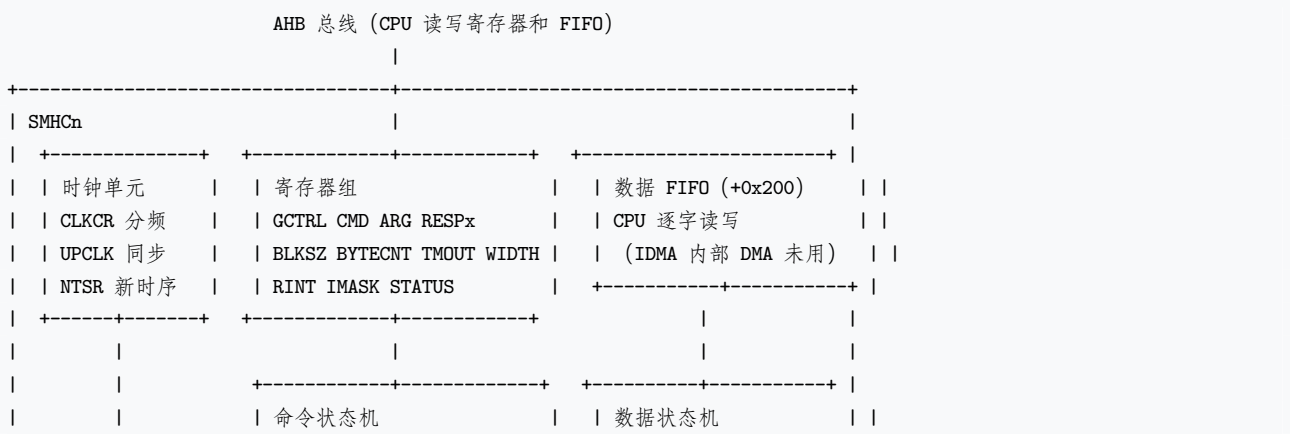
8.1.1 三个实例

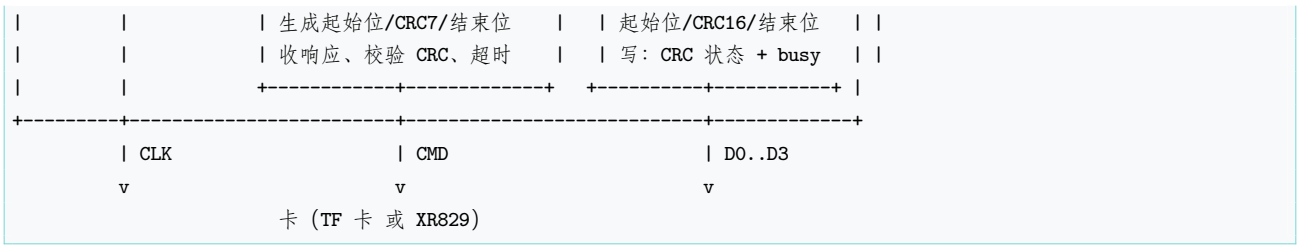
T113 有三个 SMHC，寄存器窗口相隔 0x1000（`T113_SMHC_BASE(n)`）：

实例	基址	板上用途	引脚	中断
SMHC0	0x04020000	TF 卡（启动盘、根文件系统、`/boot`）	PF0–PF5 func 2	未使用
SMHC1	0x04021000	XR829 WiFi（SDIO）	PG0–PG5 func 2	73（GIC SPI 41），已声明未使用
SMHC2	0x04022000	板上未用（通常接 eMMC）	—	—

SMHC0 的引脚按 sun20i-d1 pinctrl 排列：PF0 = D1、PF1 = D0、PF2 = CLK、PF3 = CMD、PF4 = D3、PF5 = D2。`sunxi-sd.c` 把 `PF_PULL0` 写成 0x545，也就是除 CLK（PF2）以外全部上拉。SMHC1 的 PG0 = CLK、PG1 = CMD、PG2–PG5 = D0–D3，CLK 不上拉，其余上拉，定义在 `board.c` 的 `sdc1_pins[]`。

8.1.2 内部框图





软件只负责“填参数、写 CMD、等 RINT”。起始位、方向位、CRC7/CRC16、结束位、响应接收和 CRC 校验都由状态机完成。

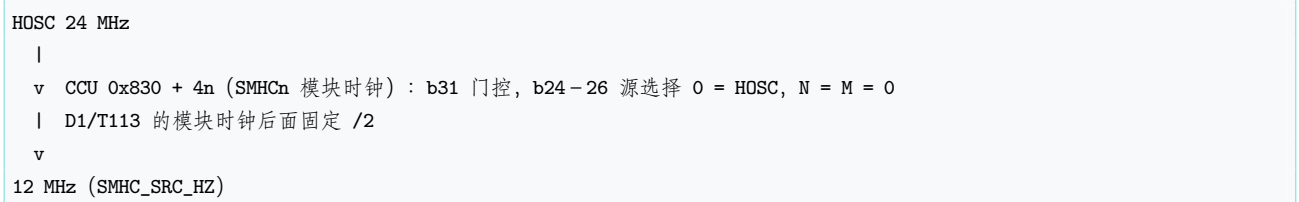
8.1.3 寄存器

两个驱动用到的寄存器完全相同（偏移来自 U-Boot `sunxi_mmc.h`，D1/T113 属于 NCAT2 布局，FIFO 在 +0x200）：

偏移	名称	用到的位	含义
0x000	GCTRL	b0–2 复位（soft/FIFO/DMA），b31 AHB	复位；b31 = 1 表示 CPU 直接读写 FIFO
0x004	CLKCR	b0–7 分频，b16 使能，b17 省电，b31 MASK_DO	卡时钟
0x008	TMOUT	b0–7 响应超时，b8–31 数据超时	写 0xffffff40：响应 64 个时钟，数据 0xffffff 个时钟
0x00c	WIDTH	0 = 1-bit，1 = 4-bit	总线宽度，两路都写 0
0x010	BLKSZ		块大小
0x014	BYTECNT		本次数据总字节数
0x018	CMD	b0–5 命令号，b6 要响应，b7 长响应，b8 校验 CRC，b9 有数据，b10 写，b13 等前一个完成，b15 发初始化时钟，b21 只更新时钟，b31 START	写入即启动
0x01c	ARG		命令参数
0x020–0x02c	RESP0–3		响应；长响应时 RESP3 是最高 32 位
0x030	IMASK		中断屏蔽，写 0（不用中断）
0x038	RINT	b2 CMD_DONE，b3 DATA_OVER，错误位掩码 0xbfc2	原始中断状态，写 1 清除
0x03c	STATUS	b2 FIFO 空，b3 FIFO 满，b9 卡忙	
0x05c	NTSR	b31 新时序模式	D1/T113 必须打开
0x078	HWRST		卡硬件复位脚，先 0 后 1
0x200	FIFO		数据端口

RINT 的错误位 0xbfc2 包括：响应错（b1）、响应 CRC 错（b6）、数据 CRC 错（b7）、响应超时（b8）、数据超时（b9）、数据饥饿超时（b10）、FIFO 溢出（b11）、命令忙非法写（b12）、起始位错（b13）、结束位错（b15）。

8.1.4 时钟树



```
  |
  v  CLKCR[7:0] = div: div = 0 直通，否则 card_clk = 12 MHz / (2 × div)
  |
卡时钟：识别阶段 div = 15 → 400 kHz；数据阶段 div = 0 → 12 MHz

CCU 0x84c（SMHC 总线门控/复位）：b(n) 门控，b(16 + n) 复位
```

两路驱动都只用 HOSC，不碰 PLL_PERIPH0，所以最高只能到 12 MHz。Linux sunxi-mmc 会把源切到 PLL_PERIPH0（600 MHz）再分频，才能跑到 SD 高速模式的 50 MHz。SMHC1 的 32k fanout 依赖 PLL_PERIPH0 已开，这一点和 SMHC 本身的时钟无关，见第 5 章的上电时序。

8.2 驱动架构

8.2.1 两条软件栈

```
      SD 卡路径（SMHC0）                          WiFi 路径（SMHC1）

minifs（/）        fat32（/boot）              xradio：fwio.c / bh.c / wsm.c
    |                  |                          | sdio_memcpy_toio/fromio
    |                bio.c（bcache）              | sdio_readb/writeb、sdio_set_block_size
    |                  |                          v
rootdev.c              |                  drivers/mmc/core.c
    |                  |                   +- sdio 总线：match / probe
    +--------+---------+                   +- 枚举：CMD0/5/3/7、CCCR/FBR/CIS
             | t113_sd_read/write          +- sdio_io_rw_direct（CMD52）
             |  （sd_lock 互斥）              sdio_io_rw_extended（CMD53）
             v                                   | host->ops->request / set_ios / power
drivers/mmc/sunxi-sd.c                           v
 直接操作 SMHC0 寄存器，                  drivers/mmc/host/sunxi-smhc.c
 没有 mmc_host 抽象                       platform_driver "sunxi-mmc"
             |                                   | 资源来自 board.c 的 sdc1_device
             v                                   v
SMHC0 @0x04020000 -- PF0..PF5 -- TF 卡   SMHC1 @0x04021000 -- PG0..PG5 -- XR829
                                            PG11 32k fanout、PG12 WL_REG_ON
```

两条栈在硬件上是同一种控制器，软件上却完全分开：

- **SD 卡路径**是一个独立的块驱动。它要在 miniBoot（SPL，没有驱动模型、没有定时器）里也能用，所以写成不依赖任何框架的形式：寄存器地址写死为 SMHC0，延时用空循环 `spin_us()`，对外只有 `t113_sd_init/read/write` 三个函数。在 miniOS 里，`MINIOS_KERNEL` 宏打开 `sd_lock` 互斥锁，因为 minifs 的 rootdev 和 FAT32 的 bio 会从不同进程访问同一张卡。
- **WiFi 路径**按 Linux 的三层组织：主机驱动（`sunxi-smhc.c`）只实现 `mmc_host_ops` 的四个回调；核心（`core.c`）负责协议和枚举；function 驱动（`xradio/sdio.c`）按 vendor/device 匹配后接管。

8.2.2 主机接口 mmc_host_ops

回调	sunxi-smhc.c 的实现	作用
request(host, cmd)	smhc_request()	发一条命令，有数据时 CPU 搬 FIFO，等 CMD_DONE（和 DATA_OVER），取响应；出错就复位控制器并恢复时钟
set_ios(host, hz, width)	smhc_set_ios()	改分频和总线宽度
power(host, on)	smhc_power()	32k fanout + WL_REG_ON 上电时序
dump(host)	smhc_dump()	失败时打印 SMHC、CCU、PG 口寄存器

struct mmc_command 只带命令号、参数、响应类型（MMC_RSP_R1/ R1B/ R4/ R5/ R6）和可选的 struct mmc_data（缓冲区、块大小、块数、方向）。这与 Linux mmc_request 的形状相同，只是没有 stop 命令、DMA 映射和完成回调。

8.2.3 注册与扫描

```
t113_driver_core_init()
+-- sdio_bus_init()                              注册 "sdio" 总线
+-- platform_device_register(&sdc1_device)       mem = SMHC1, irq = 73, platform_data = sdc1_pdata
+-- platform_driver_register(&t113_smhc_driver)
|   +-- t113_smhc_probe(sdc1)
|       +-- idx = (mem - SMHC0_BASE) / 0x1000 = 1
|       +-- CCU 0x84c: 复位位拉低 10 us 再和门控一起置 1
|       +-- CCU 0x834 = 1 << 31                   模块时钟 = HOSC
|       +-- t113_pinmux_apply(sdc1_pins)
|       +-- smhc_reset(); HWRST 0 → 1
|       +-- NTSR |= NEW_MODE; TMOUT = 0xffffff40; IMASK = 0
|       +-- smhc_set_ios(400 kHz, 1-bit)
|       +-- mmc_add_host()                        只登记到 hosts[1]
+-- sdio_driver_register(&xradio_sdio_driver)
+-- mmc_rescan_all()                             对每个还没有卡的 host
    +-- sdio_attach_card() → sdio_add_funcs() → device_register(func)
        +-- sdio_bus_match() → xradio_sdio_probe()
```

SMHC0 不在 hosts[] 里，mmc_rescan_all() 不会扫描它。TF 卡在更早的 kernel_boot() → t113_minifs_mount_rootdev() → t113_rootdev_init() → t113_sd_init() 里就初始化好了，因为根文件系统要在驱动模型启动之前挂载。

8.2.4 与 Linux sunxi-mmc 的对照

方面	Linux	miniOS
主机驱动	一个 `sunxi-mmc.c` 服务全部 SMHC	SMHC1 用 `sunxi-smhc.c`；SMHC0 用独立的 `sunxi-sd.c`
SD 卡	`mmc_rescan` → `mmc_attach_sd()` → `mmc_blk` 块设备→块层	`t113_sd_read/write`，由 rootdev 和 bio 直接调用
SDIO	`mmc_attach_sdio()` → `sdio_func` 设备→驱动	同样的结构，在 `core.c` 里
扫描时机	工作队列里的 `mmc_rescan`，支持热插拔	启动时一次 `mmc_rescan_all()`，不支持热插拔
数据搬运	IDMA 描述符链	CPU 逐字读写 FIFO
完成通知	中断 + `mmc_request_done()`	轮询 RINT
时钟	PLL_PERIPH0 分频，SD 高速 50 MHz	HOSC，最高 12 MHz
总线宽度	协商到 4-bit（ACMD6 / CCCR 0x07）	一直 1-bit
SDIO 中断	`sdio_irq` 线程或 SMHC 的 SDIO 中断	不用，`bh.c` 轮询

8.3 时序

8.3.1 命令与响应

一条命令在 CMD 线上是 48 个时钟。卡在命令结束后 2–64 个时钟（N_CR）内开始回响应。TMOUT 低 8 位写 0x40，正好是规范上限 64 个时钟。

```
CLK     _|-|_|-|_|-|_ ... _|-|_|-|_|-|_ ... _|-|_|-|_|-|_ ... _|-|_
CMD     --|S|D| 命令号 + 参数 + CRC7 |E|----------|S|D| 响应 + CRC7 |E|---
           <--------- 48 个时钟 ------->          <----- 48 个时钟 ---->
                                      |<- N_CR ->|
                                        2..64 时钟
S = 起始位 0, D = 方向位（主机发 1，卡回 0），E = 结束位 1
```

时钟	一个 48 位帧	一次 CMD52（命令 + N_CR + 响应）
400 kHz	120 us	约 0.25–0.4 ms
12 MHz	4 us	约 8–13 us

软件每发一条命令的顺序是固定的：

```
smhc_request(host, cmd)                    sunxi-sd.c 的 send_cmd() 相同
+-- [有数据] GCTRL |= AHB | FIFO_RST，等 FIFO_RST 自清
+-- [有数据] BLKSZ = blksz; BYTECNT = blksz × blocks
+-- RINT = 0xffffffff                      清掉旧状态
+-- ARG = arg
+-- CMD = START | opcode | RESP_EXPIRE | CHK_CRC [| LONG_RESP] [| DATA | WAIT_PRE [| WRITE]]
|         CMD0 额外加 INIT_SEQ：先发 80 个初始化时钟（规范要求至少 74 个）
+-- [有数据] smhc_pio()：按 STATUS 的 FIFO 空/满逐字搬运
+-- 轮询 RINT：错误位 → -EIO 或 -ETIMEDOUT；CMD_DONE（+ DATA_OVER）→ 成功
+-- 读 RESP0（长响应读 RESP0..3）
+-- [R1b] 等 STATUS.CARD_BUSY 清零（D0 被卡拉低表示忙）
+-- 出错：smhc_reset() + 重新 set_ios()，因为错误会卡住状态机
```

8.3.2 改时钟

D1/T113 的 SMHC 改分频时必须先屏蔽 D0，再发一个“只更新时钟”的伪命令，让新的 CLKCR 同步到卡时钟域：

```
smhc_set_ios(host, hz, width)
+-- CLKCR = 原值 & ~ENABLE | MASK_DO       1. 停卡时钟，屏蔽 DO
+-- CMD = START | UPCLK_ONLY | WAIT_PRE    2. 同步，等 START 自清
+-- CLKCR = 新 div | ENABLE | MASK_DO      3. 新分频，开时钟
+-- CMD = START | UPCLK_ONLY | WAIT_PRE    4. 再同步
+-- CLKCR = 新 div | ENABLE                5. 取消 DO 屏蔽
+-- WIDTH = 0 (1-bit) 或 1 (4-bit)
```

MASK_DO 和 NTSR 新时序模式都是 D1/T113 才有的要求，Linux `sunxi-mmc.c` 里同样的写法。

8.3.3 SD 卡初始化（SMHC0）

```
t113_sd_init()                                      时钟       卡状态
+-- CCU 0x84c bit0 / bit16, CCU 0x830 = HOSC
+-- PF0..PF5 func 2, 上拉 0x545
+-- 复位; HWRST; NTSR; TMOUT; WIDTH = 1-bit
+-- set_clock(400 kHz)
+-- CMD0  GO_IDLE_STATE (INIT_SEQ)               400 kHz    → idle
+-- CMD8  SEND_IF_COND 0x1AA → R7, 回显 0x1AA 就是 v2 卡
+-- 循环 (最多 1000 次, 间隔约 1 ms) :
|   +-- CMD55 APP_CMD
|   +-- ACMD41 SD_SEND_OP_COND (HCS | 0x00ff8000) → R3, b31 = 1 时就绪    → ready
|       b30 (CCS) = 1 → SDHC/SDXC, 按扇区号寻址
+-- CMD2  ALL_SEND_CID → R2 (136 位)                          → ident
+-- CMD3  SEND_RELATIVE_ADDR → R6, 取 RCA                    → stby
+-- CMD7  SELECT_CARD(RCA) → R1b, 等 CARD_BUSY               → tran
+-- [SDSC] CMD16 SET_BLOCKLEN 512
+-- set_clock(12 MHz)                            12 MHz
```

没有 ACMD6（切 4-bit）和 CMD6（切高速），所以 TF 卡一直是 1-bit、12 MHz。

8.3.4 单块读 CMD17

```
CMD   |CMD17 + 地址|   |R1|
DO    ------------------------------|S| 512 字节数据 (4096 位) | CRC16 |E|----
                 |<---- N_AC ---->|
CPU                                  逐字读 FIFO (128 个字)，直到 RINT 有 CMD_DONE | DATA_OVER
```

1-bit、12 MHz 时，数据段约 4114 个时钟，即约 343 us。理论吞吐约 1.49 MB/s，实际还要加上命令、N_AC 和逐字轮询的开销。4-bit 能把数据段缩到四分之一。

8.3.5 单块写 CMD24

```
CMD   |CMD24 + 地址|   |R1|
DO    ----------------------|S| 512 字节 | CRC16 |E|--|S|010|E|________ 忙 ________|---
                                                 CRC 状态     卡编程期间拉低 DO
CPU   逐字写 FIFO (STATUS 不满就写)                  等 CMD_DONE | DATA_OVER, 再等 CARD_BUSY 清零
```

CRC 状态 010 表示卡接受了数据；随后卡在编程闪存期间一直拉低 D0，SMHC 把它反映为 STATUS b9。`write_block()` 在这里等待，所以 `t113_sd_write()` 返回时数据已经写进卡里，FAT32 的写顺序约束靠的就是

这一点。

8.3.6 SDIO 初始化与 CMD53（SMHC1）

SDIO 的上电时序（32k → WL_REG_ON 低 10 ms →高→等 100 ms）、枚举状态机和 R4/R5/R6 位图，在第 5 章已经画过。这里只补 CMD52/CMD53 的参数格式和数据时序：

```
CMD52 参数（IO_RW_DIRECT）
 31   30..28   27    26   25 ............ 9   8   7 ....... 0
+---+--------+-----+----+-----------------+---+-----------+
|R/W| 功能号  | RAW | 0  | 寄存器地址 17 位 | 0 | 写入的字节 |
+---+--------+-----+----+-----------------+---+-----------+

CMD53 参数（IO_RW_EXTENDED）
 31   30..28   27      26     25 ............ 9   8 ........ 0
+---+--------+-------+------+-----------------+------------+
|R/W| 功能号  | 块模式 | 递增 | 寄存器地址 17 位 | 字节/块计数 |
+---+--------+-------+------+-----------------+------------+
字节模式：计数 1..512（512 写成 0）
块模式：计数 = 块数，块大小取 FBR1 0x10/0x11（xradio 设为 512）
递增 = 0：地址固定，用于 HIF 的数据端口（FIFO）
```

`sdio_io_rw_extended()` 的规则：长度 ⩽ 512 用字节模式，否则用块模式且长度必须是 `blksz` 的整数倍；缓冲区和长度都要 4 字节对齐，因为 FIFO 按 32 位访问。数据段的波形和 CMD17/CMD24 一样；SDIO 写完也有 CRC 状态和 busy。

XR829 的时钟分两步：枚举和 CMD53 自检都在 400 kHz 下做，自检通过后 `mmc_set_clock(12 MHz)` 再自检一次，失败就退回 400 kHz。

8.3.7 两路对照

方面	SMHC0：TF 卡	SMHC1：XR829
协议	SD 存储（物理层 + 存储命令集）	SDIO（同一物理层 + IO 命令集）
驱动	`sunxi-sd.c`，无框架	`sunxi-smhc.c` + `core.c` + `xradio/sdio.c`
初始化命令	CMD0、8、55/41、2、3、7（16）	CMD0、5（重复到就绪）、3、7，然后 CMD52 读 CCCR/FBR/CIS
数据命令	CMD17 读 / CMD24 写，每次 512 B	CMD53，字节模式或 512 B 块模式
寻址	扇区号（SDHC）或字节地址（SDSC）	功能号 + 17 位寄存器地址
上电	卡座直接供电，无时序	32k 时钟 + WL_REG_ON 时序
时钟	400 kHz → 12 MHz	400 kHz → 12 MHz（自检失败退回）
位宽	1-bit	1-bit
并发	`sd_lock`：rootdev、bio 两个用户	只有 xradio 一个用户，bh 单线程
出错恢复	复位 + UPCLK，单块重试 3 次	复位 + 重新 set_ios，返回错误给调用者
中断 / DMA	不用	不用（IRQ 73 只声明）

8.4 和 xv6-aarch64 的区别

xv6-aarch64 没有 SD 或 SDIO 驱动。它跑在 QEMU virt 机器上，磁盘是 virtio-blk（`kernel/virtio_disk.c`，MMIO 基址 `VIRTIO0` = KERNBASE + 0x0a000000，中断 48），另有一个 `ramdisk.c` 备用。所以能比的是“块设备驱动的写法”，以及 SDIO 那一层在 xv6 里根本不存在。

8.4.1 一次读盘的调用链

```
xv6-aarch64                                  T113 miniOS (FAT32 路径)
bread(dev, blockno)                          bread(blockno)
+-- bget()  睡眠锁 b->lock                    +-- bget() (24 个 512 B 缓冲, LRU)
+-- virtio_disk_rw(b, 0)                     +-- t113_sd_read(lba, 1, b->data)
    +-- acquire(&disk.vdisk_lock)                +-- mutex_lock(&sd_lock)
    +-- alloc3_desc(): 没有空描述符就 sleep      +-- read_block(lba)
    +-- 描述符 0: virtio_blk_req{type, sector}   |   +-- 写 BLKSZ/BYTECNT/ARG/CMD
    +-- 描述符 1: b->data (1024 B)               |   +-- 轮询 STATUS, CPU 读 128 个字
    +-- 描述符 2: 1 字节状态                     |   +-- 轮询 RINT 到 DATA_OVER
    +-- avail ring 放入描述符 0                  |   失败重试 3 次
    +-- *R(QUEUE_NOTIFY) = 0                     +-- mutex_unlock(&sd_lock)
    +-- while (b->disk == 1) sleep(b)  让出 CPU  调用者一直在轮询里等, CPU 不让出
virtio_disk_intr() (IRQ 48)
+-- 遍历 used ring
+-- b->disk = 0; wakeup(b)
+-- 释放描述符
```

8.4.2 差异表

方面	xv6-aarch64 virtio-blk	T113 SMHC0 SD 卡	T113 SMHC1 SDIO
设备	QEMU 模拟的半虚拟化磁盘	真实 TF 卡	真实 SDIO 模组
发现设备	读 MMIO 魔数 0x74726976、设备号 2	写死 SMHC0	platform 设备 + CMD5 + CIS 里的 MANFID
协商	virtio 特性位（关掉 RO、SCSI、MQ 等）	CMD8 / ACMD41 的电压和容量	OCR 电压窗口、IOEx/IORx、块大小
请求格式	3 个描述符串成链（头、数据、状态）	一条命令 + 一个数据块	CMD52 一字节 / CMD53 多字节
数据搬运	设备直接读写内存（DMA 语义）	CPU 逐字搬 FIFO	CPU 逐字搬 FIFO
完成通知	中断 + `wakeup()`	轮询 RINT	轮询 RINT
等待方式	`sleep()`，调度别的进程	忙等	忙等
锁	spinlock `vdisk_lock` + 每块睡眠锁	mutex `sd_lock`	无（单用户）
块大小	1024 B（BSIZE），一次请求两个扇区	512 B 扇区；minifs 1 KiB 块拆成两次	字节或 512 B 块
并发请求	最多 NUM = 8 个描述符在途	一次一个	一次一个
总线/驱动模型	无：`main()` 直接调 `virtio_disk_init()`	无	sdio 总线 + match/probe
出错	`panic`	复位控制器、重试	复位控制器、报错

要把 SMHC 移植进 xv6-aarch64，替换的是 `virtio_disk_rw()` 这一个函数：按 `b->blockno × 2` 读两个扇区，`virtio_disk_init()` 换成 `t113_sd_init()`。想保留 xv6 的 sleep/wakeup 语义，就要打开 SMHC 的中断（IMASK 打开 CMD_DONE、DATA_OVER 和错误位，在 GIC 上使能 SMHC0 的中断），在中断处理里 `wakeup(b)`，或者改用 IDMA 让控制器自己搬数据。

8.5 XR829 与树莓派 3 WiFi 的硬件区别

miniOS 的树莓派板级代码（`src/boards/raspberrypi/drivers/emmc/`）只有 SD 卡驱动，没有 WiFi 驱动。下表里树莓派 3 一侧来自公开的原理图说明和 Linux 设备树（`bcm2837-rpi-3-b.dts`），没有在本仓库代码里验证。

8.5.1 连接方式

```
树莓派 3B (BCM2837)                          T113-S3 (100ASK T113-Pro)
+-------------------------------+            +-------------------------------+
| SDHOST -- GPIO48..53 ALT0 -→ TF 卡           | SMHC0 -- PF0..PF5 -→ TF 卡     |
| EMMC (Arasan SDHCI)           |            | SMHC1 -- PG0..PG5 -→ XR829     |
|   +-- GPIO34..39 ALT3 -→ CYW43438 SDIO       | CLK_FANOUT1 PG11 -→ 32k LPO    |
| 固件 GPIO 扩展 expgpio 1 -→ WL_ON            | PG12 -→ WL_REG_ON              |
| 固件 GPIO 扩展 expgpio 0 -→ BT_ON            | PG10 ←- WL_WAKE_AP (未用)      |
| UART0 (PL011) GPIO32/33 -→ CYW43438 蓝牙     | 蓝牙 UART: miniOS 未接         |
+-------------------------------+            +-------------------------------+
```

树莓派 3 有两个 SD 控制器，并且可以通过引脚复用互换：Linux 默认让 SDHOST 接 TF 卡、Arasan EMMC 接 WiFi。miniOS 的 `emmc_init()` 反过来，把 GPIO34–39 设为输入（断开 WiFi），把 GPIO48–52 设为 ALT3，让 Arasan EMMC 去读 TF 卡。所以 miniOS 在树莓派上要加 WiFi，得先把 TF 卡改到 SDHOST，或者 WiFi 改用 SDHOST。T113 没有这种取舍：两个 SMHC 实例各自有固定的引脚。

8.5.2 对照表

方面	树莓派 3B：CYW43438（BCM43438）	100ASK T113-Pro：XR829
厂商	Broadcom / Cypress（现 Infineon）	全志 XRadio
SDIO vendor:device	0x02d0:0xa9a6	0x0a9e:0x2282（实测）
制式	2.4 GHz 802.11b/g/n，1T1R	2.4 GHz 802.11b/g/n，1T1R
蓝牙	有，经 UART0	有，miniOS 没用
主机控制器	Arasan SDHCI（SD Host Controller 标准寄存器）	全志 SMHC（私有寄存器组）
电源控制	WL_ON 在 VideoCore 固件管理的 GPIO 扩展上，要经 mailbox	WL_REG_ON 是 SoC 的普通 GPIO PG12
32k 睡眠时钟	Linux 设备树不需要主机驱动开	主机必须开 CLK_FANOUT1 送到模组，否则 CMD5 无响应
SDIO function	F1 = 背板（寄存器窗口），F2 = WLAN 数据	驱动只用 F1：HIF 寄存器 + 消息队列
片内处理器	ARM Cortex-M3 + SOCRAM	片内 CPU，代码 RAM 在 AHB 0x08000000
固件	`brcmfmac43430-sdio.bin` + NVRAM 文本 + CLM blob	`boot_xr829.bin` + `fw_xr829.bin` + `sdd_xr829.bin`
下载方式	经 F1 背板窗口（SBADDR 寄存器选 32 KiB 窗口）写 SOCRAM，NVRAM 放在 RAM 末尾，再放开 ARM 复位	先写 bootloader 到 AHB 0x08000000，再经下载 FIFO 按 1 KiB 块灌固件，握手
主机协议	SDPCM 帧（控制 / 事件 / 数据三个通道）+ BCDC ioctl	HIF 队列 + WSM 消息（请求 / 确认 / 指示）
MAC 分工	FullMAC：扫描、认证、关联在固件里，主机经 ioctl 下命令	主机侧走 mac80211（SoftMAC 分工），固件做下层 MAC
Linux 驱动	`brcmfmac`	`xradio_wlan`（out-of-tree）
主机唤醒	SDIO 带内中断（DAT1）或带外 GPIO	WL_WAKE_AP（PG10）或 SDIO 中断；miniOS 都不用，轮询

对 miniOS 来说，最大的区别是 MAC 分工。CYW43438 是 FullMAC，连接一个 AP 只要几个 ioctl（设 SSID、设 WPA 参数）。XR829 走 mac80211 那一套，所以 miniOS 要自己实现扫描结果处理、认证/关联帧、EAPOL 四次握手和装钥，这就是 `src/net/mac80211`、`src/net/wireless` 和 supplicant 那一层存在的原因（见第 6 章）。

第九章 VFS：super_block、inode 与 dentry

miniOS 的 T113 VFS 使用 super_block、dentry、inode、file 和 vfs_mount 组织挂载与打开文件。当前版本已经删除旧的 vfs_backend_ops 和通用适配器 backend_fops：每个文件系统直接提供自己的 inode_operations 与 file_operations，字符设备在打开时把 file->f_op 切换成驱动的操作表。

9.1 核心对象

```
struct super_block {
    uint32_t s_magic;
    const struct super_operations *s_op;
    struct dentry *s_root;
    void *s_fs_info;
};

struct inode {
    uint32_t i_ino;
    uint32_t i_size;
    uint32_t i_type;
    uint32_t i_rdev;
    struct super_block *i_sb;
    const struct inode_operations *i_op;
    const struct file_operations *i_fop;
    void *i_private;
};

struct file {
    uint32_t f_flags;
    uint32_t f_pos;
    struct dentry *f_dentry;
    struct inode *f_inode;
    const struct file_operations *f_op;
    void *private_data;
    struct mutex f_pos_lock;
};
```

- super_block 表示一次文件系统挂载。
- dentry 表示路径中的名字，并指向 inode。
- inode 表示文件、目录或设备节点本身。
- file 表示一次 open()；dup() 和 fork() 可以共享它。
- i_rdev 仅对字符设备有意义，保存 major/minor 合成的设备号。
- private_data 给设备驱动保存每次打开的私有状态。

9.2 三张操作表

```
struct super_operations {
    void (*put_super)(struct super_block *sb);
    void (*evict_inode)(struct inode *inode);
};

struct inode_operations {
    int (*lookup)(struct inode *dir, struct dentry *child);
    int (*create)(struct inode *dir, struct dentry *child);
    int (*truncate)(struct inode *inode);
};

struct file_operations {
    int (*open)(struct inode *inode, struct file *file);
    int (*read)(struct file *file, void *buf, uint32_t count);
    int (*write)(struct file *file, const void *buf, uint32_t count);
    int (*readdir)(struct inode *inode, const char *path,
                   uint32_t index, struct vfs_dirent *entry);
    void (*release)(struct inode *inode, struct file *file);
};
```

职责划分：

```
inode->i_op   路径和元数据：lookup/create/truncate
inode->i_fop  此 inode 默认的打开后操作
file->f_op    本次 open 实际使用的操作，可在 open 中替换
```

9.3 普通文件

minifs、FAT32 和 bootfs 都直接提供自己的操作表：

```
minifs inode
+-- i_op  = minifs_iops
+-- i_fop = minifs_file_fops 或 minifs_dir_fops

FAT32 inode
+-- i_op  = fat32_iops
+-- i_fop = fat32_file_fops 或 fat32_dir_fops
```

读取 /bin/ls：

```
sys_read(fd)
  → current->files[fd]
  → file->f_op->read(file, ...)
  → minifs_vfs_read()
  → 根据 file->f_inode->i_ino 读取 SD 卡 minifs 数据块
```

读取 /boot/kernel_minios.img：

```
sys_read(fd)
  → file->f_op->read
  → fat32_read()
  → file->f_inode->i_private (fat_inode)
```

```
  → FAT 簇链与 SD 块设备
```

不再经过 backend_read() 或 super_block->s_backend。

9.4 字符设备

devfs 只维护“名字到设备号”的关系：

```
"console" → inode(type=CHAR, i_rdev=1:0)
"ttyS0"   → inode(type=CHAR, i_rdev=3:0)
"key0"    → inode(type=CHAR, i_rdev=13:0)
```

字符设备 inode 的默认操作是 def_chr_fops：

```
inode->i_fop = def_chr_fops
                  |
                  +-- open = chrdev_open
```

chrdev_open() 根据 inode->i_rdev 查询全局字符设备表，然后替换本次打开文件的操作表：

```
file->f_op = driver_fops;
```

因此 /dev/console 的完整路径是：

```
open("/dev/console")
  → devfs_iops.lookup
  → inode(i_rdev=1:0, i_fop=def_chr_fops)
  → file->f_op = inode->i_fop
  → def_chr_fops.open
  → chrdev_find(1:0)
  → file->f_op = console_fops

read(fd)
  → file->f_op->read
  → console_read
  → t113_uart_read
```

GPIO 按键同样执行：

```
/dev/key0 → 13:0 → gpio_keys_fops → gpio_keys_read
```

devfs 不再提供 devfs_read() 或 devfs_write()，也不接触 UART/GPIO 寄存器。

9.5 路径查找

lookup_path() 先以最长前缀选择挂载，再从该挂载的根 dentry 开始逐分量查找：

```
/bin/ls
  → minifs root inode
  → i_op->lookup(parent inode, child dentry "bin")
  → bin inode
  → i_op->lookup(bin inode, child dentry "ls")
  → ls inode
```

文件系统的 lookup() 调用 t113_vfs_bind_inode()，把查找结果绑定到 dentry，并设置该 inode 的 i_op/i_fop/i_private。inode cache 使用 (super_block, i_ino) 作为键。

目前 devfs 直接使用 `child->d_name`；minifs 和 FAT32 虽然已经原生实现 `i_op->lookup`，内部仍利用 `child->d_path` 复用现有的完整路径解析器。后续可以把这两个解析器继续收窄为“父 inode + 单个分量”。

9.6 挂载

```
int t113_vfs_mount_fs(const char *path,
                      uint32_t magic,
                      void *fs_info,
                      const struct super_operations *s_op,
                      const struct inode_operations *root_iop,
                      const struct file_operations *root_fop);
```

当前挂载：

```
/       minifs，SD 第二分区，只读
/boot   FAT32，SD 第一分区，读写；失败时可退回 bootfs
/dev    devfs，动态字符设备节点
/sys    sysfs，只读投影 driver core 的 kobject/device/bus/driver
```

挂载根 inode 直接使用文件系统传入的 `root_iop/root_fop`。FAT32 通过 `super_operations.evict_inode` 在最后一个 VFS inode 引用释放时写回目录项并释放 `fat_inode` 引用。

9.7 与 Linux 的关系

当前主要派发方式已经与 Linux 一致：

```
路径操作        inode->i_op
普通文件 I/O    file->f_op → 文件系统 fops
字符设备 I/O    file->f_op → 驱动 fops
设备选择        inode->i_rdev → chrdev table
```

miniOS 仍然比 Linux 简化：对象来自静态池，没有 RCU、页缓存、完整权限模型、负 dentry、umount 和并发 rename；挂载切换仍先用最长字符串前缀选挂载，而不是在逐级 path walk 中穿越 mountpoint。

9.8 `vfs.h` 当前保留的重要结构体

`vfs.h` 是文件系统、devfs 和设备驱动共同依赖的公开接口。当前重要结构体可以分成“操作表”“VFS 对象”和“目录项输出”三组。

9.8.1 操作表

```
super_operations
+-- put_super(sb)        卸载或释放文件系统实例（目前尚未实现 umount）
+-- evict_inode(inode)   VFS inode 最后一个引用消失时释放文件系统私有对象

inode_operations
+-- lookup(dir, child)   在目录中查找一个名字，并给 child 绑定 inode
+-- create(dir, child)   创建普通文件并绑定 inode
+-- truncate(inode)      截断文件
```

```
file_operations
+-- open(inode, file)   初始化本次打开；也可以替换 file->f_op
+-- read(file, ...)
+-- write(file, ...)
+-- readdir(inode, path, index, entry)
+-- release(inode, file)
```

inode_operations 操作的是文件系统名字空间和文件元数据；file_operations 操作的是一次已经打开的文件。当前 readdir() 仍以 inode 而不是打开的目录 file 为参数，这是 miniOS 的简化，并非完整 Linux 接口。

9.8.2 VFS 对象

```
vfs_mount
+-- path                挂载路径，例如 /、/boot、/dev
+-- sb ------------------------------+
+-- mnt_root ----------------+       |
+-- parent                   |       |
+-- mountpoint               |       |
                             v       v
                          dentry    super_block
                          +-- d_name   +-- s_magic
                          +-- d_path   +-- s_op
                          +-- d_parent +-- s_root --> 根 dentry
                          +-- d_sb ----+
                          +-- d_inode  +-- s_fs_info
                                 |
                                 v
                               inode
                               +-- i_ino / i_size / i_type
                               +-- i_rdev
                               +-- i_sb
                               +-- i_op
                               +-- i_fop
                               +-- i_private

进程 fd 表 --> file
               +-- f_dentry --> dentry
               +-- f_inode  --> inode
               +-- f_op
               +-- f_pos / f_pos_lock
               +-- private_data
```

各对象分别回答不同问题：

对象	表示什么	关键关系
super_block	一次文件系统实例	s_root 指向根 dentry；s_fs_info 保存 minifs/FAT32/devfs 的实例数据
dentry	路径中的一个名字	d_parent 构成目录层次，d_inode 指向名字对应的 inode
inode	文件、目录或设备节点本身	i_op 管路径/元数据，i_fop 是默认打开操作
file	一次 open() 形成的打开文件描述	f_op 是本次打开最终使用的操作表；f_pos 是共享偏移
vfs_mount	把一个 super block 接到路径树上	保存挂载路径、父挂载和挂载点
vfs_dirent	readdir() 返回给调用者的一项	只是结果结构，不参与缓存和引用关系

三个私有指针不要混淆：

```
super_block.s_fs_info  每次挂载/文件系统实例的私有数据
inode.i_private        某个 inode 的文件系统私有数据，例如 FAT inode
file.private_data      某次 open 的私有数据，例如设备会话状态
```

9.8.3 引用和共享关系

当前对象来自 vfs.c 的固定大小静态池：8 个 mount、8 个 super block、128 个 inode、128 个 dentry 和 32 个 file。它们不是动态分配的。

```
mount 持有 super_block 和根 dentry
dentry 持有 inode，并持有父 dentry
file 持有 dentry；通过 dentry 间接保持 inode 有效
进程 files[fd] 持有 file
dup()/fork() 增加同一个 file->references
```

因此 dup() 或 fork() 后多个 fd/进程共享同一个 struct file，也共享 f_pos；普通文件用 f_pos_lock 串行化 I/O 和偏移更新。字符设备不使用文件位置，所以 t113_vfs_read/write() 不为字符设备获取 f_pos_lock。

vfs_lock 只保护对象池、引用计数和 file 有效性检查。读写时 VFS 先在锁内临时增加 file->references，随后释放全局锁，再调用 file->f_op->read/write()；这样 UART 阻塞读取时不会长期占用全局 VFS 锁。

9.9 chrdev.h 当前保留的内容

chrdev.h 刻意保持很薄，目前没有公开 struct cdev 或 struct chrdev_entry。它只包含：

```
MINIOS_MKDEV(major, minor)  /* 组合 16 位 major 和 16 位 minor */
MINIOS_MAJOR(dev)
MINIOS_MINOR(dev)

void t113_chrdev_init(void);
int t113_chrdev_register(uint16_t major, uint16_t minor,
                         const struct file_operations *fops);
const struct file_operations *t113_chrdev_inode_fops(void);
```

真正的注册表结构是 chrdev.c 的私有实现：

```
struct chrdev_entry {
    uint32_t dev;                       /* 完整 major:minor */
```

```
    const struct file_operations *fops;  /* 具体设备驱动操作 */
    uint32_t used;
};

static struct chrdev_entry chrdev_table[32];
```

这种封装意味着 VFS、devfs 和驱动都不能直接遍历或修改全局字符设备表，只能调用注册接口。chrdev_lock 保护注册和查找。当前没有注销接口、设备号范围、动态 major、struct cdev、引用计数或驱动卸载，因此它更接近 Linux 2.4 的简单注册表，而不是 Linux 2.6 以后的 cdev_map。

9.10 vfs.h 与 chrdev.h 的关系

chrdev.h 依赖 vfs.h，因为字符设备最终注册的是 VFS 定义的 struct file_operations：

```
具体驱动
  +-- console_fops / gpio_keys_fops
          |
          | t113_chrdev_register(major, minor, fops)
          v
chrdev_table: dev_t -> driver fops

devfs
  +-- 名字 -> dev_t
      例如 "console" -> 1:0

VFS inode
  +-- i_type = VFS_NODE_CHAR
  +-- i_rdev = 1:0
  +-- i_fop = def_chr_fops
                    | open
                    v
              chrdev_open()
                    | 根据 i_rdev 查 chrdev_table
                    v
              file->f_op = console_fops
```

这里实际存在两张彼此独立的表：

表	保存的映射	负责什么
devfs nodes[]	"console" -> (1,0)	路径名字和设备号的关系
chrdev chrdev_table[]	(1,0) -> console_fops	设备号和驱动操作的关系

t113_devfs_create_chrdev("console", 1, 0) 只创建第一种映射；t113_chrdev_register(1, 0, &console_fops) 只创建第二种映射。两步都成功后，/dev/console 才能正常打开并到达 UART 驱动。

9.11 从系统调用到对象的完整关系

以 read(fd, buffer, count) 为例：

```
用户 fd（整数）
  -> current->files[fd]            struct file *
  -> t113_vfs_read(file, ...)
```

```
      +-- 临时增加 file 引用
      +-- 普通文件：锁住 file->f_pos_lock
      +-- file->f_op->read(file, ...)
            +-- 普通文件：minifs/FAT32 fops
            |      -> file->f_inode
            |      -> inode->i_private / inode->i_sb->s_fs_info
            |      -> SD 块设备
            +-- 字符设备：console_fops
                  -> UART RX 缓冲区/寄存器
```

用户态的 fd 只是当前进程 fd 表的数组下标；内核 `struct file` 才是打开文件对象。它既不是磁盘上的文件，也不是 inode。多个 fd 可以指向同一个 file，一个 inode 也可以同时对应多个相互独立、偏移不同的 file。

9.12 sysfs：把 driver core 投影到 /sys

目录如何生成、属性怎样由驱动提供、与 Linux sysfs 的差别，详见第 10 章。

`CONFIG_SYSFS=y` 时，`kernel_boot()` 在 driver core 完成设备枚举与驱动绑定后调用 `t113_sysfs_init()`。根 minifs 只提供空的 `/sys` 挂载点，实际目录和属性由 `sysfs.c` 根据当前内存对象动态生成，不写入 SD 卡。

```
/sys
+-- devices                          按 device.parent 展开的设备拓扑
|   +-- platform
|       +-- gic
|       +-- uart3
|       +-- timer0
|       +-- user-key
|       +-- twi2
|       +-- sdc1
|           +-- mmc1:0001:1
+-- bus                              按 bus_type 分类的视图
    +-- platform
    |   +-- devices
    |   +-- drivers
    +-- i2c
    |   +-- devices
    |   +-- drivers
    +-- sdio
        +-- devices
        +-- drivers
```

设备目录公开 `name`、`bus`、`driver`、`path` 四个只读属性；驱动目录公开 `name` 与 `bound`。例如：

```
ls /sys/devices/platform
cat /sys/devices/platform/user-key/driver
cat /sys/devices/platform/user-key/value
cat /sys/devices/platform/user-key/keycode
cat /sys/bus/platform/drivers/gpio-keys/bound
cat /sys/bus/sdio/devices/mmc1:0001:1/path
```

当前 VFS 没有符号链接 inode，因此 `/sys/bus/*/devices/*` 使用可浏览的镜像目录，其 `path` 属性指回 `/devices/...`；这与 Linux sysfs 使用符号链接表达同一关系在语义上相同，但表示形式更简单。属性在每次 `read()` 时实时读取 driver core，后续注册的设备也会出现在目录中。

9.13 GPIO 按键

probe、中断、事件队列、寄存器和已知问题详见第 10 章。

100ASK T113-S3 RESET 旁的 USER KEY 已作为 platform device `user-key` 注册：

```
compatible  gpio-keys
GPIO        PB4
有效电平    低
上下拉      内部上拉
中断        GPIOB，双边沿
消抖        20 ms
键值        KEY_ENTER (28)
```

`gpio_keys` 驱动在 probe 中申请 GPIOB IRQ，建立 16 项事件环形队列和等待队列，并注册字符设备 (13,0) 与 devfs 节点 `/dev/key0`。按下和释放分别产生 `value=1`、`value=0` 的 `input_event`；队列为空时读取进程睡眠，由 IRQ 入队后唤醒。用户态可运行：

```
keytest
```

驱动模型与设备 I/O 是两条互补路径：

```
/sys/devices/platform/user-key   查看设备、总线和绑定驱动
/dev/key0                        阻塞读取实际按键事件
```

9.14 已知限制（主机侧测试结果）

把 `vfs.c`、minifs、FAT32、bio、devfs、chrdev 用主机 gcc（ASan/UBSan）编进一个测试程序，用 `make t113` 生成的 `minifs.img` 和一个 FAT32 分区拼成 SD 卡镜像，按 `kernel_boot()` 的顺序挂载后测试。下面这些都通过：

- 挂载 `/`、`/boot`、`/dev`；读 `/init`、`/bin/sh`、`/bin/init`（硬链接）、`/boot/sub/deep.txt`，内容与主机文件一致；
- `readdir` 列出 `/`、`/bin`、`/boot`、`/dev`；
- `/dev/console` 经 `chrdev_open()` 切到驱动 fops，读写成功，`f_pos` 保持 0；
- FAT32 的 `O_CREATE`、写入、重新打开、`O_TRUNC`；
- 同一文件打开两次共享一个 dentry 和 inode；
- 反复打开/关闭 500 次后 dentry、inode、file 槽位数回到开机时的值，FAT32 的 16 个 `fat_inode` 槽位也没有耗尽；没有锁重入。

发现的问题：

1. **路径中间是非目录时没有拒绝。** `lookup_path()` 不检查 `current` 是否为目录，直接调用它的 `i_op->lookup`。minifs 和 FAT32 用完整路径解析，会自己失败；devfs 只比较 `child->d_name`，所以 `/dev/console/console` 能打开控制台。在 `lookup_path()` 调 `lookup` 之前加一次 `i_type == VFS_NODE_DIR` 检查即可，这对应 Linux 的 `-ENOTDIR`。
2. **只写的字符设备打不开。** `t113_chrdev_register()` 允许只有 `write` 的 fops，但 `t113_vfs_open()` 总是要求 `f_op->read`。现在只有 `RDONLY/RDWR` 两种模式，没有 `O_WRONLY`，所以这种设备任何模式都打不开。
3. `alloc_inode()` 命中缓存时传给 `evict_inode()` 的临时 `struct inode` 只填了部分字段（`i_op`、`i_fop` 未初始化）。FAT32 只读 `i_private`，现在没问题，但新文件系统的 `evict_inode` 不能依赖其他字段。
4. bootfs 的目录 fops 没有 `readdir`，退回 bootfs 时 `ls /boot` 会失败。
5. 挂载根 inode 的 `i_ino` 固定为 0，minifs 真正的根是 1；`/bin/..` 会得到另一个 `i_ino = 1` 的 inode。只读

情况下没有影响。

第十章 sysfs 与 GPIO 按键驱动

本章按当前代码说明两件新加的东西：/sys 怎样把内存里的 driver core（kobject、device、bus、driver）投影成目录和只读文件；100ASK T113-S3 上 RESET 旁的 USER KEY 怎样经 gpio-keys 驱动变成 /dev/key0 的按键事件，以及它在 /sys 里的属性。

文件	内容
src/kernel/fs/sysfs.c	sysfs 文件系统：lookup、readdir、read
src/include/fs/sysfs.h	t113_sysfs_init()
src/fs/Kconfig	CONFIG_SYSFS，T113 板默认打开
src/include/t113/device.h	新增 struct device_attribute，struct device 加 attributes/num_attributes
src/boards/100ask_t113/device.c	解绑或 probe 失败时清空属性表
src/drivers/input/gpio_keys.c	按键驱动
src/include/input/gpio_keys.h	平台数据和 struct minios_input_event
src/boards/100ask_t113/board.c	user-key 平台设备，PB4
src/boards/100ask_t113/gpio.c	GPIO 外部中断寄存器操作
user-t113/keytest.c、user-t113/include/input.h	用户态测试程序和事件结构
tools/mkfs_minifs.c	在根文件系统里建空目录 /sys 作为挂载点

10.1 sysfs

10.1.1 挂载时机

```
kernel_boot()                                  kernel/init/main.c
+-- t113_minifs_mount_rootdev()      /
+-- t113_fat32_mount_vfs()           /boot
+-- t113_chrdev_init()
+-- t113_devfs_init()                /dev
+-- ...
+-- t113_driver_core_init()          建 devices/bus/drivers 三个 kset，注册设备和驱动，扫 MMC
+-- t113_sysfs_init()                /sys，失败 → PANIC "cannot mount sysfs at /sys"
    +-- t113_vfs_mount_fs("/sys", 0x62656572, 0, 0, &sysfs_iops, &sysfs_dir_fops)
```

0x62656572 是 Linux 的 SYSFS_MAGIC（ASCII "beer"）。sysfs 不保存任何数据：super_block 没有 s_fs_info，也没有 super_operations。每次 lookup、readdir、read 都直接去查 driver core 当时的状态，所以先挂后挂都可以；放在 t113_driver_core_init() 之后只是为了让挂载时树已经完整。

mount 时 VFS 要先在 minifs 里找到 /sys 目录，所以 mkfs_minifs 新建了一个空的 sys 目录。旧的 minifs.img 没有它，必须重新 make t113 并烧写分区 2，否则启动会停在上面的 PANIC。

10.1.2 目录结构

sysfs 只认两棵子树：

```
/sys
+-- devices/                          按 device.parent 展开的设备拓扑
|   +-- <dev>/[<child>/...]
|       +-- name   bus   driver   path       每个设备都有的 4 个属性
|       +-- <驱动提供的属性>                   device->attributes[]
+-- bus/
```

```
+-- <bus>/
    +-- name
    +-- devices/<dev>/                  与 /sys/devices 下同一设备的属性相同
    +-- drivers/<drv>/
        +-- name
        +-- bound                       绑定到这个驱动的设备名，空格分隔
```

下面是主机侧测试的实际输出（见第 10.3 节；注册了 uart3、timer0、user-key 三个平台设备和 t113-uart、gpio-keys 两个驱动）。板子上还会有 gic、twi2、sdc1、csi0，以及 i2c、sdio 两条总线和它们下面的设备。

```
/sys/
  devices/
    platform/
      uart3/
        name   = "uart3"
        bus    = "platform"
        driver   = "t113-uart"
        path   = "/devices/platform/uart3"
      timer0/
        name   = "timer0"
        bus    = "platform"
        driver   = "(none)"
        path   = "/devices/platform/timer0"
      user-key/
        name   = "user-key"
        bus    = "platform"
        driver   = "gpio-keys"
        path   = "/devices/platform/user-key"
        value    = "0"
        keycode    = "28"
      name   = "platform"
      bus    = "(none)"
      driver   = "(none)"
      path   = "/devices/platform"
  bus/
    platform/
      devices/
        uart3/ ...  timer0/ ...  user-key/ ...      内容同上
      drivers/
        t113-uart/
          name   = "t113-uart"
          bound    = "uart3"
        gpio-keys/
          name   = "gpio-keys"
          bound    = "user-key"
      name   = "platform"
```

/sys/devices/platform 自己也是一个 struct device（platform_bus），所以它下面既有子设备目录，也有 4 个属性文件。

10.1.3 路径怎样对应到 driver core

kobject 的路径由 kobject_get_path() 沿 parent 链拼出。device_add() 把每个设备加进 devices kset；没有 parent 的 kobject 以 kset 自己的 kobject 为 parent，所以平台设备的路径是 /devices/platform/<name>

(platform_device_register() 把 dev.parent 设成 platform_bus)。

sysfs 把 /sys/devices/a/b 去掉 /sys 后得到 /devices/a/b，再在 devices kset 里找路径相同的 kobject：

```
split_path("/sys/devices/platform/user-key/value")
  → part[] = { "devices", "platform", "user-key", "value" }，最多 8 级，每级 ⩽ 30 字节
sysfs_path_type()
  +-- device_model_path(全部 4 级) = "/devices/platform/user-key/value"
  |   find_device_path() → 没有这个设备
  +-- device_model_path(前 3 级)   = "/devices/platform/user-key"
  |   find_device_path() → user-key 的 struct device (get_device 加引用)
  +-- 最后一级 "value"：不是 name/bus/driver/path，查 device->attributes[] → 找到
  +-- 返回 VFS_NODE_FILE，put_device()
```

/sys/bus/... 则按层数解析：

路径	类型	查法
/sys/bus	目录	固定
/sys/bus/<bus>	目录	kset_find_obj(buses, name)
/sys/bus/<bus>/name	文件	bus->name
/sys/bus/<bus>/devices、drivers	目录	固定
/sys/bus/<bus>/devices/<dev>	目录	bus_find_device_by_name()
/sys/bus/<bus>/devices/<dev>/<attr>	文件	同上，再查属性名
/sys/bus/<bus>/drivers/<drv>	目录	bus_for_each_drv() 按名字找
/sys/bus/<bus>/drivers/<drv>/name、bound	文件	同上

10.1.4 三个 VFS 操作

sysfs 只有一张 inode_operations 和两张 file_operations：

```
static const struct inode_operations sysfs_iops = { .lookup = sysfs_lookup };
static const struct file_operations sysfs_dir_fops = { .readdir = sysfs_readdir };
static const struct file_operations sysfs_file_fops = { .read = sysfs_read };
```

lookup。 sysfs_lookup(dir, child) 用 child->d_path 跑一遍 sysfs_path_type()，存在就 t113_vfs_bind_inode()：i_ino 是整条路径的 32 位 FNV-1a 哈希，i_size = 0，目录配 sysfs_dir_fops，文件配 sysfs_file_fops。

readdir。 sysfs_readdir(inode, path, index, entry) 现算第 index 项：

```
/sys                      index 0 = devices, 1 = bus
/sys/devices[/...]        先列 parent 等于当前设备的子设备 (kset_for_each)，
                          再列 4 个公共属性和 device->attributes[]
/sys/bus                  buses kset 里的第 index 个 bus
/sys/bus/<bus>            devices、drivers、name
/sys/bus/<bus>/devices    bus_for_each_dev() 的第 index 个设备
/sys/bus/<bus>/drivers    bus_for_each_drv() 的第 index 个驱动
/sys/bus/<bus>/devices/<d>  4 个公共属性 + 驱动属性
/sys/bus/<bus>/drivers/<d>  name、bound
```

read。 sysfs_read(file, buf, n) 每次调用都按 file->f_dentry->d_path 重新生成整个值（最多 256 字节），再从 f_pos 处拷贝。读到末尾返回 0，所以 cat 一类的循环读能正常结束。

```
sysfs_read()
+-- sysfs_generate(path)
```

```
    +-- /sys/devices/.../<attr>             show_device(device, attr)
    +-- /sys/bus/<bus>/name                 bus->name + "\n"
    +-- /sys/bus/<bus>/devices/<d>/<attr>   show_device()
    +-- /sys/bus/<bus>/drivers/<d>/<attr>   show_driver(): name, 或 bound 列表
show_device()
+-- name / bus / driver                     字符串 + "\n", 没有时写 "(none)"
+-- path                                    kobject_get_path() + "\n"
+-- 其他                                    device->attributes[i].show(dev, buf, cap)
```

sysfs 没有 write，t113_vfs_open() 对 O_RDWR 直接拒绝，所以 /sys 是只读的。

10.1.5 驱动怎样加属性

device.h 新增：

```
struct device_attribute {
    const char *name;
    int (*show)(struct device *dev, char *buffer, uint32_t capacity);
};

struct device {
    ...
    const struct device_attribute *attributes;   /* 驱动 probe 时设置 */
    uint32_t num_attributes;
};
```

驱动在 probe 成功前把静态数组挂到 dev- >attributes。show 返回写入的字节数，出错返回负数。driver core 在三处把它清零：device_initialize()、probe 失败的 really_probe()、解绑的 device_release_driver_locked()。属性的生命周期因此和“设备绑定了这个驱动”一致，解绑后 /sys 里不会留下指向已释放驱动数据的文件。

10.1.6 引用和锁

- lookup 和 readdir 在 VFS 的 vfs_lock 里运行（由 lookup_path() 和 t113_vfs_readdir() 调用）；read 不在这把锁里，t113_vfs_read() 调驱动前已经放开了它。
- 找到的 device、bus_type、device_driver 都先加 kobject 引用（get_device、kset_find_obj、kobject_get），用完马上放掉，不在 inode 里长期持有。所以设备删除后，下一次访问就会看到它不存在了。
- 遍历时持有的锁是 kset 的 spinlock 或 driver core 的 core_lock，回调里只拿 kobject 自己的 ref.lock，没有同一把锁重入。
- 驱动的 show 由 sysfs_read() 调用，在 vfs_lock 之外。readdir 不调用 show。gpio_keys 的 show 只拿一把 spinlock。

10.1.7 与 Linux sysfs 的差别

方面	Linux	miniOS
实现	kernfs：每个 kobject、属性在注册时建 kernfs_node	不建节点，按路径实时查 driver core
设备属性	DEVICE_ATTR_RO() + sysfs_create_group()，可读写	dev->attributes[]，只读
driver、subsystem	符号链接	driver、bus 是文本章件
/sys/bus/*/devices/*	指向 /sys/devices/... 的符号链接	同样内容的目录，path 属性写真实路径
驱动目录	每个设备一个符号链接，加 bind、unbind、uevent	name 和 bound 两个文本章件
/sys/class、/sys/module、uevent	有	没有
inode 号	kernfs 分配，唯一	路径的 32 位哈希

10.2 GPIO 按键驱动

10.2.1 硬件和平台数据

USER KEY 接在 PB4，按下接地，平时靠内部上拉保持高电平。board.c 在 CONFIG_INPUT_GPIO_KEYS=y 时注册平台设备：

```
static const struct resource user_key_resources[] = {
    DEFINE_RES_IRQ(T113_IRQ_GPIOB),             /* 101 = GIC SPI 69 */
};
static const struct gpio_keys_platform_data user_key_pdata = {
    .gpio = T113_GPIO(T113_PB, 4),              /* 1 * 32 + 4 = 36 */
    .irq = T113_IRQ_GPIOB,
    .code = MINIOS_KEY_ENTER,                   /* 28, 与 Linux KEY_ENTER 相同 */
    .active_low = 1u,
    .pull = T113_PULL_UP,
    .debounce_ms = 20u,
};
static struct platform_device user_key_device = {
    .name = "user-key",
    .compatible = "gpio-keys",
    ...
};
```

gpio_keys_driver 用 of_match_table = { "gpio-keys" } 与它匹配，驱动名是 gpio-keys。

用到的 PIO 寄存器（T113_PIO_BASE = 0x02000000，每个端口 0x30 字节，外部中断寄存器从 0x200 开始，每个端口 0x20 字节，PB 是第一组）：

寄存器	地址	PB4 的位	写入值
PB_CFG0	0x02000030	[19:16]	6 = T113_GPIO_IRQ（PB_EINT4）
PB_DAT	0x02000040	[4]	读电平
PB_PULL0	0x02000054	[9:8]	1 = 上拉
PB_EINT_CFG0	0x02000200	[19:16]	4 = 双边沿
PB_EINT_CTL	0x02000210	[4]	1 = 使能
PB_EINT_STATUS	0x02000214	[4]	读 pending，写 1 清除
PB_EINT_DEB	0x02000218	整个端口	0x71

10.2.2 probe

```
driver_register(gpio-keys) → platform 总线 match → really_probe()
+-- gpio_keys_probe(pdev)
    +-- pdata = dev.platform_data; platform_get_irq(pdev, 0) 必须等于 pdata->irq
    +-- key0 = { head = tail = 0, last_irq_us = 0 }, 初始化 spinlock 和等待队列
    +-- t113_gpio_set_pull(PB, 4, 上拉)
    +-- t113_gpio_direction_input(PB4); 读一次电平作为初始 value
    +-- t113_chrdev_register(13, 0, &gpio_keys_fops)       设备号 → 驱动 fops
    +-- t113_devfs_create_chrdev("key0", 13, 0)             /dev/key0 → 设备号
    +-- t113_request_irq(101, gpio_keys_irq, &key0)
    +-- t113_gpio_irq_config(PB4, BOTH)
    |   +-- 关 PB4 中断，写 EINT_CFG0，写 EINT_DEB = 0x71
    |   +-- PB_CFG0 切到 EINT 功能
    |   +-- 清掉 pending
    +-- platform_set_drvdata(pdev, &key0)
    +-- dev.attributes = { value, keycode }                 /sys 里的两个属性
    +-- t113_gpio_irq_enable(PB4); t113_gic_enable_irq(101)
    +-- "user-key: PB4 active-low key -> /dev/key0 irq 0x65"
```

major 13 沿用 Linux 输入子系统的主设备号（INPUT_MAJOR）。驱动状态是一个静态 key0，所以现在只支持一个按键。

10.2.3 中断路径

```
PB4 电平变化
+-- GIC SPI 69 → IRQ 101 → 异常向量 → t113_irq_dispatch(101)
    +-- gpio_keys_irq(irq, &key0)
        +-- PB_EINT_STATUS 第 4 位没置 → 返回（GPIOB 整组共用这一个中断）
        +-- 写 1 清 pending
        +-- now = t113_time_us(); 读电平; active_low 取反得到 value
        +-- value 与上次相同，或距上次被接受的事件不到 20 ms → 丢弃
        +-- spin_lock_irqsave(&key0.lock)
        |   +-- last_irq_us = now; key0.value = value
        |   +-- 队列没满: events[head] = { now/1000, EV_KEY, 28, value }, head++
        +-- spin_unlock_irqrestore
        +-- wake_up_all(&key0.wait)
```

事件队列是 16 项的环形缓冲区，head == tail 为空，head + 1 == tail 为满，满了新事件直接丢掉（实际能存 15 个）。

10.2.4 阻塞读

```
read(fd, &ev, sizeof ev)                    keytest
+-- sys_read → t113_vfs_read(file)
    +-- file->f_op->read = gpio_keys_read   (chrdev_open 已把 f_op 换成驱动 fops)
        +-- count < 12 → -1
        +-- 循环:
            +-- spin_lock_irqsave(&key0.wait.lock); spin_lock(&key0.lock)
            +-- 队列非空 → 取出一个事件，解锁，拷给用户，返回 12
            +-- 队列空   → 解开 key0.lock，wait_queue_sleep_locked()
                           (入等待队列、解开 wait.lock、t113_sched_yield())
```

先拿等待队列的锁、关着中断检查队列，再入队睡眠，所以“检查为空”和“开始睡眠”之间中断插不进来，不会丢唤醒。字符设备不走 `f_pos_lock`，所以一个进程阻塞在/dev/key0 上不会挡住别的 fd。一次 read 只返回一个事件。

事件结构（内核 `struct minios_input_event` 与用户态 `struct input_event` 布局相同，12 字节）：

```
struct input_event {
    uint32_t time_ms;     /* t113_time_us() / 1000 */
    uint16_t type;        /* EV_KEY = 1 */
    uint16_t code;        /* KEY_ENTER = 28 */
    int32_t value;        /* 1 按下，0 松开 */
};
```

10.2.5 sysfs 属性

文件	show	内容
/sys/devices/platform/user-key/value	gpio_keys_show_value()	当前状态 0/1，在 key0.lock 下读
/sys/devices/platform/user-key/keycode	gpio_keys_show_keycode()	28

同样的两个文件也出现在 /sys/bus/platform/devices/user-key/ 下。

```
/sys/devices/platform/user-key   看设备、总线、绑定的驱动和当前状态
/dev/key0                        阻塞读按键事件
```

10.2.6 用户态

下面的输出是示例，时间值随实际按键变化：

```
$ keytest
keytest: press the USER key beside RESET
key code=28 pressed time=12345 ms
key code=28 released time=12456 ms
$ cat /sys/devices/platform/user-key/value
0
$ cat /sys/bus/platform/drivers/gpio-keys/bound
user-key
```

10.2.7 与 Linux gpio-keys 的差别

方面	Linux	miniOS
设备描述	设备树 gpio-keys 节点，每个子节点一个键	board.c 里的平台数据，只有一个键
设备文件	输入子系统分配 /dev/input/eventN（major 13）	固定 /dev/key0（13:0）
事件	struct input_event { timeval, type, code, value }，每次变化后跟 EV_SYN	12 字节，毫秒时间，没有 EV_SYN
消抖	gpiod_set_debounce()，否则延迟工作队列在消抖时间后重新采样	在中断里比较时间戳，窗口内的边沿直接丢
读	一次可读多个事件，支持 O_NONBLOCK、poll	一次一个，只能阻塞

10.3 测试和发现的问题

make t113 在 -Wall -Wextra 下没有警告。另外把 vfs.c、sysfs.c、kobject.c、device.c、platform_bus.c、gpio_keys.c 用主机 gcc（ASan/ UBSan）编在一起，GPIO、IRQ、时钟用桩函数代替，按板子的顺序注册设备和驱动后：

- 递归 readdir + read 整个 /sys，得到第 10.1.2 节的树，每个属性都能读；
- /sys/nope、/sys/devices/platform/uart3/value 打不开，O_RDWR 打开属性被拒；
- 结束时 VFS 只剩开机时的 3 个 dentry，没有遗留的 file；
- 模拟按键：拉低 PB4 并触发中断后 value 从 0 变成 1。

发现的问题：

1. **消抖会丢掉最后一个边沿，按键可能卡在“按下”**。中断里只要距上次被接受的事件不到 20 ms 就丢弃，之后没有任何东西再采样一次电平。测试中按下后 5 ms 松开，松开的中断被丢掉，200 ms 后 value 仍是 1，/dev/key0 也收不到松开事件，直到下一次按键。Linux gpio-keys 的做法是在中断里只启动一个消抖定时器，定时器到期时再读电平上报。
2. **PB_EINT_DEB = 0x71 和注释不一致（推断，需对照 T113 手册确认）**。注释写的是“32 kHz 时钟、/128”。按 Allwinner sun8i 系列手册，这个寄存器的 bit0 是时钟选择，0 为 32 kHz LOSC、1 为 24 MHz HOSC，bit[6:4] 是 2^n 分频。照此 0x71 选的是 24 MHz/128，硬件消抖只有几微秒；要得到注释里的效果应写 0x70。
3. probe 中途失败时，已经注册的字符设备号、devfs 名字和 IRQ 都没有撤销（现在也没有注销接口），重试 probe 会因为重复注册失败。
4. sysfs 的 inode 号是路径哈希。两条不同路径哈希相同时，VFS 的 inode 缓存会把第二条当成第一条，类型和 fops 可能用错。现在的路径只有几十条，碰撞概率很低。